

Machine Learning desde cero

Apuntes completos

31 apuntes escritos mientras se aprendía, sobre una planta real

De “¿qué es un modelo?” hasta la recomendación final de **no** desplegar ML.

Construido sobre 737.453 filas de una planta de flotación inversa de hierro (marzo–septiembre 2017), con los tropiezos incluidos: enseñan más que los aciertos.

Biblioteca de Kevin · predecir el % de SiO_2 del concentrado

Dataset: *Quality Prediction in a Mining Process* (Kaggle)

Compilado el 2026-08-05

Índice

Módulo 0 — Conceptos (prólogo, sin código)	1
Módulo 1 — El dato: análisis exploratorio	10
Apunte 01 — Cargar datos sin que te mientan	10
Apunte 02 — Estadística descriptiva: la radiografía del dataset	12
Apunte 03 — Series de tiempo: las cicatrices del dato real	14
Apunte 04 — Correlación: cuando “moverse juntos” te engaña	17
Apunte 05 — Fuga de datos: las tres formas de hacer trampa sin querer	20
Apunte 06 — Radiografía del dataset	22
Módulo 2 — Regresión: árbol y random forest	25
Apunte 07 — Tu primer modelo: el árbol de decisión	25
Apunte 08 — ¿Cómo se califica una predicción? (RMSE, MAE, R^2)	29
Apunte 09 — Sobreajuste, en vivo con tus datos	32
Apunte 10 — Random forest: el comité de árboles	34
Apunte 11 — ¿Qué variables importan? (feature importance)	36
Apunte 12 — Qué aprendió el modelo (cierre del Módulo 2)	38
Módulo 3 — Cómo aprenden los modelos	40
Apunte 13 — Regresión lineal (y por qué empezar por aquí)	40
Apunte 14 — Descenso de gradiente: el motor de todo	43
Apunte 15 — Escalar las variables y regularizar	46
Módulo 4 — Preprocesamiento y validación	48
Apunte 16 — Pipelines: preprocesar sin fugas	48
Apunte 17 — Validación cruzada: muchos exámenes en vez de uno	50
Apunte 18 — Ajustar las perillas (hiperparámetros) sin trampa	52
Módulo 5 — Ingeniería de variables y boosting	54
Apunte 19 — Ingeniería de variables: el salto real	54

Apunte 20 — Gradient boosting: el circuito que limpia lo que falta	56
Módulo 6 — Clasificación	58
Apunte 21 — De regresión a clasificación	58
Apunte 22 — Matriz de confusión, precisión y recall	60
Módulo 7 — Interpretabilidad	62
Apunte 23 — Importancia honesta... y el baseline correcto	62
Apunte 24 — SHAP: el porqué de cada predicción	65
Módulo 8 — ¿Dónde SÍ aporta el machine learning?	68
Apunte 25 — ¿Dónde SÍ aporta el machine learning?	68
Módulo 9 — Aprendizaje no supervisado	71
Apunte 26 — k-means: los estados operativos de tu planta	71
Apunte 27 — PCA: ver en 2D lo que vive en 21 dimensiones	74
Módulo 10 — Redes, producción y cierre	76
Apunte 28 — Redes neuronales (y por qué aquí pierden)	76
Apunte 29 — De modelo a producto: desplegar, monitorear, re-entrenar	79
Apunte 30 — Cierre: lo que aprendiste y cómo contarlo	81
Extras — fuera del temario principal	84
Apunte 31 (extra) — PyTorch y tu GPU: el bucle a mano	84

Machine Learning desde cero

Módulo 0: los conceptos, explicados para un metalurgista · sin código todavía

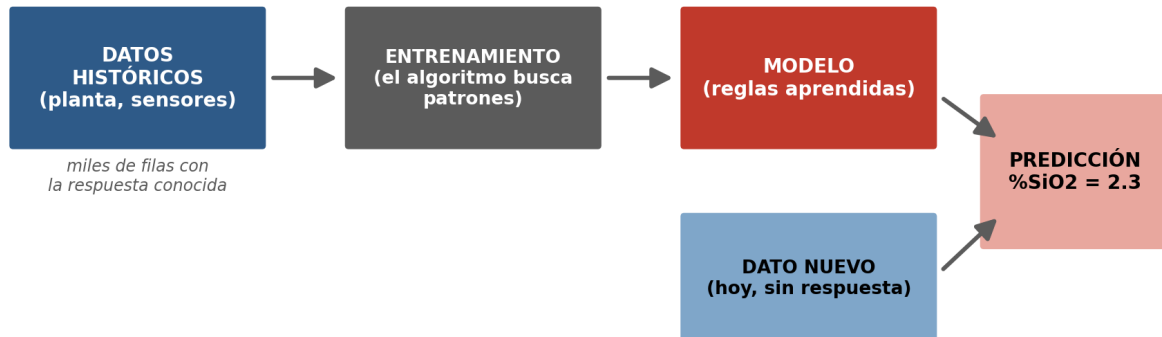
La idea de este documento

Antes de tocar Python vas a entender **qué** hace cada algoritmo y **por qué**, usando ejemplos del proceso que ya dominas: la flotación de hierro. Cuando en el Módulo 1 descarguemos el dataset real de la planta (737.000 filas, 24 variables), cada línea de código que escribamos va a corresponder a una figura de este documento. Nada será magia.

El problema que vamos a resolver todo el curso: en una planta de flotación, el % de SiO_2 del concentrado se mide en laboratorio y el resultado tarda una hora en llegar. Una hora operando a ciegas. Queremos un modelo que **prediga esa medición en tiempo real** a partir de las variables que sí se leen al instante: pH, flujos de amina y almidón, densidad de pulpa, aire de las columnas.

1. ¿Qué es un modelo de machine learning?

Un modelo es una **función que no programaste tú**: la construyó un algoritmo mirando miles de ejemplos históricos donde la respuesta ya se conocía. En vez de que un ingeniero escriba reglas del tipo "si el pH baja de 9.5 y la amina sube, la sílice sube", el algoritmo **descubre esas reglas solo**, midiendo qué combinaciones de variables ocurrieron junto a qué resultados.

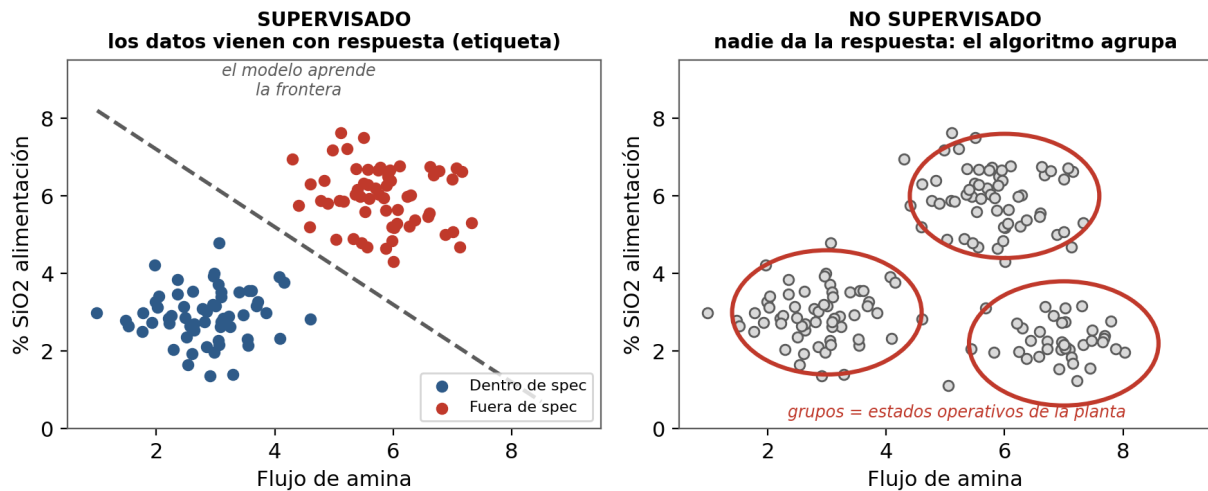


El ciclo completo: datos históricos → entrenamiento → modelo → predicción sobre datos nuevos.

Fíjate en la analogía con calibrar un instrumento: usas patrones de concentración conocida (datos de entrenamiento) para ajustar la curva de calibración (el modelo), y luego mides muestras desconocidas (predicción). ML es eso, pero con curvas de miles de dimensiones que ningún humano podría dibujar.

2. Las dos familias: supervisado y no supervisado

Todo lo básico que quieres dominar cae en una de dos familias, y la diferencia es una sola pregunta: **¿los datos históricos traen la respuesta correcta o no?**

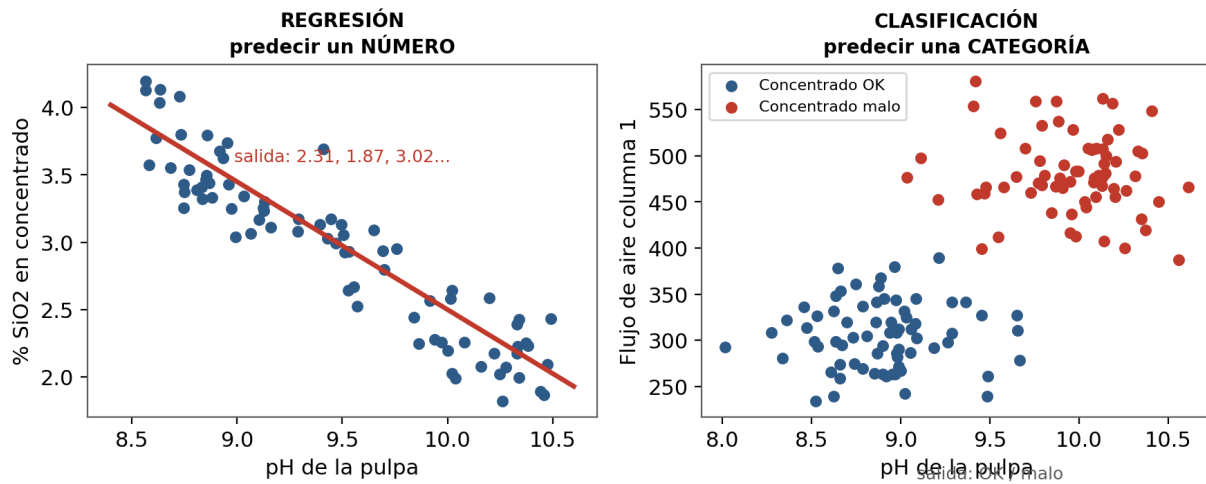


	Supervisado	No supervisado
¿Qué recibe?	Variables + la respuesta correcta (etiqueta) de cada fila	Solo variables, sin respuesta
¿Qué aprende?	A predecir la respuesta para filas nuevas	A encontrar estructura oculta: grupos, patrones
Ejemplo flotación	Predecir % SiO ₂ del concentrado	Descubrir cuántos "estados operativos" tiene la planta
Algoritmos de tu lista	Árbol de decisión, random forest	K-means

Idea clave: supervisado = aprender con profesor que corrige. No supervisado = explorar sin profesor. En geoquímica ya haces no supervisado sin saberlo: cuando miras un scatter de tus muestras de Adamuša y dices "aquí hay dos poblaciones", estás haciendo mentalmente lo que k-means hace con matemática.

3. Dentro de supervisado: regresión vs clasificación

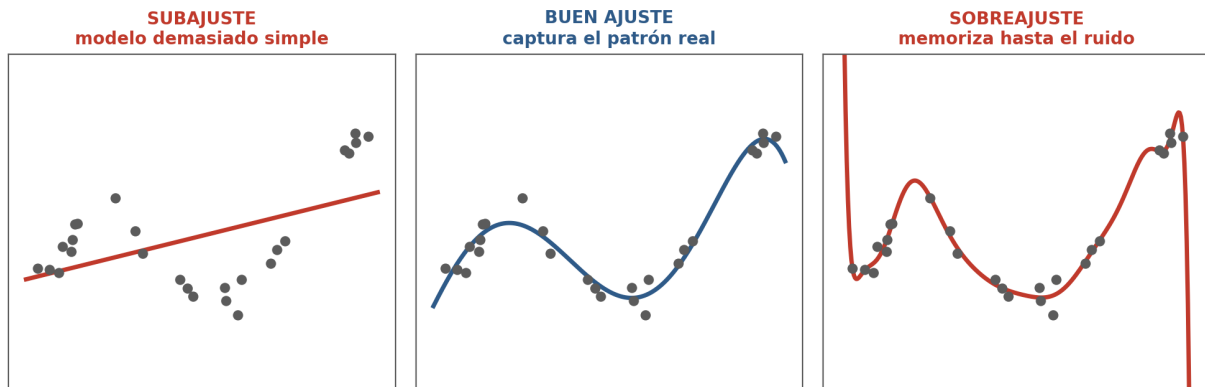
Segunda bifurcación, también una sola pregunta: **¿la respuesta que quieres predecir es un número o una categoría?**



Mismo proceso físico, dos formulaciones distintas del problema. Predecir " $\% \text{SiO}_2 = 2.31$ " es regresión. Predecir "este concentrado saldrá fuera de especificación: sí/no" es clasificación. El mismo algoritmo (un árbol, un random forest) tiene versión para ambas; solo cambia qué devuelve la hoja final.

4. El concepto más importante de todos: sobreajuste

Si solo te llevas una idea de este documento, que sea esta. Un modelo puede "acertar" el 100% de los datos con los que se entrenó y aún así ser **inútil**, porque en vez de aprender el patrón físico real, memorizó el ruido de esos datos concretos. Eso se llama **sobreajuste (overfitting)**.



Los tres paneles ajustan los mismos puntos grises. El de la izquierda es demasiado rígido (subajuste). El del centro captura la tendencia real. El de la derecha pasa por casi todos los puntos, y justo por eso es el peor: esas oscilaciones no son física, son ruido de medición que el modelo se aprendió de memoria. Con un dato nuevo, fallaría escandalosamente.

¿Cómo se detecta? Con el examen sorpresa: train/test split



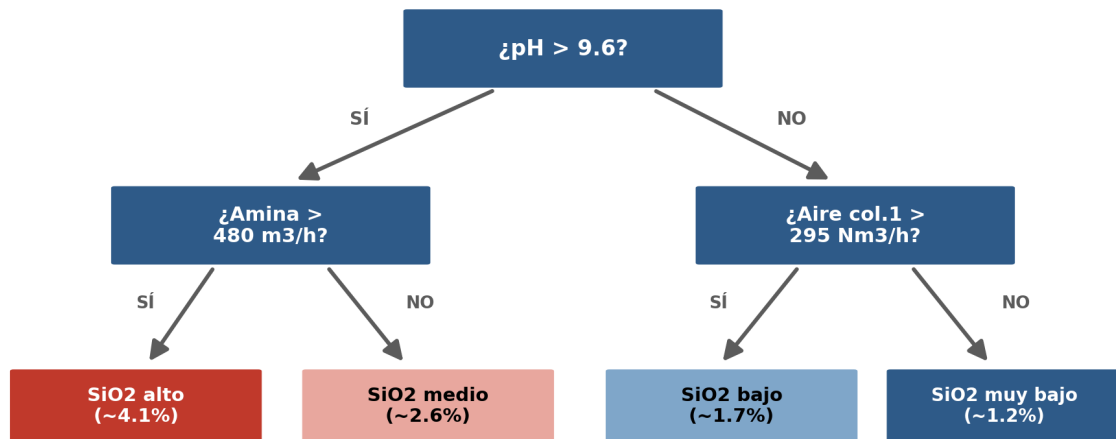
La prueba NUNCA se muestra durante el entrenamiento. Si el modelo acierta ahí, aprendió de verdad.

Antes de entrenar, apartas ~20% de las filas y las escondes. Entrenas solo con el 80%. Al final, evalúas el modelo con el 20% que nunca vio. Si rinde bien ahí, aprendió el patrón; si rinde mucho peor que en entrenamiento, memorizó. Es idéntico a estudiar con problemas resueltos y examinarse con problemas nuevos.

Detalle que casi todos hacen mal (y tú no harás): nuestro dataset de flotación es una serie de tiempo. Si partes las filas al azar, el modelo entrena con datos de las 3:00 pm y se "examina" con datos de las 2:59 pm del mismo día: está viendo el futuro. El split correcto es temporal: entrenar con marzo–julio, probar con agosto–septiembre. Esto se llama evitar la **fuga de datos (data leakage)**.

5. El árbol de decisión: el algoritmo que piensa como un operador

Un árbol de decisión es una cascada de preguntas de sí/no sobre las variables. Se lee exactamente como el razonamiento de un operador con experiencia:



Cada pregunta divide los datos en dos grupos cada vez más homogéneos.
La hoja final da la predicción: el promedio (regresión) o la clase mayoritaria (clasificación).

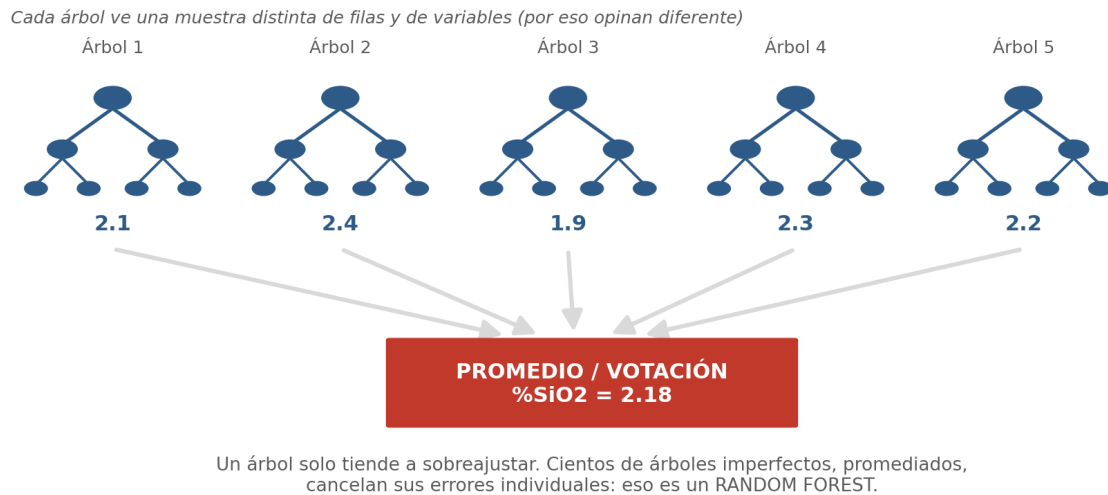
El árbol ELIGE solo qué variable y qué umbral preguntar: el que mejor separa los datos.

Lo importante no es el diagrama sino **cómo se construye**: el algoritmo prueba todas las variables y todos los umbrales posibles, y en cada nodo elige la pregunta que mejor separa los datos (la que deja los dos grupos resultantes lo más "puros" posible). Repite hacia abajo hasta que las hojas son homogéneas o se queda sin datos. Nadie le dijo que el pH importaba: lo descubrió midiendo.

Fortaleza	Debilidad
100% interpretable: puedes leer las reglas y validarlas contra la fisicoquímica de flotación que ya conoces	Sobreajusta con facilidad: si lo dejas crecer, crea una rama por cada fila y memoriza el ruido
Maneja variables numéricas y categóricas sin preparación especial	Es inestable: cambiar unas pocas filas puede producir un árbol totalmente distinto

Esa debilidad (inestable + sobreajusta) no se corrige haciendo un árbol mejor. Se corrige haciendo **muchos árboles imperfectos** y promediándolos. Página siguiente.

6. Random forest: la sabiduría del comité



Un random forest entrena cientos de árboles, pero con dos trucos de aleatoriedad para que no salgan todos iguales: (1) cada árbol entrena con una **muestra al azar de las filas** (con reemplazo, técnica llamada bootstrap), y (2) en cada nodo solo puede elegir entre un **subconjunto al azar de las variables**. El resultado: árboles diversos que se equivocan en cosas distintas. Al promediar sus predicciones (regresión) o hacerlos votar (clasificación), los errores individuales se cancelan.

Es el mismo principio por el que promedias réplicas de un análisis químico: cada medición tiene error aleatorio, pero el promedio de 30 réplicas es mucho más confiable que cualquier medición individual.

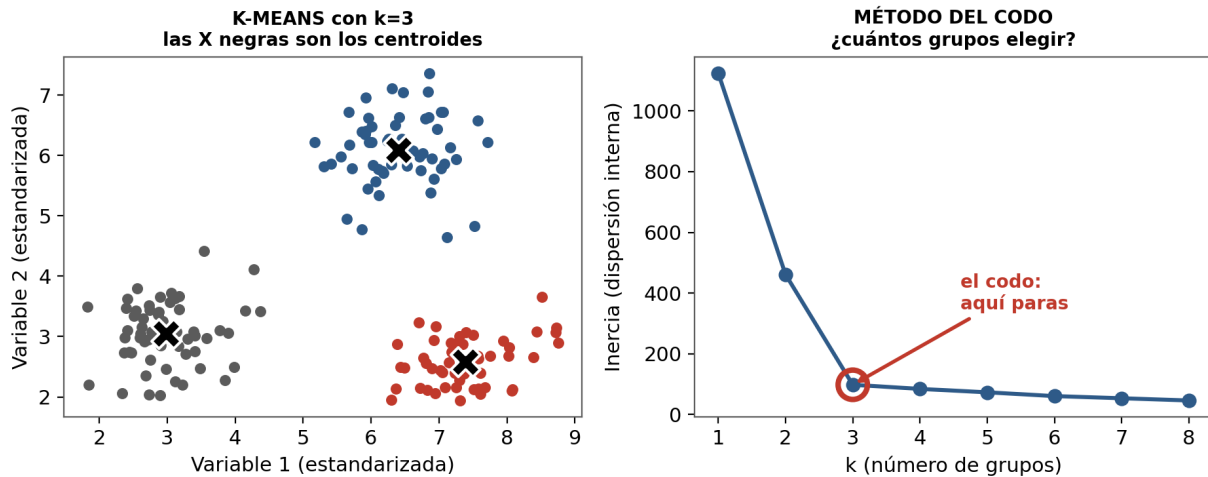
El bonus que conecta con el PhD de Curtin: importancia de variables

Como el bosque probó miles de divisiones, puede contabilizar cuánto contribuyó cada variable a mejorar las predicciones. Eso produce el ranking de **feature importance**: "el flujo de amina explica el 31% del modelo, el pH el 24%...". Herramientas como **SHAP** van más lejos y explican predicciones individuales: "esta predicción de sílice alta se debe sobre todo a que el aire de la columna 3 estaba bajo". Eso es la "explainable AI" del anuncio que viste: no es otra tecnología, es esto aplicado con rigor.

Resumen mental: árbol = un operador con experiencia pero terco. Random forest = un comité de 300 operadores, cada uno con experiencia parcial distinta, votando. El comité le gana al individuo casi siempre.

7. K-means: encontrar grupos sin que nadie te diga cuáles

Cambio de familia: aquí no hay respuesta que predecir. K-means recibe los datos pelados y les encuentra **k grupos** (tú eliges k). El algoritmo es sorprendentemente simple: (1) pone k centroides al azar, (2) asigna cada punto a su centroide más cercano, (3) mueve cada centroide al centro de sus puntos asignados, y repite 2–3 hasta que nada se mueve.

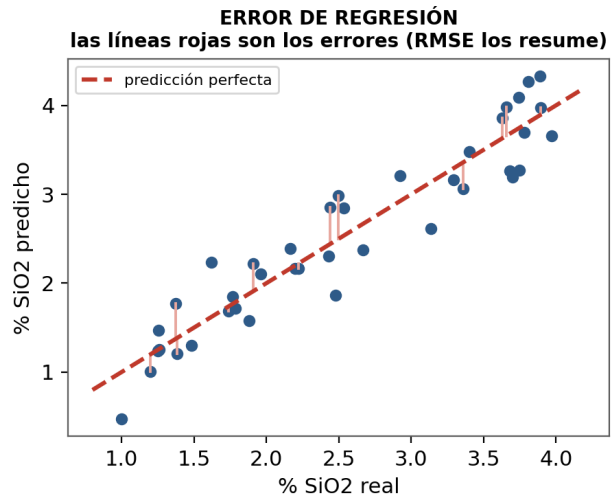
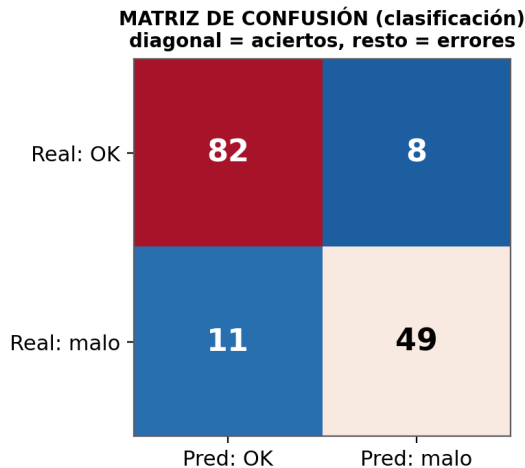


¿Y cómo eliges k si nadie sabe la respuesta? Con el **método del codo** (derecha): corres k-means con $k=1,2,3,\dots$ y grafica la inercia (qué tan dispersos quedan los puntos dentro de sus grupos). La inercia siempre baja al agregar grupos, pero llega un punto donde agregar otro grupo ya casi no ayuda: ese quiebre es el codo, y ahí paras.

Requisito de k-means	Por qué
Estandarizar las variables antes (StandardScaler: restar media, dividir por desviación)	K-means mide distancias. Si el flujo de aire va de 175 a 375 y el pH de 8.5 a 10.5, el aire dominaría la distancia solo por sus unidades. Estandarizar pone todas las variables en la misma escala.
Elegir k con criterio (codo + conocimiento del proceso)	Un $k=7$ matemáticamente aceptable puede no significar nada operativamente; un $k=3$ que corresponde a "arranque, régimen estable, perturbación" vale oro.

En el proyecto lo usaremos para descubrir los estados operativos de la planta de flotación, y en tu mundo geo es la misma herramienta con la que se separan poblaciones geoquímicas o dominios de estimación.

8. ¿Cómo sabemos si el modelo es bueno? Métricas



Para regresión (predecir números)

Métrica	Qué mide	Cómo leerla
RMSE	Raíz del promedio de errores al cuadrado	En las unidades de la variable: RMSE = 0.4 significa que el modelo se equivoca típicamente ± 0.4 puntos de % SiO ₂ . Penaliza fuerte los errores grandes.
MAE	Promedio del valor absoluto de los errores	Similar pero sin castigar extra los errores grandes. Más fácil de explicar a un gerente.
R ²	Fracción de la variabilidad que el modelo explica	1.0 = perfecto, 0 = no mejor que predecir siempre el promedio. Un R ² de 0.85 explica el 85% de la variación.

Para clasificación (predecir categorías)

Métrica	Qué mide	Cuándo importa
Accuracy	% total de aciertos	Engañosa con clases desbalanceadas: si el 95% de las horas la planta está OK, un modelo que siempre dice "OK" tiene 95% de accuracy y es inútil.
Precisión	De las alarmas que dio el modelo, cuántas eran reales	Cuando las falsas alarmas cuestan (parar la planta sin necesidad).
Recall	De los problemas reales, cuántos detectó el modelo	Cuando dejar pasar un problema cuesta (mandar concentrado malo al cliente).

La matriz de confusión (figura izquierda) es la radiografía completa: la diagonal son aciertos, las esquinas son los dos tipos de error. Precisión y recall se calculan directo de ella, y elegir cuál priorizar es una decisión de ingeniería, no de estadística.

9. El mapa completo y lo que sigue

Módulo	Concepto	Qué haremos con el dataset de flotación	Entregable
1	Exploración de datos (EDA)	Cargar las 737k filas, entender cada variable, correlaciones, visualizar la serie de tiempo	PDF: radiografía del dataset
2	Regresión con árbol y random forest	Predecir % SiO ₂ con split temporal, comparar árbol solo vs bosque, medir RMSE y R ²	PDF: qué aprendió el modelo
3	Interpretabilidad (feature importance + SHAP)	Ranking de variables y explicación de predicciones individuales en lenguaje de proceso	PDF: el porqué de cada predicción
4	K-means	Estandarizar, método del codo, descubrir estados operativos y perfilarlos	PDF: los regímenes de la planta

Cada módulo repite el mismo ritmo: primero una mini explicación visual del concepto nuevo, luego lo aplicamos al dato real, y cerramos con el PDF que documenta qué pasó. Al terminar el Módulo 4 tendrás tu primer proyecto de portafolio completo y, más importante, entenderás cada decisión que se tomó.

Módulo 1 — El dato: análisis exploratorio

Apunte 01 — Cargar datos sin que te mientan

Módulo 1 (EDA) · Machine Learning desde cero

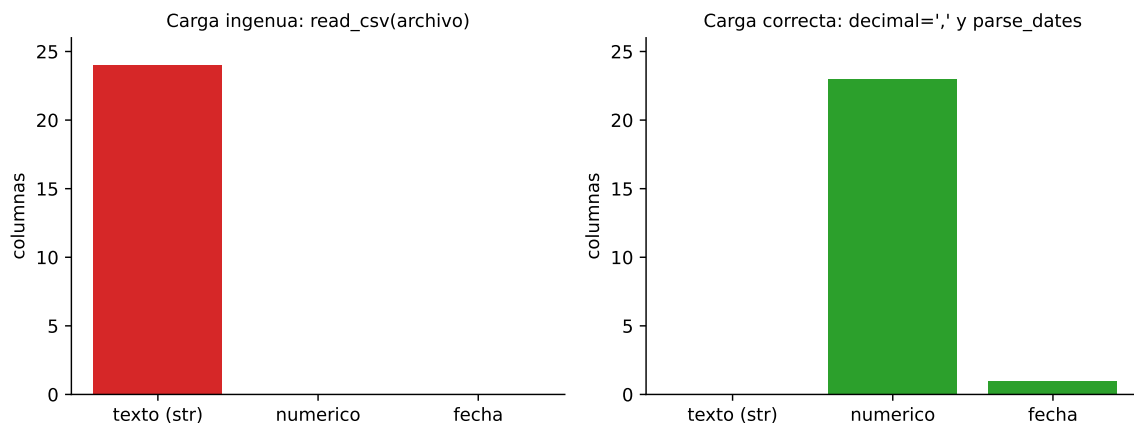
La escena

Descargamos de Kaggle el dataset *Quality Prediction in a Mining Process*: 737,453 filas $\times$ 24 columnas, seis meses (marzo–septiembre 2017) de una planta brasileña de flotación inversa de hierro. Lo cargamos con la instrucción más natural del mundo (`read_csv`) y... no hubo ningún error. Y sin embargo el dataset quedó inservible.

Concepto

Los errores más peligrosos en datos son los **fallos silenciosos**: no revientan con mensaje rojo, se propagan. La planta es brasileña y escribe 55,2 con **coma decimal**; pandas espera punto, no reconoció los números, y cargó las 24 columnas como **texto**. Todo “funcionó”: las filas están, las columnas están — pero no puedes promediar, graficar ni entrenar nada con el texto "55,2". La solución fue una palabra: `read_csv(..., decimal=',', parse_dates=['date'])`.

El fallo silencioso: mismas 737,453 filas, utilidad opuesta



El ritual de verificación (siempre, con todo dataset)

1. **Dimensiones** (`df.shape`): ¿están las filas y columnas que esperaba? Aquí: (737453, 24). ✓
2. **Tipos** (`df.dtypes`): ¿los números son números y las fechas son fechas? Aquí fue donde saltó la trampa: todo era `str`.
3. **Sentido físico** (`df.head()`): ¿las primeras filas son plausibles? Un pH de 9.7 y un Fe de alimentación de 55 % sí lo son.

Ojo

“Cargó sin errores” $\neq$ “cargó bien”. En este proyecto la trampa fue la coma decimal; otras clásicas: fechas como texto, unidades mezcladas, ceros que en realidad significan “sin dato”. La pregunta profesional no es ¿cargó?, sino **¿qué evidencia tengo de que cargó correctamente?**

Pruébalo

La analogía de laboratorio: que el espectrómetro encienda y entregue un número no significa nada — antes de confiar corres un **estándar certificado** y verificas que la calibración dé dentro de tolerancia. `shape`, `dtypes` y `head` son tus estándares certificados: baratos, rápidos, y te salvan de reportar un análisis con el equipo descalibrado. Pregunta de entrevista para practicar en voz alta: *“Cargaste 737 mil filas sin error, ¿por qué no te basta para confiar en el dataset?”*

El tropiezo que enseña

Lo vivimos en orden deliberado: primero la carga ingenua (todo texto), luego el diagnóstico (`dtypes`), luego la cura (`decimal=','`). El código definitivo quedó en `src/load_data.py` del repo: una función `load_floatation_data()` que todos los módulos reutilizarán, con el porqué documentado para quien clone el proyecto.

Apunte 02 — Estadística descriptiva: la radiografía del dataset

Módulo 1 (EDA) · Machine Learning desde cero

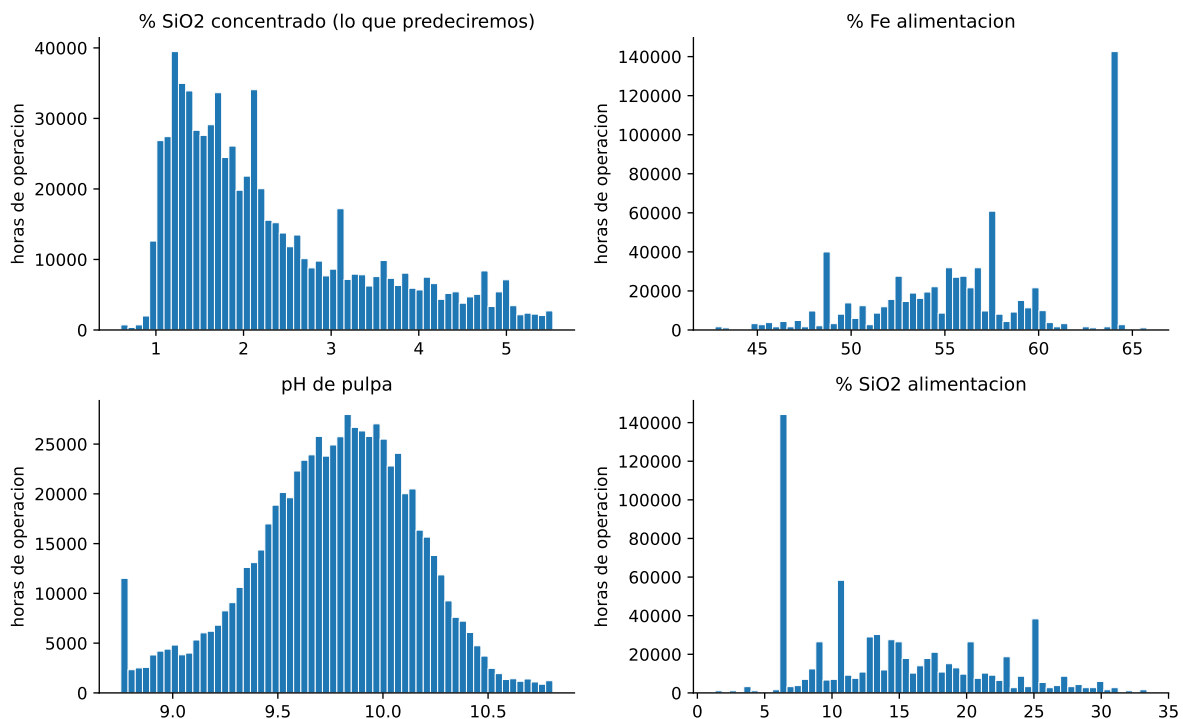
Cuatro números que resumen 737 mil filas

Nadie puede leer 737,453 filas. La estadística descriptiva comprime cada variable en unos pocos números: **mínimo** y **máximo** (los bordes del sobre operativo), **media** (el centro de gravedad) y **desviación estándar** σ (cuánto se aleja típicamente la variable de su media — su “inquietud”).

Concepto

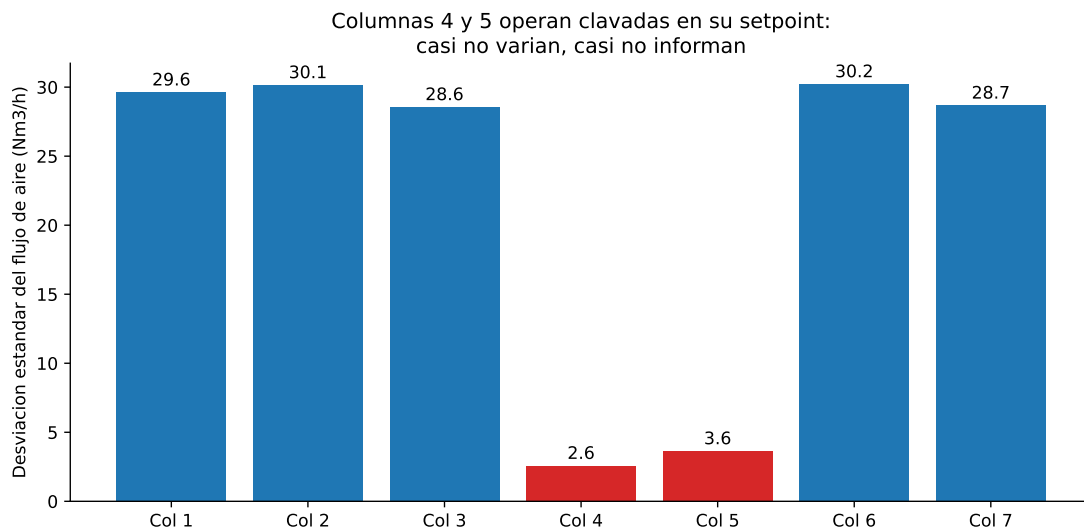
La estadística descriptiva es el **balance metalúrgico de reconocimiento** del dataset: antes de optimizar nada verificas que las leyes de entrada y salida sean creíbles y cierren. Para nuestro caso: pH de pulpa 8.75–10.8 (exactamente la ventana de flotación catiónica inversa con amina), densidad 1.52–1.85, Fe de alimentación 42.7–65.8 %, y el objetivo a predecir — % SiO₂ del concentrado— entre 0.6 y 5.53 % con media 2.33 %.

Distribucion de variables clave (737,453 filas, mar-sep 2017)



El histograma es la distribución de cada variable: cuántas horas de operación pasaron en cada rango de valores. Fíjate que la sílice del concentrado (arriba izquierda) se apila cerca de 1–2 % (la planta normalmente hace bien su trabajo) con una cola larga hacia 5 % (los episodios malos: justo lo que un modelo útil debe anticipar).

El hallazgo: variables que no varían no informan



Concepto

Cinco de las siete columnas de flotación mueven su aire con $\sigma \approx 30$; las columnas 4 y 5 están **clavadas en su setpoint** ($\sigma = 2,6$ y $3,6$). Una variable casi constante casi no aporta a un modelo predictivo: **no se puede aprender el efecto de algo que nunca cambia**. Es la ley de un elemento que no varía entre muestras de un yacimiento: inútil para delimitar dominios. Solo lo que varía, informa.

Las dos sospechas del detective

Ojo

(1) **Cero valores faltantes** en seis meses de datos industriales es demasiado bonito: los sensores fallan, el laboratorio se atrasa. Cuando un dataset de planta viene “perfecto”, casi siempre alguien lo rellenó o recortó antes de publicarlo — hay que averiguar qué le hicieron antes de confiar. (2) **Hay 1,171 filas totalmente duplicadas** y el flujo de almidón tiene un mínimo de 0.002 (¿planta parada?, ¿sensor caído?). Ambas pistas se investigan en el Apunte 03 con la serie de tiempo.

Pruébalo

Pregunta de entrevista para practicar: “¿Qué buscabas en las estadísticas descriptivas y qué encontraste que un ojo no experto pasaría por alto?” Estructura de la respuesta: (a) plausibilidad física de rangos — aquí tu química de flotación es el patrón de referencia, (b) qué variables tienen variabilidad real (columnas 4–5 clavadas), (c) señales de manipulación previa (cero faltantes sospechoso). Esa respuesta en tres actos suena a ingeniero senior, no a estudiante de curso online.

Apunte 03 — Series de tiempo: las cicatrices del dato real

Módulo 1 (EDA) · Machine Learning desde cero

De 20 segundos a seis meses legibles

Nuestro dataset trae 737,453 filas muestreadas cada ~ 20 segundos. Nadie puede leer eso, y matplotlib tampoco dibuja seis meses de señal de 20 s sin volverse una mancha. La solución es el **resampleo**: agrupar por hora y promediar, pasando de 737,453 puntos a 4,097 horas legibles. Es lo mismo que haces al reportar una ley promedio por turno en vez de cada lectura instantánea del analizador.

Concepto

Una **serie de tiempo** es una secuencia de mediciones ordenadas por su marca temporal, donde el **orden importa**: el valor de ahora se parece al de hace un minuto —eso se llama **autocorrelación**. Esto rompe la suposición cómoda del resto del machine learning (que las filas son independientes) y cambia dos cosas: (1) cómo se grafica —contra el tiempo, para ver tendencias, huecos y saltos— y (2) cómo se evalúa un modelo —con un corte **temporal**, no aleatorio (Módulo 2).

La estructura oculta: ¿cada cuánto cambia cada variable?

Antes de confiar en el dataset medimos, con evidencia, cada cuánto cambia *de verdad* cada variable: contamos cuántos valores distintos toma dentro de cada hora. El resultado corrigió una suposición inicial nuestra.

Variable	Rol	Valores únicos/hora
pH de pulpa	proceso (sensor ~ 20 s)	~ 171
Flujo de amina	proceso (sensor ~ 20 s)	~ 176
% Fe alimentación	laboratorio (horario)	1.0
% SiO ₂ alimentación	laboratorio (horario)	1.0
% SiO ₂ concentrado (<i>objetivo</i>)	lab / derivado	~ 14

Lectura: las variables de **proceso** (pH, flujos, aire, niveles) tienen 150–180 valores por hora —son sensores reales cada 20 s. Los ensayos de **alimentación** son perfectamente constantes dentro de la hora (un solo valor): el laboratorio los reporta una vez por hora y se repiten en las ~ 180 filas. Y el **objetivo** —% SiO₂ del concentrado— está en tierra de nadie: varía (~ 14 valores/hora), pero mucho más grueso que un sensor de proceso. *Ojo con esto*: la idea intuitiva de que “el dato de lab se repite 180 veces por hora” solo es cierta para la alimentación, no para el objetivo. Los datos mandan sobre la intuición.

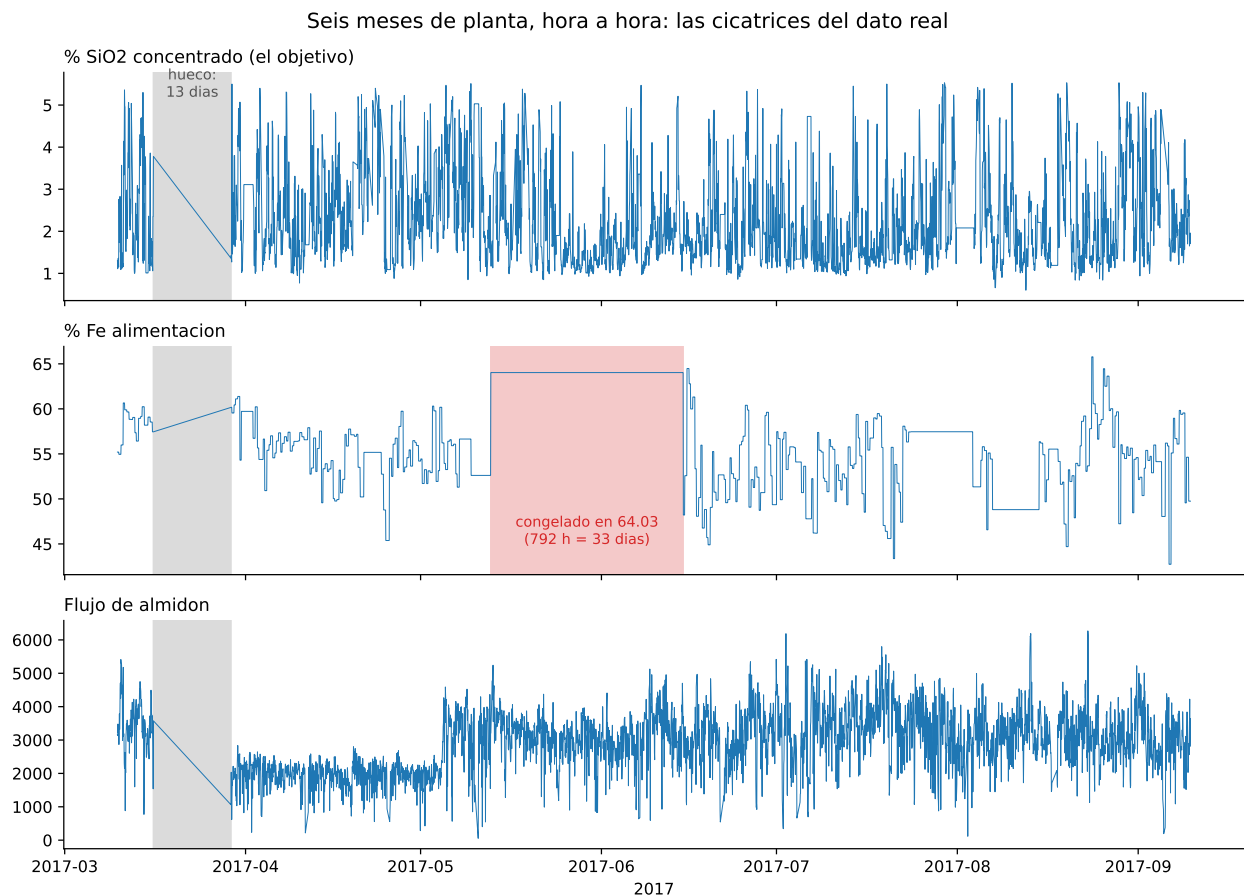
Ojo

¿Por qué importa esta microestructura? Porque las filas **no son independientes**. Dos filas separadas por 20 segundos son casi gemelas: sus variables de proceso apenas cambiaron, sus ensayos de alimentación son idénticos y su objetivo está muy cerca. Si repartes las filas

al azar en 80 % entrenamiento y 20 % prueba, cada fila de prueba tiene casi-gemelas en entrenamiento: el modelo parece brillante, pero solo memorizó vecindarios, no aprendió física. Esto es **fuga de datos** (*data leakage*), y es el error que Módulo 2 evitará con un corte temporal.

Las cicatrices del dato real

Graficar las variables contra el tiempo (resampleadas por hora) revela tres cicatrices que un histograma jamás mostraría:



- **Hueco de 13 días** (16–29 de marzo): la planta o la adquisición se detuvo. La serie **no es continua** —algo que tendremos en cuenta al modelar.
- **Ensayo de Fe congelado**: 792 horas (~33 días) clavado en 64.03. No es que el mineral no cambiara: el laboratorio dejó de actualizar y el último valor se arrastró.
- **Almidón casi en cero**: 3,976 filas por debajo de 100 —episodios de dosificación detenida o sensor caído.

Ojo

El tramo congelado de 33 días es la trampa más peligrosa porque **no se ve**: los números están, parecen datos, pero durante ese mes el % Fe de alimentación no aporta información real —es una constante disfrazada de medición, y un modelo que “aprenda” de ahí aprende ruido. La lección meta: nuestra suposición era que *todos* los ensayos de lab estaban repetidos 180 veces por hora; los datos la corrigieron. **Nunca asumas la estructura de tus datos:**

mídela.

Pruébalo

Pregunta de entrevista: “¿Por qué un *train/test split* aleatorio es una trampa en datos de planta, y qué harías en su lugar?”

Respuesta modelo: “Porque es una serie de tiempo con fuerte autocorrelación: filas vecinas son casi idénticas. Un split aleatorio deja casi-gemelas en entrenamiento y prueba, así que el conjunto de prueba no es realmente nuevo y la métrica sale inflada —el modelo fallará con datos futuros de verdad. En su lugar uso un **split temporal**: entreno con marzo–julio y pruebo con agosto–septiembre, replicando cómo se usaría en producción (predecir el futuro desde el pasado). Eso es evitar la fuga de datos.”

La analogía metalúrgica: validar el modelo con filas vecinas es como comprobar la exactitud de un analizador en línea (tipo Courier/XRF) usando **la misma alícuota de pulpa** con la que lo calibraste: coincidirá siempre —es la misma pulpa— y no probaste nada. La comprobación honesta usa pulpa de *otro* momento, independiente. En el tiempo, “otro momento” son los meses que apartas para el test.

Apunte 04 — Correlación: cuando “moverse juntos” te engaña

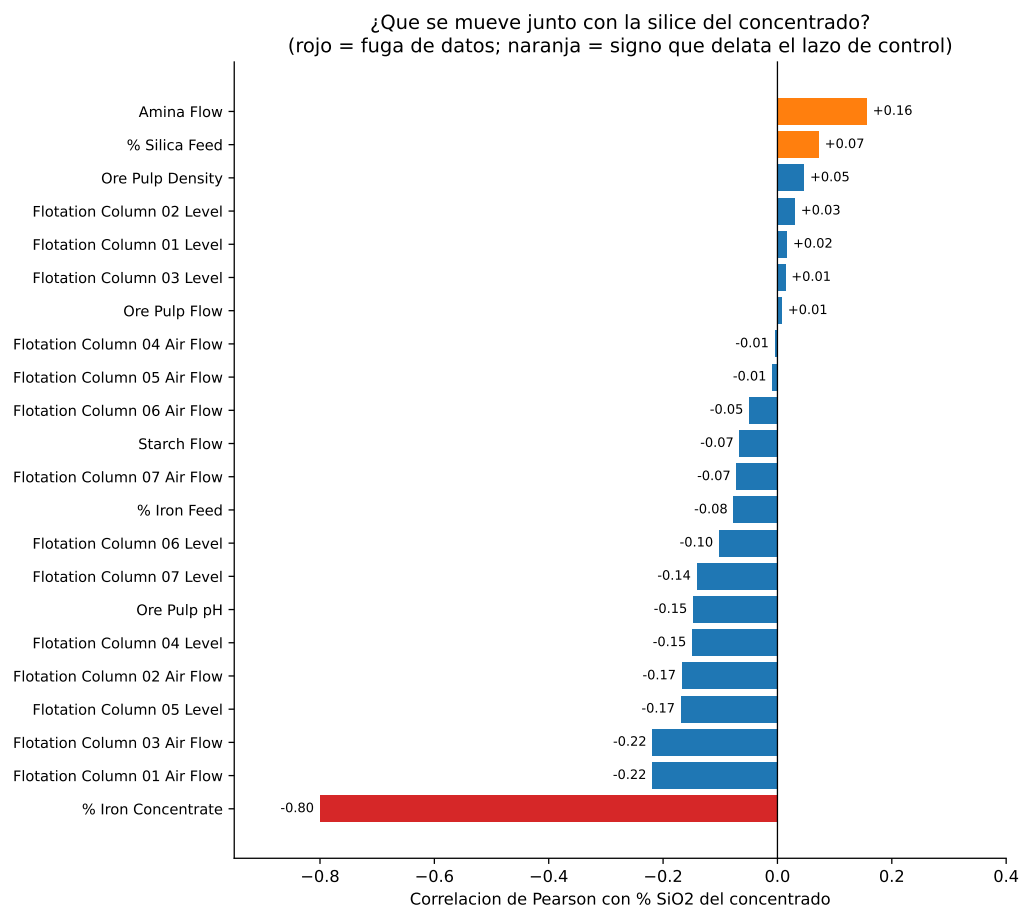
Módulo 1 (EDA) · Machine Learning desde cero

Una sola pregunta: ¿se mueven juntas?

Con 23 variables no puedes mirar todas las parejas a ojo. La **correlación** resume, en un solo número entre -1 y $+1$, qué tan de la mano se mueven dos variables. Es la primera lupa para preguntarle al dataset: “¿quién acompaña a la sílice del concentrado?”

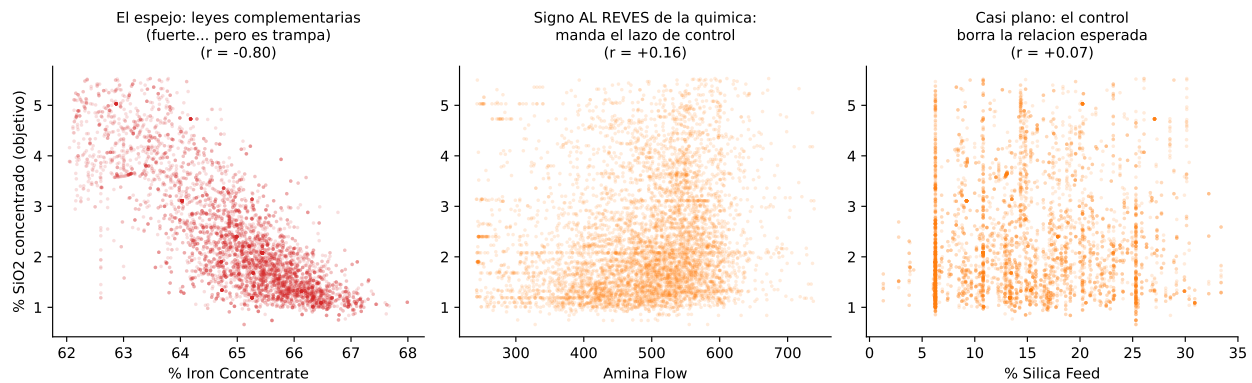
Concepto

La **correlación de Pearson** mide la fuerza y el sentido de la relación **lineal** entre dos variables. $+1$: cuando una sube, la otra sube en proporción perfecta. -1 : cuando una sube, la otra baja. 0 : sin relación lineal. Dos avisos que cambian todo: mide **co-movimiento**, **no causa**, y solo ve **líneas rectas** —una relación curva fuerte puede dar correlación baja.



Lectura del ranking: la sílice del concentrado casi no correlaciona linealmente con nada...salvo **tres casos** que valen una clase entera.

Correlacion no es causa: tres graficas que lo demuestran en tu planta



- **% Fe concentrado ($-0,80$) — el espejo, y una trampa.** Un concentrado de hierro es, a grandes rasgos, óxido de hierro + ganga sílicea: si sube el Fe, baja la sílice, casi por definición. Correlación altísima y *real*...pero inútil y peligrosa: el % Fe del concentrado también sale del laboratorio una hora después. En tiempo real **no lo tienes**. Usarlo sería meterle la respuesta al modelo por la ventana —eso es **fuga de datos**.
- **Flujo de amina ($+0,16$) — el signo imposible.** La amina *flota* la sílice hacia afuera; su efecto causal sobre la sílice del concentrado es **negativo**. Pero en los datos sale **positivo**. ¿Por qué? El **lazo de control**: el operador sube la amina justo cuando la sílice está alta. Amina alta y sílice alta aparecen juntas, aunque la amina esté ahí para bajarla.
- **% SiO₂ alimentación ($+0,07$) — el plano sospechoso.** Esperarías que más sílice de entrada → más sílice de salida. Casi nada. El control lo compensa: cuando entra más sílice, se dosifica más amina y se ajusta el proceso, **borrando** la relación natural en los datos.

Ojo

Correlación no es causa —y en una planta **controlada** es peor: las acciones de control confunden e incluso **invierten** los signos (a esto se le llama *confusión* o *confounding*). Dos consecuencias para modelar: (1) **no elijas variables por su correlación cruda**: una fuerte puede ser fuga (% Fe concentrado), una débil puede ser clave vía efectos no lineales que un random forest sí captura (Módulo 2); (2) toda correlación en datos de proceso se lee **con el diagrama de control en la mano**.

Pruébalo

Pregunta de entrevista: “En tus datos, el flujo de amina correlaciona positivo con la sílice del concentrado, pero la química dice que la amina la flota hacia afuera (debería ser negativo). ¿Cómo lo explicas, y qué te enseña sobre usar correlaciones?”

Respuesta modelo: “Es el lazo de control. El operador sube la amina precisamente cuando la sílice está alta, para bajarla; en los datos ambas aparecen juntas aunque el efecto causal de la amina sea el contrario. La correlación mide co-movimiento, no causa, y en una planta controlada las acciones de control confunden e invierten los signos. Por eso no elijo variables por su correlación cruda: una fuerte puede ser fuga (% Fe concentrado, casi el complemento de la sílice y que tampoco tengo en tiempo real) y una débil puede importar por efectos no lineales.”

La analogía metalúrgica: es como concluir que “más colector empeora la recuperación” mirando el historial de planta, cuando en realidad el metalurgista sube el colector *precisamente* cuando el mineral viene difícil de flotar. El colector no es el culpable: es la **reacción** a un problema previo. Confundes la medicina con la enfermedad —y ese es justo el error que

la correlación cruda te invita a cometer.

Apunte 05 — Fuga de datos: las tres formas de hacer trampa sin querer

Módulo 1 (EDA) · concepto transversal · Machine Learning desde cero

El error que te hace quedar mal en producción

De todos los errores de machine learning, este es el que más caro se paga: el modelo da un número espectacular en la prueba, lo llevas a la planta...y falla. No es mala suerte: casi siempre es **fuga de datos**. Vale la pena un apunte propio porque es transversal —aparece en el split (Apunte 03) y en la correlación (Apunte 04)— y porque saber detectarlo es lo que te separa de un principiante en una entrevista.

Concepto

Hay **fuga de datos** (*data leakage*) cuando el modelo, mientras entrena, recibe información que en la vida real **no tendría en el instante de predecir**. El efecto siempre es el mismo: la métrica de prueba sale inflada y **te miente**, porque el modelo aprobó un examen con las respuestas a la vista. La pregunta que lo caza es una sola: *¿tendría este dato, en este momento, en la planta real?*

¿Que puedo usar para predecir la sílice AHORA?

instante de predicción (ahora)

DISPONIBLE en tiempo real	AUN NO EXISTE (+1 hora)
pH de pulpa	% Fe concentrado
Flujo de amina	% SiO ₂ concentrado
Flujo de almidón	(el objetivo: lo que quiero predecir)
Densidad de pulpa	
Aire columnas 1-7	
Niveles columnas 1-7	
señales de proceso - cada ~20 s	resultado de laboratorio - cada 1 h

Regla anti-fuga: una variable de entrada solo es lícita si YA existe a la izquierda de la línea.

Las tres formas de que se te cuele

1. Fuga de la respuesta (o del futuro)

Meter como variable de entrada algo que ya *contiene* la respuesta o que solo se conoce *después*. En tu dataset: predecir el % SiO₂ del concentrado usando el % **Fe del concentrado**. Son casi complementarios (concentrado $\approx$ óxido de hierro + ganga silíceo), por eso correlacionan $-0,80$. El R^2 sería de fantasía, pero el % Fe del concentrado **también sale del laboratorio una hora después**: en tiempo real no lo tienes. *Regla*: toda variable de entrada debe existir en el instante de predecir —a la izquierda de la línea de la figura.

2. Contaminación entrenamiento–prueba

Que el examen tenga casi-copias del estudio. En una serie de tiempo pasa solo: filas vecinas (a 20 s) son casi idénticas, y un reparto al azar deja gemelas en ambos lados. La cura es el **split temporal** (entrenar con los primeros meses, probar con los últimos). *Analogía*: comprobar un analizador en línea con la misma alícuota de pulpa con que lo calibraste —coincide siempre y no prueba nada.

3. Fuga por preprocesamiento (la silenciosa)

Preparar los datos usando información de **todo** el dataset —incluido el test— antes de separarlos. El caso clásico: estandarizar (restar la media, dividir por la desviación) calculando esa media con *todas* las filas, incluidas las de prueba. La media del test se filtró al entrenamiento. *Cura*: calcular media y desviación **solo** con el entrenamiento y aplicar esos mismos números al test.

Ojo

La tercera es la más peligrosa porque **no se ve**: no hay una columna obviamente prohibida, solo un cálculo hecho en el orden equivocado. *Analogía metalúrgica*: es calibrar tu balanza o tu analizador con un estándar que, sin darte cuenta, incluye muestras del lote que **todavía no has procesado**. Por eso la regla meta se aplica a todo el flujo, no solo a las columnas: *¿esta información ya existía, en este momento, en la planta real?*

Pruébalo

Pregunta de entrevista: “Tu modelo da $R^2 = 0.98$ en la prueba, pero en planta falla feo. ¿Qué es lo primero que sospechas y cómo lo cazas?”

Respuesta modelo: “Fuga de datos, casi seguro. La cazo revisando las tres fuentes: (1) que cada variable de entrada exista en tiempo real y no sea una salida de laboratorio disfrazada —como el % Fe del concentrado, que llega una hora tarde; (2) que el split sea **temporal** y no aleatorio, porque es una serie de tiempo con filas vecinas casi idénticas; y (3) que la estandarización y cualquier preprocesamiento se hayan ajustado **solo con el entrenamiento**. La prueba de fuego: si el modelo se cae al evaluarlo sobre un periodo genuinamente futuro, la métrica anterior estaba inflada.”

La analogía metalúrgica: un ensayo de leyes que “cuadra” perfecto porque mediste dos veces el **mismo** estándar no demuestra exactitud —demuestra que sabes copiar. La exactitud real se prueba contra material independiente. En ML, ese material independiente es un periodo que el modelo nunca vio.

Apunte 06 — Radiografía del dataset

Cierre del Módulo 1 (EDA) · Machine Learning desde cero

Lo que ahora sabemos del dataset

Cerramos el Módulo 1. Empezamos con un CSV desconocido de 737.453 filas y hoy lo conocemos por dentro: qué mide, cómo cuánto, qué variables importan y —sobre todo— dónde están sus trampas. Esta es la **radiografía**: la foto que se le muestra al médico antes de operar. En el Módulo 2 operamos (entrenamos el modelo); aquí confirmamos que sabemos con qué cuerpo trabajamos.

Concepto

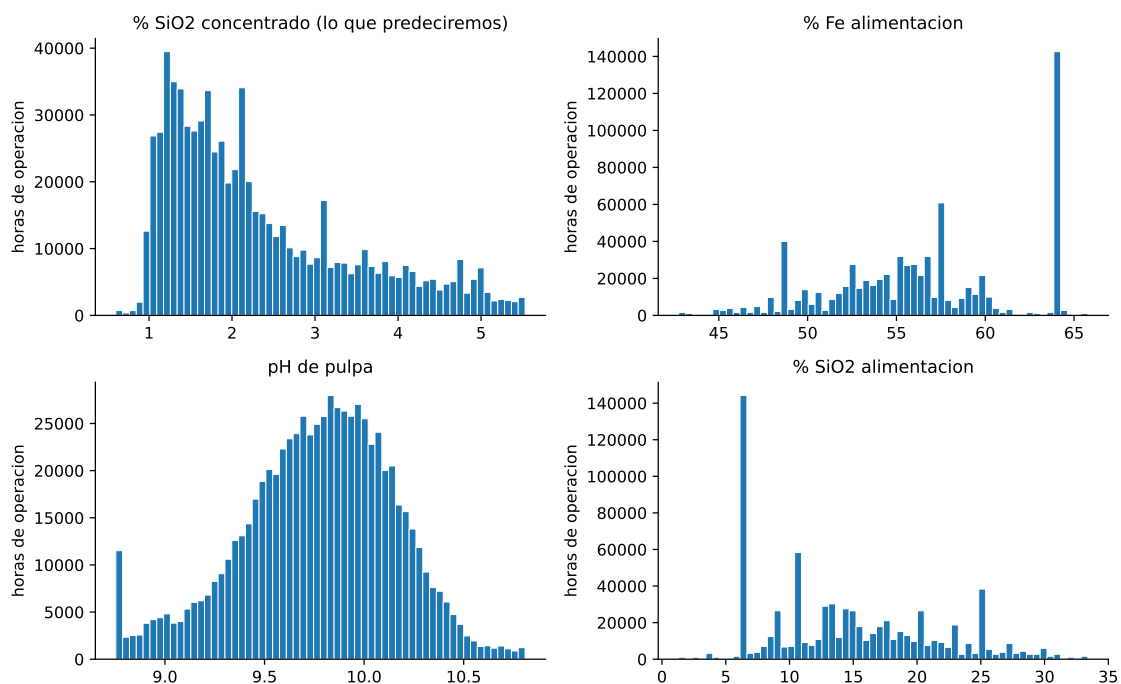
El problema, en una frase: predecir en tiempo real el % de SiO_2 del concentrado (hoy el laboratorio tarda 1 hora) a partir de las variables de proceso que sí se leen al instante. Es una **regresión** sobre una **serie de tiempo**: dos hechos que mandan sobre todas las decisiones que vienen.

Las 23 variables, por rol

Grupo	N.º	Rol	Disponibilidad
Alimentación (% Fe, % SiO_2)	2	ley de entrada (lab)	1/hora, constante en la hora
Reactivos y pulpa	5	proceso	cada ~20 s
Aire de columnas 1–7	7	proceso	cada ~20 s
Niveles de columnas 1–7	7	proceso	cada ~20 s
Salidas de laboratorio	2	calidad (lab)	1/hora, +1 h de retraso

Objetivo: % SiO_2 concentrado —rango 0.6–5.53, media 2.33.

Distribucion de variables clave (737,453 filas, mar-sep 2017)



Lo que aprendimos, apunte por apunte

#	Concepto	Hallazgo clave en tus datos
01	Cargar sin mentiras	la coma decimal cargaba las 24 columnas como texto
02	Estadística descriptiva	columnas 4–5 clavadas en setpoint; 0 faltantes (sospechoso)
03	Series de tiempo	180 filas/hora; hueco de 13 días; Fe feed congelado 33 días
04	Correlación	% Fe conc $r = -0,80$ (fuga); amina $+0,16$ (signo invertido por control)
05	Fuga de datos	tres familias y la regla anti-fuga

Estado de salud y decisiones para el Módulo 2

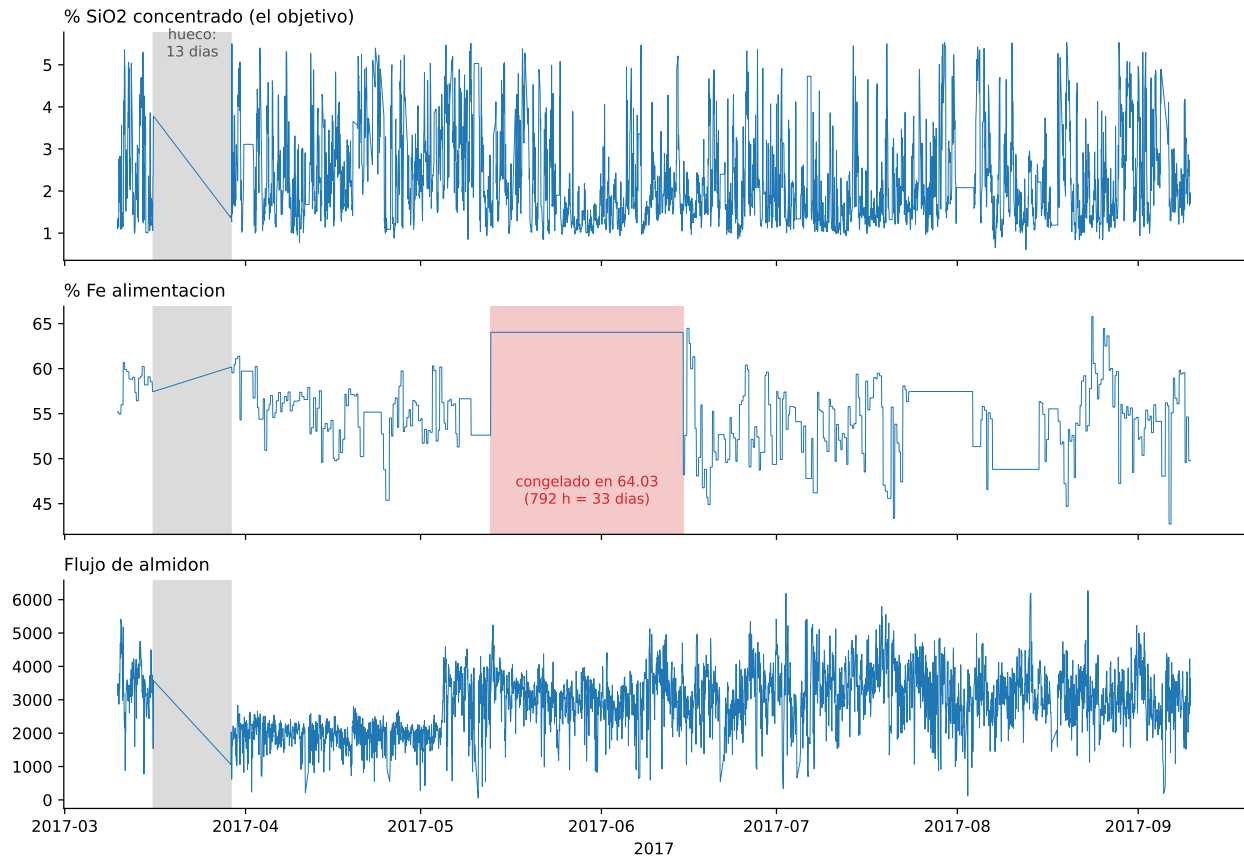
Tres cicatrices que hay que decidir cómo tratar. *Yo propongo con datos; tú, como dueño del proceso, validas.*

Issue	Magnitud (medida)	Recomendación
Filas duplicadas	1.171 filas, en 15 horas	eliminar (impacto nulo)
Hueco temporal	13 días (16–29 mar)	dejarlo; el split temporal lo salta
Ensayo feed congelado	19,4 % filas, <i>solo</i> 2 columnas	conservar filas; feed débil

Ojo

La decisión más fina: el tramo congelado son 142.740 filas (19,4 %), pero **solo** las 2 columnas de ensayo de alimentación están clavadas; el proceso y el objetivo siguen vivos ahí. **Botar ese 19,4 % sería tirar datos buenos.** Por eso la recomendación es conservar las filas y, en el Módulo 2, decidir qué hacer con esas 2 columnas de feed (son además predictores débiles, $|r| < 0,08$): lo natural es conservarlas y dejar que el random forest juzgue su importancia. **Esa decisión es tuya.**

Seis meses de planta, hora a hora: las cicatrices del dato real



Pruébalo

Pregunta de entrevista (integradora): “Te entregan este dataset de planta y una semana. ¿Cuáles son las tres primeras cosas que revisas antes de entrenar cualquier modelo?”

Respuesta modelo: “(1) Que **cargara bien**: tipos correctos, sin la trampa del formato regional (aquí, la coma decimal). (2) La **estructura temporal**: es una serie de tiempo con filas casi idénticas dentro de la hora, así que el split debe ser **temporal**, no aleatorio. (3) Las **fugas**: que cada variable de entrada exista en tiempo real (fuera el % Fe del concentrado, que llega 1 h tarde) y leer toda correlación con el diagrama de control en la mano. Recién entonces entreno.”

La analogía metalúrgica: es el **muestreo antes del ensayo**. Ningún metalurgista serio manda una pulpa al análisis sin verificar primero que la muestra sea representativa, esté bien homogeneizada y sin contaminación. El EDA es ese protocolo de muestreo: si te lo saltas, todo el “análisis” posterior —por sofisticado que sea— hereda el error de una muestra mala.

Módulo 1: cerrado. Seis apuntes en tu biblioteca. Siguiendo parada: el Módulo 2, donde por fin entrenamos el árbol y el random forest —con split temporal, ya sabes por qué.

Módulo 2 — Regresión: árbol y random forest

Apunte 07 — Tu primer modelo: el árbol de decisión

Módulo 2 (Regresión) · Machine Learning desde cero

Hasta ahora conocimos el dato (Módulo 1). Hoy construimos el **primer modelo** que predice el % de SiO₂. Vamos MUY despacio: cada palabra nueva se define antes de usarla.

Glosario — en palabras simples

- **Modelo:** una fórmula que, dadas las condiciones de la planta, escupe un número predicho (aquí, el % de SiO₂ del concentrado). No la escribes tú; la arma un algoritmo mirando datos.
- **Feature** (variable de entrada, la “X”): cada columna que usamos como *pista* para predecir (pH, flujos, aire, niveles...). En inglés se dicen *features*; en tu hoja de laboratorio serían las columnas de condiciones.
- **Objetivo** (la “y”): la columna que queremos adivinar: % SiO₂ concentrado.
- **Entrenar:** el proceso de que el algoritmo *mire* muchos ejemplos con la respuesta conocida y saque sus reglas.
- **Predecir:** usar esas reglas sobre una fila nueva para estimar su % SiO₂.

1. Separar las pistas (X) de la respuesta (y)

Nuestra tabla tiene 24 columnas. Las repartimos en dos papeles:

- **y (lo que queremos predecir):** % SiO₂ del concentrado.
- **X (las pistas):** 21 columnas de proceso y alimentación.

¿Por qué 21 y no 22? Porque sacamos el **% Fe del concentrado**. Aunque es la variable que más correlaciona con la sílice (Apunte 04), también sale del laboratorio *una hora después*: en tiempo real no la tendríamos. Usarla sería **fuga de datos** (Apunte 05). La dejamos fuera a propósito. Tus 2 columnas de alimentación, en cambio, sí se quedan (tu decisión: que el modelo mida cuánto valen).

2. Las cuatro piezas: entrenar y examinar

Recuerda del Apunte 03: como es una serie de tiempo, partimos **por fecha**, no al azar.

Pieza	Qué es	Tamaño real
X_train	pistas de mar-jul (para estudiar)	563.482 filas × 21 col.
y_train	respuestas de mar-jul	563.482
X_test	pistas de ago-sep (para el examen)	172.800 filas × 21 col.
y_test	respuestas de ago-sep (se esconden)	172.800

El modelo estudia con el 77 % (mar-jul) y lo examinamos con el 23 % (ago-sep) que nunca vio. Justo, como en producción: aprender del pasado para predecir el futuro.

3. ¿Cómo “piensa” un árbol? (mini-ejemplo de juguete)

Antes del dato real, un ejemplo diminuto de 6 horas inventadas, con *una* sola pista (el aire de una columna) y su % SiO₂:

Hora	Aire (Nm ³ /h)	% SiO ₂
1	280	4.5
2	290	4.2
3	295	3.9
4	310	1.8
5	320	1.5
6	330	1.2

Concepto

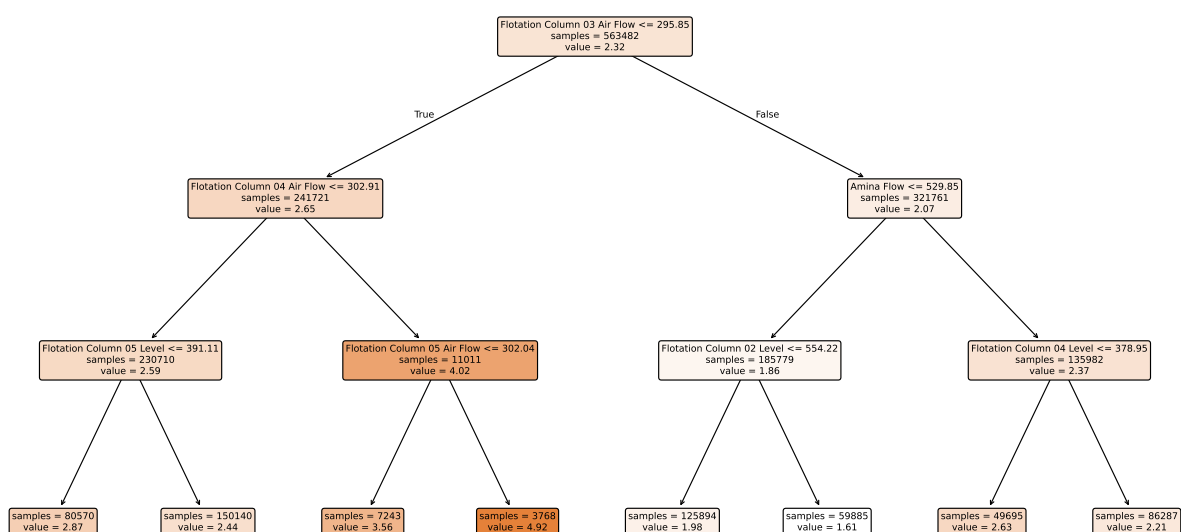
Un **árbol de decisión** hace una sola cosa, repetida: elige una pregunta de sí/no sobre una variable (¿aire ≤ 300 ?) que parta los datos en dos grupos **lo más parejos posible por dentro**. “Parejo” = que los % SiO₂ de cada grupo se parezcan entre sí (poca dispersión). Prueba **TODOS** los cortes posibles y se queda con el que deja los dos grupos menos revueltos.

En el juguete: si corta en **aire ≤ 300** , a la izquierda quedan {4,5,4,2,3,9} (todas altas, parecidas) y a la derecha {1,8,1,5,1,2} (todas bajas, parecidas). ¡Dos grupos tidísimos! Cualquier otro corte (p.ej. ≤ 285) dejaría un grupo revuelto (mezclando 4.2 con 1.2). Por eso el árbol elige ≤ 300 . Y a cada grupo le asigna como predicción el **promedio** de su % SiO₂ (4.2 y 1.5). Nadie le dijo que el aire importaba: lo descubrió midiendo.

4. El árbol de verdad, con tus datos

Ahora el algoritmo hace exactamente eso, pero probando las 21 pistas y repitiendo el corte **3 veces hacia abajo** (le pusimos profundidad = 3 para poder leerlo):

Tu primer arbol de decision (profundidad 3): se lee de arriba hacia abajo



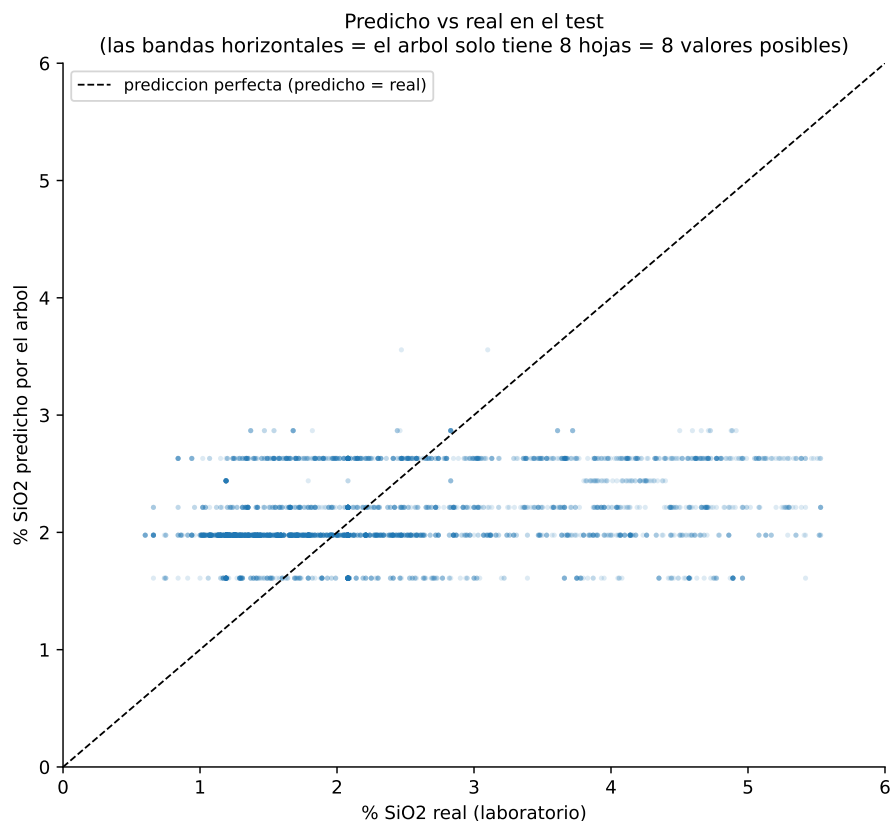
Se lee de arriba hacia abajo. La primera pregunta que el árbol eligió *solo*, entre las 21, fue sobre el **aire de una columna de flotación** — coherente con lo que vimos en correlación (Apunte

04), donde los flujos de aire eran de lo poco que se movía con la sílice. Cada caja dice **value** = el promedio de % SiO₂ de las filas que caen ahí. Las 8 cajas de abajo (las **hojas**) son las 8 respuestas posibles del modelo.

Glosario — en palabras simples

Nodo: cada caja con una pregunta. **Hoja:** las cajas del fondo, sin más preguntas; ahí el árbol entrega su predicción (el promedio del grupo). **Profundidad:** cuántas preguntas encadenadas hay de la copa a la hoja (aquí, 3).

5. La primera predicción...y una dosis de honestidad



Cada punto es una hora del examen: en el eje horizontal su % SiO₂ *real*, en el vertical el *predicho*. Si el modelo fuera perfecto, todos caerían sobre la línea punteada. Lo que ves es un modelo **tímido**: predice casi siempre entre 2 y 2.7, sin atreverse a los extremos. Las **bandas horizontales** son esas 8 hojas: un árbol de profundidad 3 solo puede dar 8 números distintos.

Ojo

Resultado honesto: este árbol es flojo —acierta poco más que si siempre dijeras “2.32” (el promedio). Explicación: con solo 3 preguntas es **demasiado simple** (a eso se le llama *subajuste*). No es un fracaso: es nuestra **línea base**, el número a superar. En el Apunte 08 aprenderemos a *medir* exactamente qué tan bueno es, en el 09 lo dejaremos crecer (y aparecerá el problema opuesto, *sobreajuste*), y en el 10 lo cambiaremos por un *bosque*. Ojo: muchos tutoriales de internet reportan aciertos altos en este dataset...porque parten al azar y hacen **fuga de datos**. Nuestro número humilde es el honesto.

Pruébalo

Pregunta de entrevista: “¿Qué es un árbol de decisión y cómo aprende sus reglas sin que un ingeniero se las programe?”

Respuesta modelo: “Es una cascada de preguntas de sí/no sobre las variables. El algoritmo prueba todos los cortes posibles y en cada paso elige el que parte los datos en dos grupos lo más homogéneos posible respecto al objetivo; repite hacia abajo y en cada hoja predice el promedio del grupo. Nadie programa las reglas: emergen de medir qué cortes separan mejor. Su gran ventaja es que se **lee** —puedo validar cada regla contra la fisicoquímica del proceso.”

La analogía metalúrgica: es un **operador con experiencia** razonando en voz alta: “¿el aire de la columna está bajo? entonces la sílice tiende a subir; ¿y además la amina va baja? entonces sube más...”. La diferencia con el operador es que al árbol *nadie* le enseñó esas reglas: las destiló de seis meses de datos, probando cortes.

Siguiente: Apunte 08 — cómo ponerle una nota justa a estas predicciones (RMSE, MAE, R^2).

Apunte 08 — ¿Cómo se califica una predicción? (RMSE, MAE, R^2)

Módulo 2 (Regresión) · Machine Learning desde cero

En el Apunte 07 dijimos “el árbol es flojo” mirándolo a ojo. Eso no basta: necesitamos un **número** que le ponga nota, para poder comparar modelos sin discutir. Tres notas se usan siempre en regresión. Vamos con calma.

Glosario — en palabras simples

- **Error (o residuo):** lo que te equivocaste en una fila = predicho – real.
- **MAE:** el promedio de los errores *en valor absoluto* (sin signo).
- **RMSE:** parecido, pero *castiga más* los errores grandes.
- **R^2 :** qué fracción de la variación del objetivo logra explicar el modelo, comparado con simplemente adivinar el promedio.
- **Baseline:** el “modelo tonto” de referencia: predecir siempre el promedio.

1. Un mini-ejemplo con 3 predicciones

Imagina que el modelo predijo 3 horas y ya sabemos el valor real de lab:

Real	Predicho	Error (pred–real)	Error ²
2.0	2.2	+0,2	0.04
3.0	2.8	–0,2	0.04
5.0	4.5	–0,5	0.25

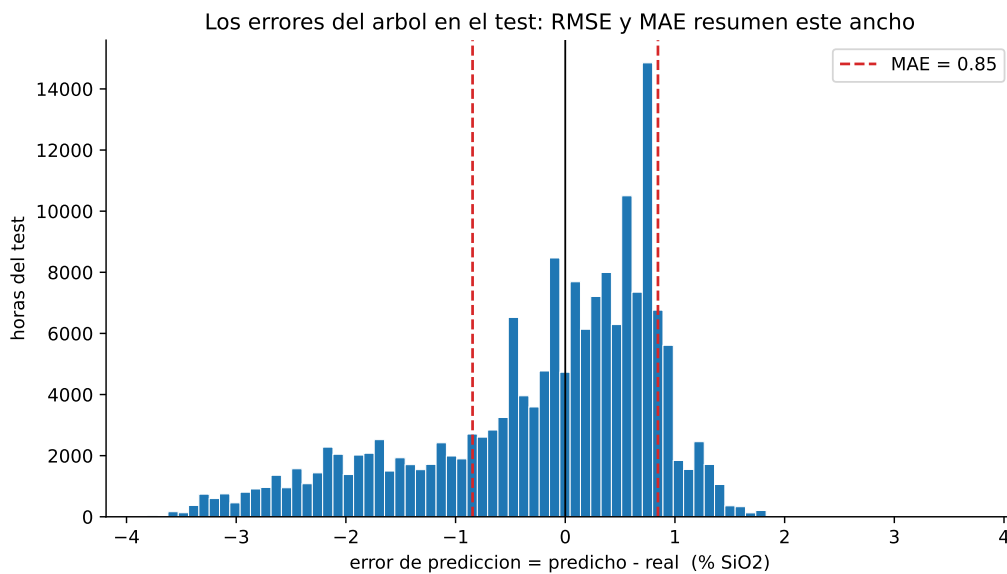
Concepto

MAE = promedio de $|\text{error}| = (0,2 + 0,2 + 0,5)/3 = \mathbf{0,30}$. Se lee directo: “en promedio me equivoco 0.30 puntos de % SiO_2 ”.

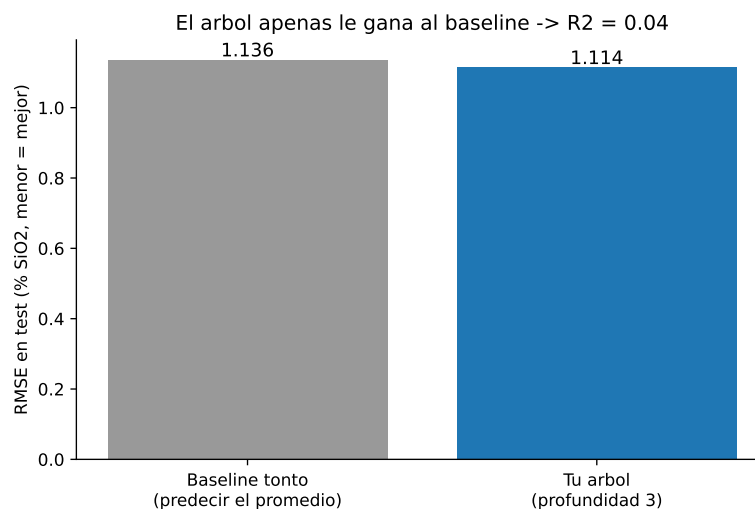
RMSE = raíz del promedio de los errores **al cuadrado** = $\sqrt{(0,04 + 0,04 + 0,25)/3} = \sqrt{0,11} = \mathbf{0,33}$. Como eleva al cuadrado antes de promediar, un error grande (el 0.5) pesa más: por eso $\text{RMSE} \geq \text{MAE}$, y la diferencia crece cuando hay errores gruesos.

R^2 : el promedio real es 3,33. Si adivinaras siempre 3,33 (el baseline), tu “error total al cuadrado” sería 4,67. El del modelo es 0,33. Entonces $R^2 = 1 - 0,33/4,67 = \mathbf{0,93}$: el modelo explica el 93 % de la variación. 1 = perfecto; 0 = igual que adivinar el promedio; *negativo* = peor que el promedio.

2. Las notas de tu árbol (Apunte 07), de verdad



El histograma son los errores del árbol en las 172.800 horas del test. La mayoría se concentran cerca de 0 (bien), pero hay una cola larga de errores de -2 , -3 (el modelo tímido se queda corto cuando la sílice real es alta). Resumido: **MAE = 0.85** y **RMSE = 1.11** puntos de % SiO₂.



Aquí se ve el R^2 con los ojos: el “baseline tonto” (predecir siempre 2.32) tiene RMSE 1,136; tu árbol, 1,114. Le gana...por un pelo. Ese “pelo” es **$R^2 = 0.04$** : el árbol de profundidad 3 explica apenas el 4% de la variación. Confirma con número lo que en el 07 vimos a ojo.

Ojo

Ninguna nota sola cuenta toda la historia. MAE es la más fácil de explicarle a un gerente (“ ± 0.85 ”). RMSE es la que reportarás si los errores grandes son los que duelen (mandar concentrado muy fuera de especificación). R^2 te dice si vale la pena frente a no tener modelo. Y **R^2 puede salir negativo**: si tu modelo predice peor que el promedio, eso es lo que significa. Siempre miramos las tres, y siempre en el **test**, nunca en el train.

Pruébalo

Pregunta de entrevista: “¿Por qué reportas $RMSE$ y MAE , si miden casi lo mismo? ¿Y qué significa un R^2 de 0.04?”

Respuesta modelo: “ MAE es el error típico en las unidades del problema, fácil de comunicar. $RMSE$ castiga más los errores grandes porque los eleva al cuadrado; si $RMSE$ es bastante mayor que MAE , hay predicciones muy desviadas que conviene revisar. Un R^2 de 0.04 significa que el modelo explica solo el 4 % de la variabilidad —apenas mejor que predecir el promedio: es una línea base pobre, no un modelo listo para producción.”

La analogía metalúrgica: son las notas de un **ensayo de laboratorio**. MAE es la desviación típica de tu ensayo frente al valor certificado; $RMSE$ es esa desviación pero penalizando el ensayo que se fue *muy* lejos (un blooper cuesta más que tres pequeños sesgos); y R^2 es cuánto mejor es tu método que simplemente reportar la ley promedio del yacimiento para toda muestra. Un método con R^2 bajo...mejor sigue en desarrollo.

Siguiente: Apunte 09 — dejamos crecer el árbol y vemos aparecer, en vivo, el **sobreajuste**.

Apunte 09 — Sobreajuste, en vivo con tus datos

Módulo 2 (Regresión) · Machine Learning desde cero

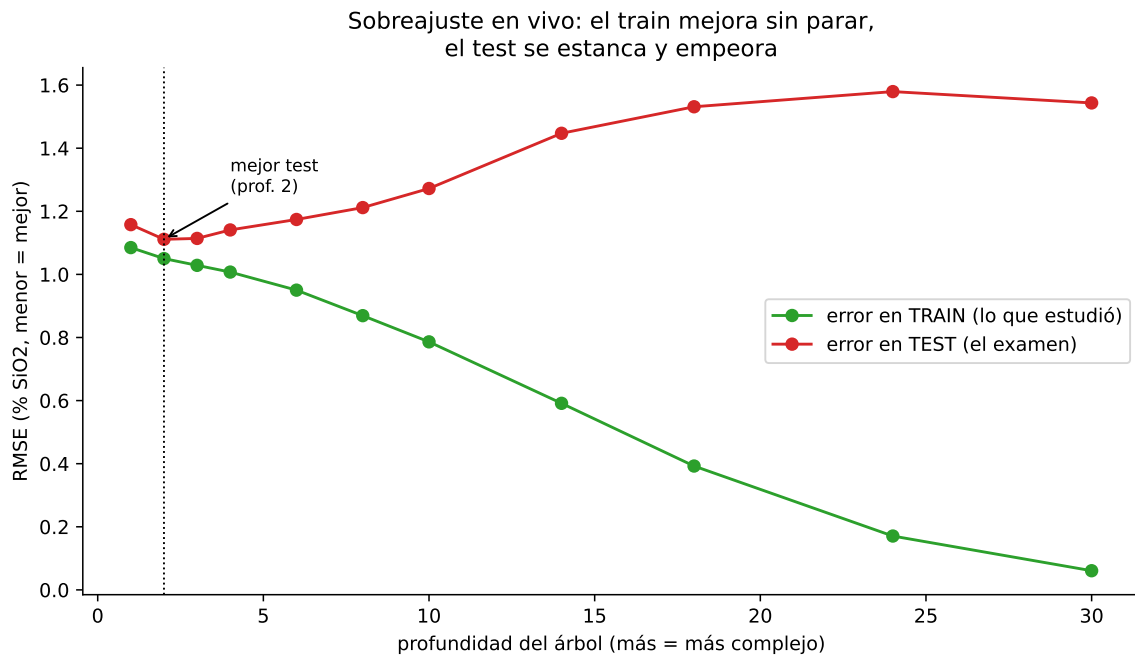
En el Módulo 0 dijimos que el **sobreajuste** es el concepto más importante de todos. Ahora lo vamos a *ver* pasar, con tu árbol y tus datos. La perilla que lo controla es la **profundidad**: cuántas preguntas encadenadas dejamos que haga el árbol.

Glosario — en palabras simples

- **Subajuste:** el modelo es demasiado simple y ni siquiera captura el patrón (como el árbol de profundidad 3: rinde mal en *todo*).
- **Sobreajuste:** el modelo es tan complejo que se *memoriza* el ruido de los datos con que estudió; brilla en el train y falla en el test.
- **Generalizar:** rendir bien con datos *nuevos*. Es lo único que importa.
- **Curva de validación:** gráfica del error en train y en test a medida que subimos la complejidad. Es el instrumento para elegir la perilla.

1. La perilla de la complejidad

Un árbol de profundidad 1 hace *una* pregunta: demasiado tosco. Uno de profundidad 30 puede hacer una rama por cada puñado de filas: memoriza. En algún punto intermedio está el mejor equilibrio. ¿Cómo lo encontramos? No lo adivinamos: lo **medimos**, entrenando el árbol a muchas profundidades y calificando cada uno con el RMSE del Apunte 08.



Concepto

Lee las dos curvas. La **verde (train)** baja sin parar: cuanto más profundidad, mejor “recita” lo que estudió —hasta casi 0 en profundidad 30 (se lo aprendió de memoria). Pero la **roja (test)** —lo único que importa— toca fondo tempranísimo (profundidad 2) y de ahí en adelante **empeora**. Esa **separación** que se abre entre verde y roja **es** el sobreajuste: el modelo dejó de aprender el proceso y empezó a memorizar el ruido de marzo–julio, que no sirve para agosto–septiembre.

2. El punto dulce...y una verdad incómoda

El mejor árbol en el test es uno **muy poco profundo** (profundidad 2), con RMSE 1,111. Ahora la verdad incómoda: ese “mejor” árbol apenas le gana al baseline tonto (1,136). Traducción: **un solo árbol no da para más** en este problema —sea simple (subajusta) o profundo (sobreajusta). No es que lo hayamos configurado mal; es el techo de un árbol individual. Justo por eso existe lo que viene en el Apunte 10: juntar *muchos* árboles.

Ojo

El sobreajuste es **invisible** si solo miras el train: ahí el modelo profundo se ve *perfecto* (RMSE 0,06). Recién al examinarlo con datos que nunca vio (test) se destapa el desastre (1,5). **Por eso apartamos siempre un test** —y en serie de tiempo, uno *temporal* (Apunte 03/05). Un modelo que solo reportas en train es un modelo en el que no puedes confiar.

Pruébalo

Pregunta de entrevista: “*Tu árbol tiene $R^2 = 0.98$ en entrenamiento. ¿Está listo? ¿Cómo eliges su profundidad?*”

Respuesta modelo: “No, un R^2 altísimo en entrenamiento es sospechoso de sobreajuste; solo cuenta el desempeño en un test que el modelo nunca vio. Para elegir la profundidad grafico la **curva de validación**: el error en train siempre baja al subir la complejidad, pero busco el punto donde el error de *test* es mínimo, antes de que empiece a subir. En estos datos ese punto es un árbol muy poco profundo, y aun así apenas supera al baseline —señal de que necesito un modelo más potente, como un random forest.”

La analogía metalúrgica: un modelo sobreajustado es el que **calibraste tan fino a un único lote de mineral** que predice ese lote de memoria...y se cae con el mineral del día siguiente. El objetivo nunca es clavar el lote de calibración: es que el modelo siga sirviendo cuando cambie la alimentación. Un modelo demasiado “ajustado” a una campaña es tan inútil como uno demasiado tosco.

Siguiente: Apunte 10 — si un árbol no da para más, juntemos cientos: el **random forest**.

Apunte 10 — Random forest: el comité de árboles

Módulo 2 (Regresión) · Machine Learning desde cero

Un árbol solo tocó techo (Apunte 09). La idea clásica para superarlo no es hacer un árbol mejor, sino **muchos árboles imperfectos y promediarlos**. Eso es un *random forest* (bosque aleatorio). Veremos qué es...y un resultado honesto que sorprende.

Glosario — en palabras simples

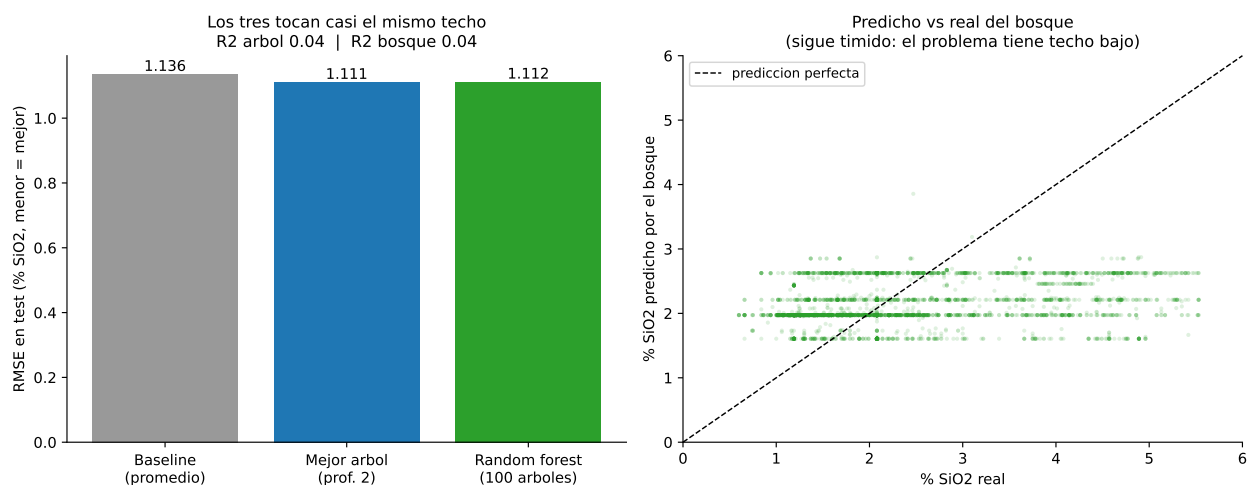
- **Random forest:** un conjunto de muchos árboles (aquí 100) cuyas predicciones se promedian.
- **Bootstrap:** a cada árbol se le da una muestra al azar de las filas (con repetición), así ninguno ve exactamente los mismos datos.
- **Subconjunto de variables:** en cada corte, cada árbol solo puede elegir entre un puñado de columnas al azar. Así los árboles salen *distintos*.

1. Por qué promediar suele ayudar

Concepto

Si entrenas 100 árboles ligeramente distintos (cada uno con su muestra y sus variables al azar), cada uno se equivoca en cosas diferentes. Al **promediar** sus predicciones, los errores aleatorios de unos cancelan los de otros y queda una predicción más estable. Es el mismo principio por el que promedias **réplicas de un análisis químico**: cada medición tiene error aleatorio, pero el promedio de 30 réplicas es mucho más confiable que cualquiera suelta.

2. El comité contra el árbol...empate



Aquí la sorpresa honesta: en *nuestro* problema, el bosque ($R^2 = 0,04$) **no le gana** al mejor árbol ($R^2 = 0,04$). Ambos rozan el mismo techo, apenas por encima del baseline. El gráfico de la derecha lo confirma: el bosque sigue igual de tímido.

Ojo

Y hay más: un bosque **sin podar** (árboles profundos) rinde *peor* que el baseline, porque cada árbol memoriza marzo–julio (sobreajuste, Apunte 09). Tuvimos que limitarle la profundidad, eligiéndola con datos:

Profundidad del bosque	R^2 en test
3 (elegida)	+0,04
5	−0,06
9	−0,12
sin límite	−0,25

Moraleja: el random forest **no es magia**; también necesita control de complejidad.

3. ¿Por qué aquí no ayudó?

El promedio del comité arregla el error **aleatorio** (la inestabilidad del árbol). Pero nuestro problema no falla por inestabilidad: falla porque **predecir un mes futuro** a partir de estas variables es *genuinamente difícil* —el proceso cambia entre julio y septiembre, y esa parte no está en las pistas. Promediar más árboles no crea información que no existe. El techo lo pone el problema, no el algoritmo.

Pruébalo

Pregunta de entrevista: “¿Por qué un random forest suele ganarle a un árbol solo, y por qué a veces no?”

Respuesta modelo: “Suele ganar porque promedia muchos árboles diversos y cancela el error aleatorio de cada uno —reduce la varianza. Pero si el problema es difícil por falta de señal (no por inestabilidad), promediar no ayuda: el techo lo marca la información disponible, no el modelo. Y ojo: un bosque de árboles sobrecrecidos igual sobreajusta; hay que controlar su complejidad, elegida sobre un test honesto.”

La analogía metalúrgica: promediar 100 árboles es como promediar 100 réplicas de un ensayo para bajar el error *aleatorio*. Pero si tu método tiene un **sesgo** o la muestra no contiene la información que buscas, mil réplicas no lo arreglan —solo te dan un promedio muy preciso de un número equivocado. El comité ayuda con el ruido, no con la ignorancia.

Siguiente: Apunte 11 — aunque el modelo sea humilde, ¿en qué variables se apoya? (importancia de variables).

Apunte 11 — ¿Qué variables importan? (feature importance)

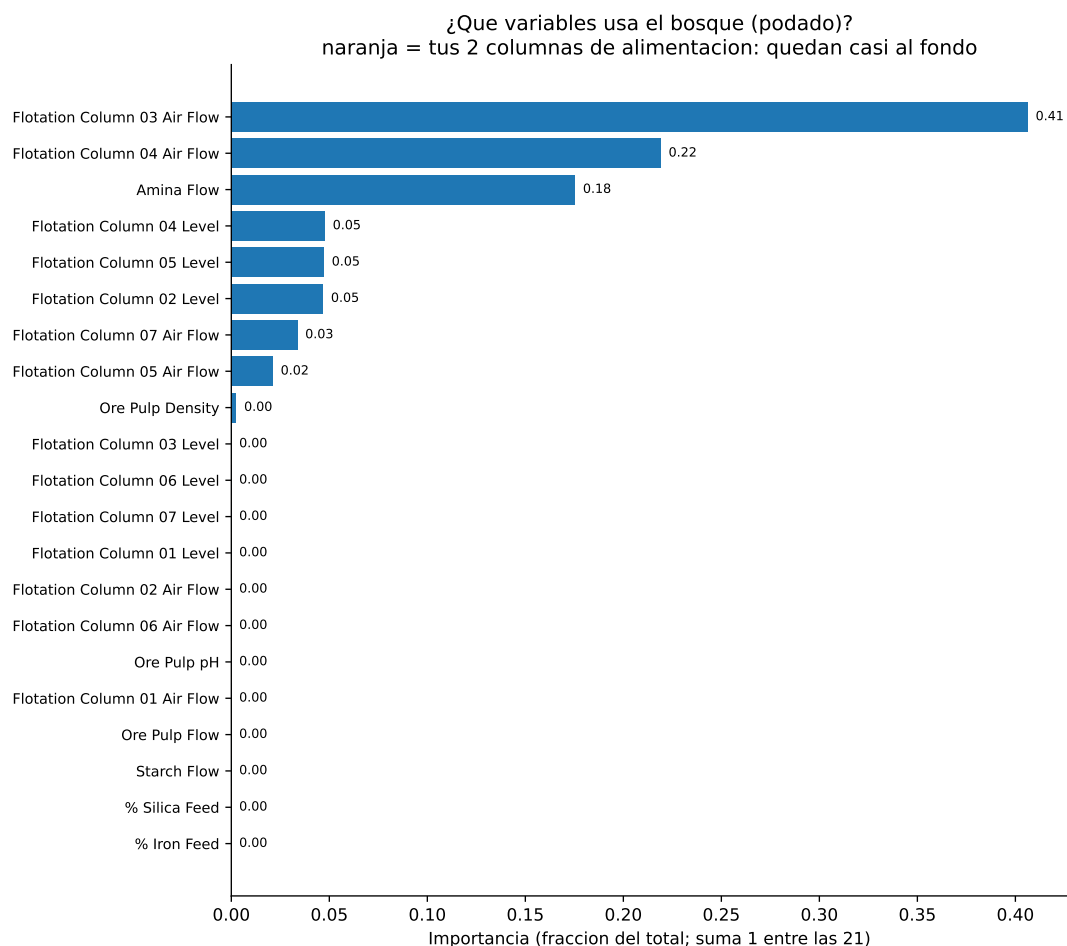
Módulo 2 (Regresión) · Machine Learning desde cero

Aunque el modelo sea humilde, puede decirnos algo valioso: **en qué variables se apoyó**. Este era tu pedido —medir cuánto valen las 2 columnas de alimentación— y la respuesta trae una lección honesta.

Glosario — en palabras simples

Importancia de variables (feature importance): un número por columna que dice cuánto ayudó esa variable a mejorar los cortes del bosque. Las 21 importancias **suman 1**: es un reparto del “mérito” total.

1. En qué se apoya el bosque



Concepto

El bosque se apoya casi por completo en **tres** variables: el **aire de las columnas 3 y 4** y el **flujo de amina**. Y esto **concuerda** con todo lo anterior: en el Apunte 04 los flujos de

aire eran de lo que más correlacionaba con la sílice, y el árbol del Apunte 07 también partió primero por el aire de una columna. Tres pistas independientes apuntando al mismo lugar: buena señal de que es real.

2. La lección honesta (y tu decisión)

Tus 2 columnas de alimentación (% Fe y % SiO₂ feed) quedaron **al fondo**, con importancia casi 0. Es coherente con el Módulo 1, donde su correlación con la sílice era $|r| < 0,08$. **Traducción de tu decisión de conservarlas:** no hicieron daño, pero el modelo que generaliza casi no las usa. Mantenerlas fue una apuesta razonable —“que el modelo decida”— y el modelo decidió que aportan poco. Eso *también* es un resultado.

Ojo

Tres advertencias para no sobreinterpretar este ranking:

- **Importancia $\neq$ relevancia física.** El bosque no usa el pH...y el pH es centralísimo en la flotación. No es que el pH no importe al proceso: es que, para predecir *este* test futuro, el aire ya captura la poca señal aprovechable y al modelo (poco profundo) no le quedan cortes para el pH.
- **Depende del modelo.** Un bosque más profundo repartiría la importancia entre más variables (subirían pH y hasta las de feed)...pero ese modelo sobreajusta (Apunte 10). Cambias el modelo, cambia el ranking.
- **El modelo apenas le gana al baseline** ($R^2 = 0,04$). Cuando un modelo predice poco mejor que el promedio, su ranking de importancia hay que leerlo **con pinzas**: está repartiendo mérito de una señal chica.

Pruébalo

Pregunta de entrevista: “*Tu modelo dice que el pH tiene importancia 0. ¿Concluyes que el pH no afecta la sílice?*”

Respuesta modelo: “No. La importancia mide cuánto usó el modelo esa variable para sus cortes, no su papel físico. Un modelo poco profundo gasta sus pocos cortes en las variables más fuertes (los flujos de aire) y puede ignorar otras que sí importan al proceso, como el pH. Además la importancia depende del modelo y, si el R^2 es bajo, es poco confiable. Para afirmar causalidad hago un experimento controlado, no leo un ranking de datos observacionales — donde además los lazos de control confunden (Apunte 04).”

La analogía metalúrgica: es como rankear qué palancas mueven más la ley mirando el **historial de planta:** te dice cuáles variaron junto con la ley, no cuáles la *causan*. Para eso harías una prueba controlada moviendo una palanca a la vez. El ranking es una brújula para investigar, no un veredicto.

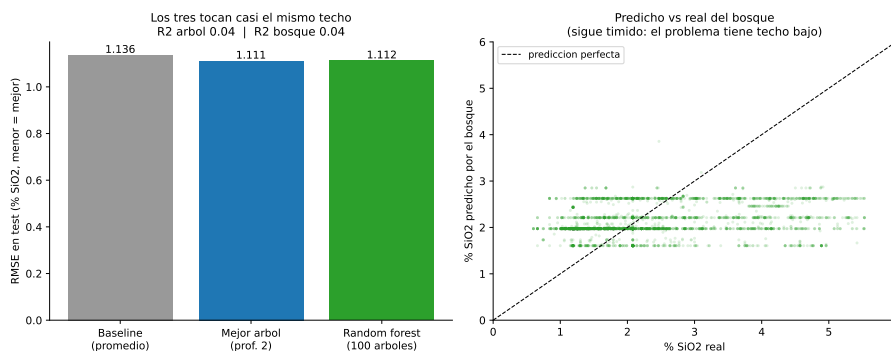
Siguiente: Apunte 12 — cerramos el Módulo 2: qué aprendió (y qué no) el modelo, sin maquillaje.

Apunte 12 — Qué aprendió el modelo (cierre del Módulo 2)

Módulo 2 (Regresión) · Machine Learning desde cero

Empezamos el Módulo 2 queriendo predecir el % de SiO_2 del concentrado en tiempo real. Construimos un árbol, aprendimos a calificarlo, vimos el sobreajuste, montamos un bosque y medimos en qué se apoya. Toca el resumen honesto: **¿qué aprendió, de verdad?**

1. El resultado, sin maquillaje



En un examen **honesto** (entrenar con mar-jul, probar con ago-sep, sin fuga de datos), el mejor modelo —árbol o bosque— alcanza $R^2 \approx 0,04$: apenas mejor que predecir siempre el promedio. No es un bug ni mala configuración: es el **techo real** de este problema con estos datos.

Concepto

Dos razones del techo bajo: (1) **el proceso cambia con el tiempo** —lo que sirvió en mayo no describe del todo a septiembre, y esa deriva no está en las pistas; (2) el objetivo depende de cosas que **no medimos**. Ningún algoritmo inventa información que no está en los datos. La potencia del modelo no era el cuello de botella.

Ojo

¿Y los tutoriales que muestran $R^2 = 0,9$ con este mismo dataset? Casi todos parten las filas **al azar**, y como cada hora tiene ~ 180 filas casi gemelas, el examen se llena de copias del estudio: **fuga de datos** (Apuntes 03 y 05). Su 0.9 es un espejismo; en producción ese modelo se caería. Nuestro 0.04 es humilde pero **honesto**, y por eso vale mucho más.

2. Lo que sí aprendió

Aunque predice poco, el modelo es **consistente** en dónde está la (poca) señal: el **aire de las columnas 3–4** y el **flujo de amina**. Tres métodos independientes coinciden —la correlación del Módulo 1, el primer corte del árbol y la importancia del bosque— lo cual da confianza en que es real, no casualidad. Y confirmó, con número, que las 2 columnas de alimentación aportan poco (tu decisión de conservarlas: sin daño, sin gloria).

3. El recorrido del módulo

#	Tema	Lo que te llevas
07	Árbol de decisión	cómo un modelo saca reglas de los datos, y se leen
08	Métricas	RMSE/MAE/R ² : ponerle una nota objetiva a una predicción
09	Sobreajuste	memorizar $\neq$ aprender; elegir complejidad con un test
10	Random forest	promediar árboles reduce el ruido, no la ignorancia
11	Importancia	en qué se apoya el modelo; importancia $\neq$ causa

4. Cómo se sube el techo (Módulos que vienen)

El 0.04 no es el final: es la línea base honesta. Formas realistas de mejorarlo, para más adelante: **ingeniería de variables** (agregar promedios móviles y valores de horas previas, porque el proceso tiene inercia), **re-entrenar seguido** para seguir la deriva, predecir a un **horizonte más corto**, o modelos más potentes (gradient boosting). El Módulo 3 abrirá la caja negra por predicción (SHAP) y el Módulo 4 buscará los estados operativos con k-means.

Pruébalo

Pregunta de entrevista: “*Tu modelo final tiene $R^2 = 0,04$. ¿El proyecto fracasó?*”

Respuesta modelo: “No. El proyecto entregó algo más valioso que un número bonito: una evaluación **honest**a, sin fuga de datos, que revela la dificultad real del problema y evita desplegar un modelo que brillaría en pruebas y fallaría en planta. Además identificó dónde está la señal (aire y amina) y dejó un camino claro para mejorar (variables con memoria, re-entrenamiento, horizonte más corto). Un resultado modesto bien medido es ingeniería; un resultado inflado por leakage es una trampa esperando a estallar.”

La analogía metalúrgica: es un **balance metalúrgico honesto**. Un reporte de reconciliación que admite que la recuperación real fue 88 % —y no el 95 % del diseño— es incómodo, pero es el único sobre el que puedes tomar decisiones. Un reporte maquillado te deja feliz...hasta que el inventario no cuadra. En ML, el “inventario que no cuadra” es el modelo fallando en producción.

Módulo 2: cerrado. Doce apuntes en tu biblioteca. Ya sabes construir, medir y —sobre todo— **no engañarte** con un modelo. Eso es lo que separa a un ingeniero de ML de alguien que solo corre `.fit()`.

Módulo 3 — Cómo aprenden los modelos

Apunte 13 — Regresión lineal (y por qué empezar por aquí)

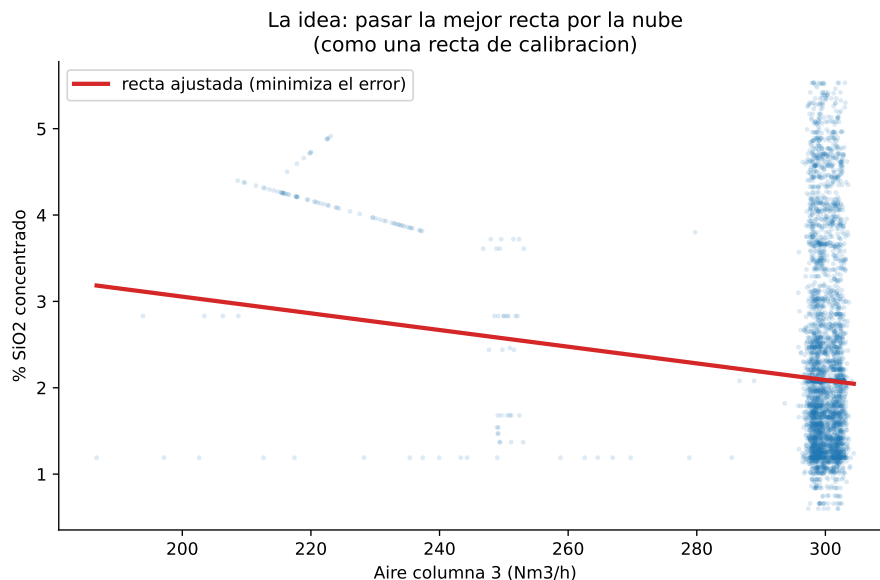
Módulo 3 (Cómo aprenden los modelos) · Machine Learning desde cero

Los árboles (Módulo 2) parten los datos con preguntas sí/no. Ahora conocemos la otra gran familia, la más antigua y la base de casi todo el ML moderno: **ajustar una recta**. Es el modelo que en la mayoría de cursos se ve *primero*; lo vemos ahora que ya tienes criterio, para que lo aprecies.

Glosario — en palabras simples

- **Regresión lineal:** predice con una **recta** (o su versión con muchas variables, un *plano*): $\text{predicción} = \text{intercepto} + c_1x_1 + c_2x_2 + \dots$
- **Coefficiente (c_i):** cuánto y en qué sentido empuja cada variable la predicción. Positivo = la sube; negativo = la baja.
- **Intercepto:** el valor de partida cuando todas las variables valen su promedio.
- **Estandarizar (escalar):** poner todas las variables en la misma escala (restar la media, dividir por la desviación). Sin esto, los coeficientes no se pueden comparar (el almidón va en miles, el pH en unidades). Lo veremos a fondo en el Apunte 15.

1. La idea: la mejor recta por la nube



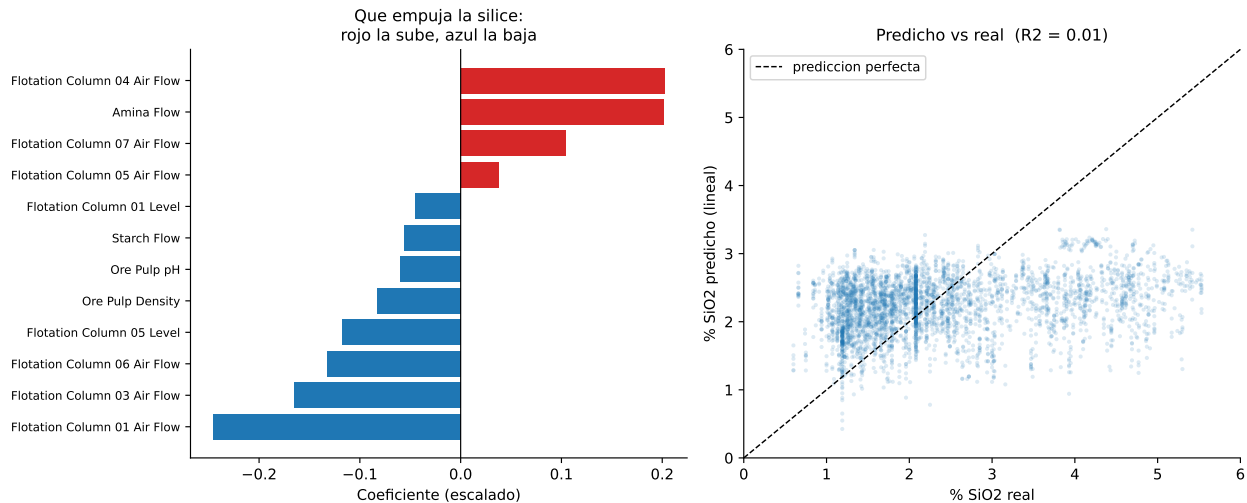
Concepto

Con *una* variable, el modelo busca la **recta que pasa lo mejor posible** por la nube de puntos. “Lo mejor posible” tiene un significado exacto: la recta que hace **lo más pequeño posible el error al cuadrado** (el RMSE del Apunte 08). Entre todas las rectas imaginables, elige la de menor error total. Con 21 variables es lo mismo, pero en vez de una recta es un

plano en muchas dimensiones —imposible de dibujar, idéntico de concepto.

La recta de arriba baja: más aire en la columna 3 → algo menos de sílice. Es exactamente el tipo de relación que la correlación del Apunte 04 ya insinuaba.

2. La gran ventaja: los coeficientes se leen



A diferencia de un bosque (una caja negra de 100 árboles), un modelo lineal te entrega un **coeficiente por variable**: rojo la sube, azul la baja. Aquí el aire de las columnas 1, 3 y 6 baja la sílice, mientras el de la columna 4 y la **amina** la suben. Ese signo positivo de la amina es el mismo **eco del lazo de control** del Apunte 04 (suben la amina cuando la sílice ya está alta): la recta lo hereda igual que la correlación.

Ojo

Pero mira la derecha: $R^2 = 0.01$, **aún más flojo** que el árbol (0.04). ¿Por qué? Porque una recta solo captura relaciones **derechas**. La flotación tiene respuestas **curvas** y combinaciones (interacciones) que una recta no puede doblar. Ahí los árboles le ganan a la recta. **Moraleja clave**: ningún modelo es el mejor siempre —lineal brilla cuando la relación es recta e interpretable; los árboles cuando es curva. Y ojo: los signos son **observacionales** (con lazos de control de por medio), no causales.

3. ¿Cómo encuentra la mejor recta?

Dijimos “elige la recta de menor error”, pero ¿cómo la *encuentra* entre infinitas? No las prueba todas. Usa un procedimiento llamado **descenso de gradiente**, el verdadero motor de casi todo el machine learning —y es justo el Apunte 14.

Pruébalo

Pregunta de entrevista: “¿Cuándo preferirías una regresión lineal sobre un random forest?”

Respuesta modelo: “Cuando necesito **interpretabilidad** —un coeficiente por variable que pueda explicar y auditar— y cuando la relación es aproximadamente lineal. También como **baseline** rápido: si un modelo complejo no le gana claramente a una recta, quizá el problema no da para más o hay algo mal. Su límite es que solo modela relaciones rectas; si el fenómeno

es curvo o con interacciones, un árbol o un bosque capturan más.”

La analogía metalúrgica: la regresión lineal es una **recta de calibración** multivariable —relacionas la señal del instrumento con la ley por una recta. Simple y transparente, pero si la respuesta del instrumento se **satura** o es curva, una recta calibra mal y necesitas un modelo más flexible. Igual aquí.

Siguiente: Apunte 14 — **descenso de gradiente:** cómo un modelo “baja la montaña” del error para encontrar sus números. El motor de todo.

Apunte 14 — Descenso de gradiente: el motor de todo

Módulo 3 (Cómo aprenden los modelos) · Machine Learning desde cero

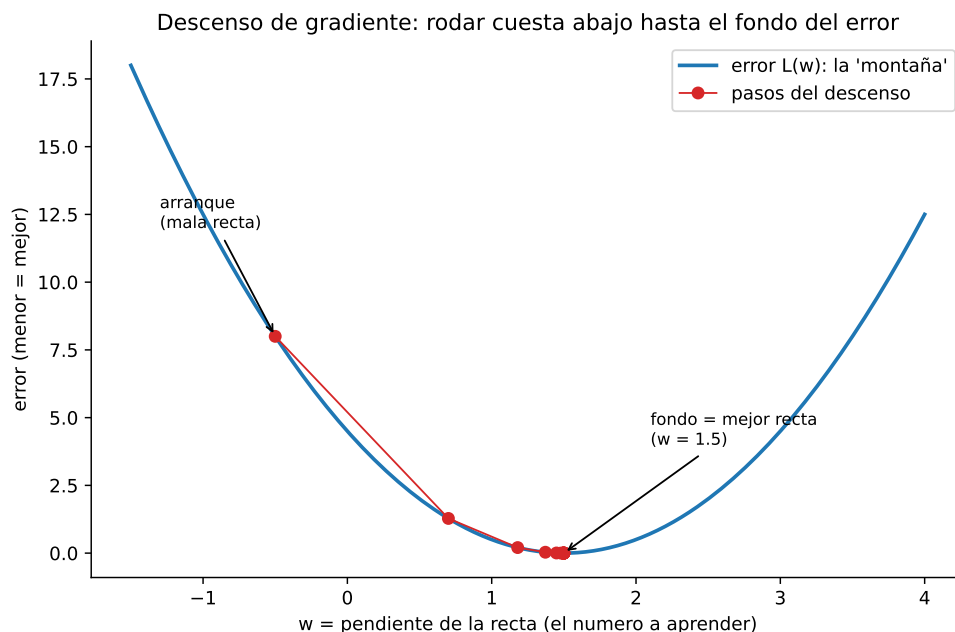
En el Apunte 13 dijimos que la recta “elige los coeficientes que minimizan el error”. Pero ¿cómo los *encuentra*, si hay infinitas rectas posibles? No las prueba todas. Usa el **descenso de gradiente**: el mismo motor que entrena la regresión lineal, el gradient boosting y las redes neuronales. Si entiendes esto, entiendes cómo “aprende” casi todo el machine learning.

Glosario — en palabras simples

- **Función de pérdida (loss):** un número que mide qué tan mal predice el modelo (p.ej. el error al cuadrado del Apunte 08). Aprender = hacerlo lo más pequeño posible.
- **Gradiente:** la *pendiente* de esa pérdida —hacia dónde sube. Ir en contra es ir cuesta abajo.
- **Tasa de aprendizaje (learning rate):** el tamaño de cada paso.
- **Iteración:** cada paso del descenso. Se repite hasta llegar al fondo.

1. La imagen: bajar al fondo de un valle

Imagina el error como un **valle**. Tu posición horizontal es el número que buscas (la pendiente de la recta). La altura es el error. Aprender = **llegar al fondo**, donde el error es mínimo. Estás con los ojos vendados: solo sientes la **inclinación** del suelo bajo tus pies (el gradiente) y das un paso cuesta abajo. Repites hasta que el suelo se aplana: llegaste.



Concepto

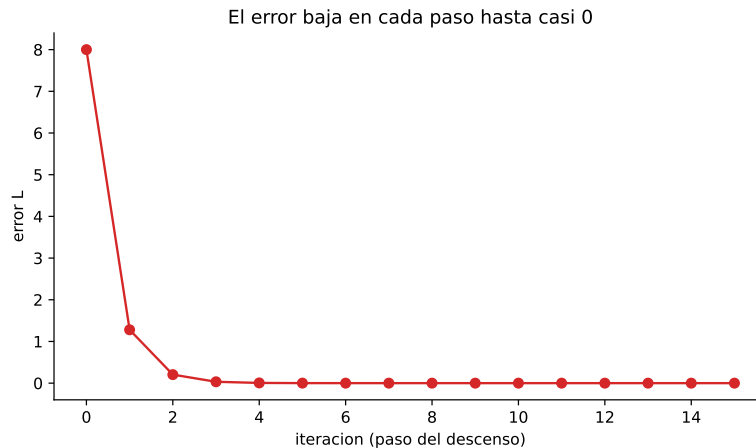
El algoritmo, en 3 pasos que se repiten:

1. **Arranca** con un valor cualquiera (aquí, una pendiente $w = -0,5$: mala recta, error

$= 8$).

2. **Mide la pendiente** del error en ese punto (el gradiente).
3. **Da un paso en contra** de esa pendiente, del tamaño de la tasa de aprendizaje.

Repíte. Cada paso baja el error, y los pasos se **achican solos** cerca del fondo (la pendiente es menor). Aquí, en ~ 10 pasos, llega a $w = 1,5$ con error ≈ 0 : la mejor recta.



2. Un paso, con números

En el arranque ($w = -0,5$) la pendiente del error apunta “hacia abajo-derecha”. El paso:

$$w_{\text{nuevo}} = w - (\text{tasa}) \times (\text{gradiente}) = -0,5 - 0,15 \times (-8) = 0,7.$$

El error cae de 8 a $\sim 1,3$. Otro paso, otro, y en una decena de iteraciones el valor se asienta en 1,5. Eso es todo el “aprendizaje”: restar, un pasito a la vez, hasta el fondo.

Ojo

La **tasa de aprendizaje** es delicada: si es **muy grande**, el paso se pasa del fondo y rebota (o se dispara); si es **muy chica**, tarda un mundo. Además, para la regresión lineal el valle es un tazón perfecto (un único fondo), pero en modelos complejos (redes) el terreno tiene **hoyos y lomas**: el descenso puede quedarse en un fondo que no es el más bajo. Por eso entrenar modelos grandes es, en parte, un arte de elegir bien estos ajustes.

Pruébalo

Pregunta de entrevista: “Explica el descenso de gradiente y qué papel juega la tasa de aprendizaje.”

Respuesta modelo: “Es un método para minimizar la función de pérdida: se arranca con parámetros cualquiera, se calcula el gradiente —la dirección en que la pérdida sube— y se da un paso en sentido contrario; se repite hasta converger. La tasa de aprendizaje fija el tamaño del paso: muy alta y oscila o diverge, muy baja y tarda demasiado. Es el motor que entrena regresión lineal, boosting y redes neuronales; los árboles no lo usan, crecen por cortes.”

La analogía metalúrgica: es **optimizar un setpoint de planta a tientas**. No sabes la dosis exacta de reactivo para la mejor recuperación, así que la ajustas un poco, miras si la recuperación mejoró, y sigues empujando en la dirección que mejora —pasos pequeños hasta que ya no puedes mejorar. Si mueves demasiado de golpe, desestabilizas la planta (tasa muy

alta); si mueves de a microgramos, no acabas nunca (tasa muy baja).

Siguiente: Apunte 15 — por qué hay que **escalar** las variables y qué es la **regularización** (evitar que los coeficientes se desboquen).

Apunte 15 — Escalar las variables y regularizar

Módulo 3 (Cómo aprenden los modelos) · Machine Learning desde cero

Cerramos el Módulo 3 con dos herramientas que acompañan a casi todo modelo lineal (y a muchos otros): **escalar** las variables y **regularizar**.

Glosario — en palabras simples

- **Escalar / estandarizar:** llevar cada variable a la misma escala (restar su media, dividir por su desviación), para que todas “pesen” parejo.
- **Regularizar:** añadir un castigo a los coeficientes grandes, para que el modelo no se desboque. **Ridge** los encoge; **Lasso** puede ponerlos en **cero** (selección automática de variables).
- **Alpha:** la perilla que fija cuánta regularización aplicas.

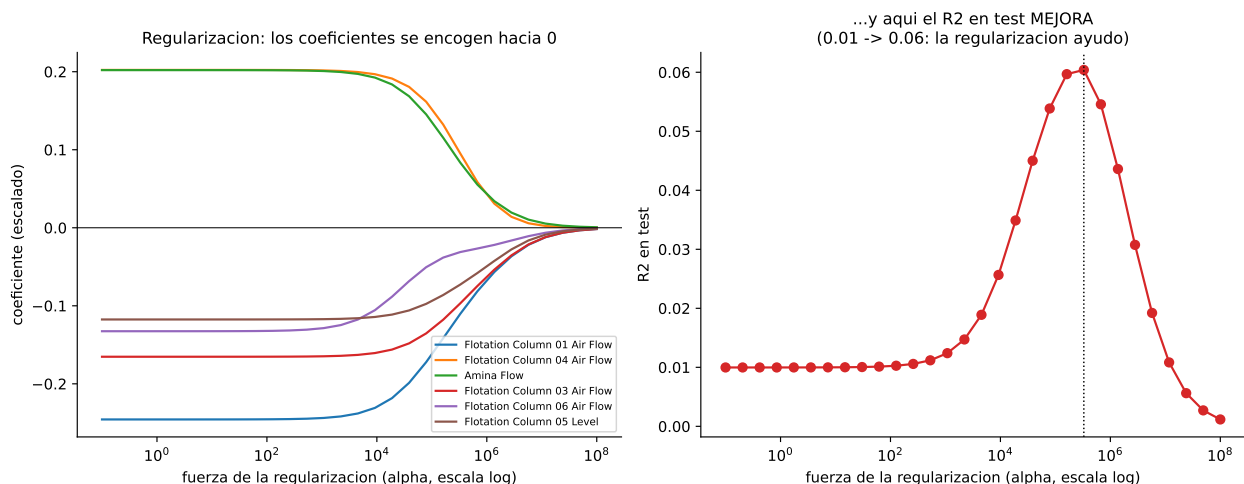
1. Escalar: que todas las variables jueguen parejo

Concepto

Tus variables viven en escalas dispares: el almidón en *miles*, el pH entre 8.7 y 10.8. Sin escalar, pasan dos cosas malas: (1) los coeficientes no se pueden **comparar** (por eso en el Apunte 13 escalamos antes de leerlos); y (2) el descenso de gradiente y todo lo que mide **distancias** (como el k-means del Módulo 8) se dejan **dominar** por la variable de números más grandes, aunque no sea la más importante. Estandarizar pone a todas en la misma cancha.

2. Regularizar: castigar coeficientes desbocados

Un modelo con coeficientes enormes suele estar **sobreajustando** —reacciona exageradamente a cada variable. La regularización añade a la pérdida un castigo por coeficientes grandes, obligando al modelo a ser más **prudente**.



Concepto

A la izquierda: al subir **alpha**, todos los coeficientes se **encogen hacia 0**. A la derecha, el resultado honesto sobre *tus* datos: el R^2 en test **mejora** de 0,01 a $\sim 0,06$ —el mejor del proyecto hasta ahora— y luego **vuelve a caer** si te pasas (coeficientes casi 0 = predecir el promedio = subajuste). Hay un **punto dulce**, igual que con la profundidad del árbol (Apunte 09): ni muy flojo ni muy apretado.

Por qué ayudó aquí: la recta sin regularizar confiaba de más en coeficientes ajustados a marzo–julio; encogerlos la hace más **conservadora** y generaliza mejor a agosto–septiembre. Es el **equilibrio sesgo-varianza**: un poco más de sesgo a cambio de menos varianza.

Ojo

Dos trampas que **no** caeremos:

- **Escalar solo con el train.** La media y la desviación se calculan con el entrenamiento y se aplican al test. Si las calculas con todo, el test se filtra: **fuga de datos** (Apunte 05).
- **Elegir alpha mirando el test.** Aquí lo grafiqué contra el test *para enseñar*, pero en la práctica el punto dulce se elige con un conjunto de **validación** aparte —nunca tocando el test. Eso es justo el Módulo 4.

Pruébalo

Pregunta de entrevista: “¿Para qué sirve la regularización y qué diferencia hay entre Ridge y Lasso?”

Respuesta modelo: “Regularizar castiga los coeficientes grandes para reducir el sobreajuste: cambia un poco más de sesgo por menos varianza, y su fuerza (alpha) se elige por validación.

Ridge encoge todos los coeficientes hacia cero sin anularlos; **Lasso** puede ponerlos en cero exacto, así que además **selecciona variables**. Antes de regularizar hay que **escalar**, para que el castigo sea justo entre variables de distinta magnitud.”

La analogía metalúrgica: regularizar es como un **control de planta amortiguado**: un lazo que reacciona con mesura a cada fluctuación es más estable que uno que persigue cada pico de ruido. Y escalar es **normalizar los ensayos a unidades comparables** antes de combinarlos —no sumas gramos con porcentajes.

Módulo 3: cerrado. Ya sabes qué es un modelo lineal, **cómo** aprende (descenso de gradiente) y cómo cuidarlo (escalar + regularizar). Siguiendo: **Módulo 4** — preprocesamiento y validación bien hechos (pipelines, validación cruzada e hiperparámetros).

Módulo 4 — Preprocesamiento y validación

Apunte 16 — Pipelines: preprocesar sin fugas

Módulo 4 (Preprocesamiento y validación) · Machine Learning desde cero

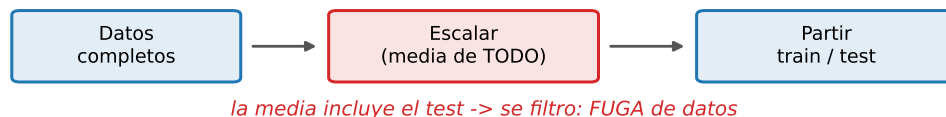
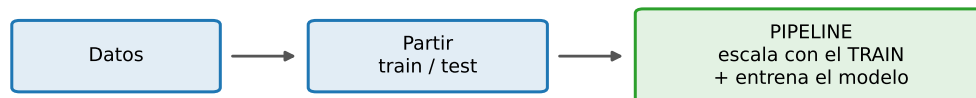
Ya veníamos usando un *pipeline* sin explicarlo (el `make_pipeline(StandardScaler, ...)` de los Apuntes 13–15). Toca entenderlo, porque es la herramienta que garantiza preprocesar **sin fuga de datos**.

Glosario — en palabras simples

- **Preprocesar:** preparar los datos antes del modelo (escalar, codificar categorías, rellenar huecos...).
- **Pipeline:** una “tubería” que encadena los pasos de preprocesamiento y el modelo en **un solo objeto**. Al entrenarlo, cada paso aprende *solo con el train*; al predecir, aplica lo aprendido.
- **Codificar (encoding):** convertir texto de categorías en números que el modelo entienda (ver sección 2).

1. El orden importa: escalar después de partir

Preprocesar sin fugas: el orden importa

MAL*la media incluye el test -> se filtro: FUGA de datos***BIEN***el pipeline aprende la media SOLO del train -> sin fuga*

Concepto

Escalar usa la **media** de las variables. Si la calculas con *todos* los datos y *después* partes en train/test, esa media ya vio el test: se **filtró** información del examen al estudio —**fuga de datos** (Apunte 05), en su forma más silenciosa. Lo correcto: partir **primero**, y que el escalado aprenda la media **solo del train**. Un **Pipeline** hace justo eso de forma automática: cuando le das `.fit(train)`, escala con el train; cuando le das datos nuevos, aplica esa misma regla sin volver a mirar.

2. Codificar categorías (cuando las haya)

Tu dataset es todo numérico, pero muchos no lo son. Si tuvieras una columna **Turno** con valores A/B/C, el modelo no entiende texto. Se convierte con **one-hot**: una columna indicadora por categoría.

Turno	→	es_A	es_B	es_C
A		1	0	0
B		0	1	0
C		0	0	1

Ese paso también va **dentro** del pipeline, para que aprenda las categorías del train y trate igual cualquier dato nuevo.

Ojo

La fuga por preprocesamiento es tan común porque el código “se ve bien”: escalas, luego partes, todo corre sin error. Por eso la regla de oro no es “¿corrió?” sino “¿este paso aprendió algo del test?”. El pipeline convierte esa disciplina en algo **automático** y, de paso, deja el código limpio y listo para la validación cruzada (Apunte 17).

Pruébalo

Pregunta de entrevista: “¿Por qué meter el escalado dentro de un Pipeline en vez de escalar el dataset a mano al principio?”

Respuesta modelo: “Para evitar fuga de datos: el escalador debe aprender la media y la desviación **solo del train** y aplicarlas al test; si escalo todo a mano antes de partir, el test se filtra y la métrica sale optimista. El Pipeline garantiza que cada paso se ajuste únicamente con los datos de entrenamiento en cada partición —esencial cuando uso validación cruzada, donde el train cambia en cada vuelta. Además encapsula preprocesamiento y modelo en un objeto reproducible.”

La analogía metalúrgica: un pipeline es tu **protocolo estándar de preparación y ensayo**: calibras el instrumento con **estándares del lote que estudias**, no con muestras que aún no llegan, y cada muestra pasa por los mismos pasos en el mismo orden. Reproducible y sin contaminación cruzada —lo mismo que aquí evitamos con el modelo.

Siguiente: Apunte 17 — en vez de un solo examen, muchos: la **validación cruzada**.

Apunte 17 — Validación cruzada: muchos exámenes en vez de uno

Módulo 4 (Preprocesamiento y validación) · Machine Learning desde cero

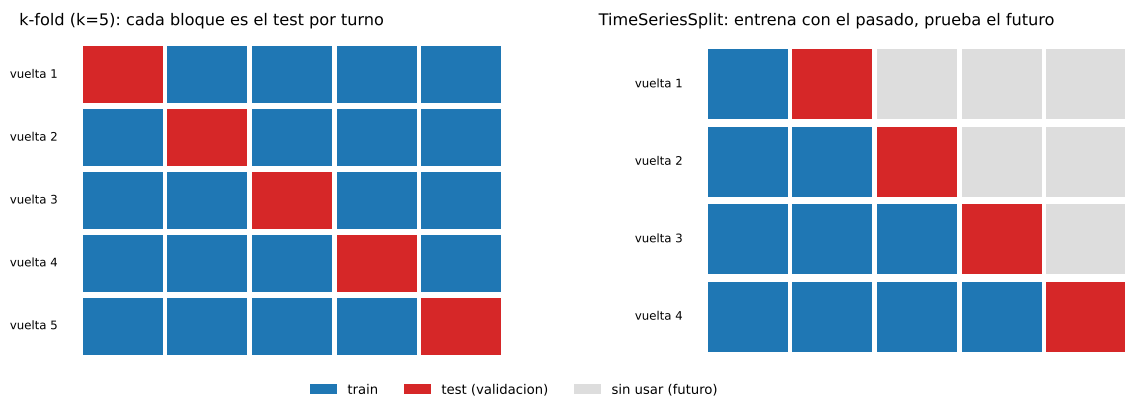
Hasta ahora calificamos con **un** solo examen (el test de agosto–septiembre). ¿Y si ese mes fue rarísimamente fácil o difícil? Un solo número puede engañarte. La **validación cruzada** toma varios exámenes y mira el promedio *y* la variación.

Glosario — en palabras simples

- **Fold (pliegue):** cada trozo en que partimos los datos.
- **Validación cruzada k-fold:** parte en k trozos; cada uno hace de test por turno mientras los otros entrenan. Da k notas; promedias.
- **TimeSeriesSplit:** la versión para series de tiempo —siempre entrena con el **pasado** y prueba con el trozo siguiente (nunca al revés).
- **Validación:** el trozo que se usa para *elegir* modelo/perillas; NO es el test final.

1. La idea: rotar el examen

Validación cruzada: muchos exámenes en vez de uno



Concepto

Izquierda (k-fold): cada bloque es el test una vez; entrenas 5 modelos y promedias sus 5 notas —así no dependes de qué trozo te tocó. **Derecha (TimeSeriesSplit):** en series de tiempo no puedes usar el futuro para entrenar, así que el train solo **crece hacia adelante** y el test es siempre el tramo que sigue. Nunca entrenas con datos posteriores al examen —eso sería fuga (Apunte 05).

2. El resultado honesto sobre tus datos

Corrimos TimeSeriesSplit (5 vueltas) sobre el modelo lineal regularizado. Las notas por vuelta:

Vuelta	1	2	3	4	5
R^2	-0,01	-0,21	-0,27	-0,02	-0,12

Promedio: $-0,13 \pm 0,10$. **Esto es una revelación:** nuestro test de agosto–septiembre ($R^2 \approx 0,05$) resultó ser de los tramos más **favorables**; en otros periodos el modelo predice *peor* que el promedio. La validación cruzada nos salvó de un exceso de optimismo —y confirma, con más fuerza, la conclusión del Apunte 12: este problema, medido con honestidad, tiene muy poca señal predecible a este horizonte.

Ojo

Jamás uses k-fold aleatorio en una serie de tiempo: mezclarías pasado y futuro y entrenarías con datos posteriores al examen (fuga). Usa **TimeSeriesSplit**. Y recuerda: la validación cruzada se hace sobre los datos de **entrenamiento**, para elegir modelos y perillas; el test final se toca **una sola vez**, al cerrar.

Pruébalo

Pregunta de entrevista: “¿Por qué usar validación cruzada si ya tienes un train/test split? ¿Y qué cambia en series de tiempo?”

Respuesta modelo: “Un solo split da una estimación que depende de qué trozo cayó en el test; puede salir optimista o pesimista por azar. La validación cruzada promedia varios exámenes y me da también su **variabilidad**, que es información crítica sobre la confianza. En series de tiempo no puedo barajar las filas —entrenar con el futuro es fuga—, así que uso TimeSeriesSplit: el entrenamiento crece hacia adelante y siempre pruebo con el tramo siguiente.”

La analogía metalúrgica: nunca juzgas una veta —ni un método— con **una sola muestra** o una sola campaña. Tomas varias, independientes, y miras el promedio *y* la dispersión: esa varianza te dice cuánto confiar. Estimar la ley de un depósito con un único sondeaje es apostar; la validación cruzada es tomar varias muestras del desempeño del modelo en vez de fiarte de una.

Siguiente: Apunte 18 — usar la validación cruzada para **elegir las perillas** (hiperparámetros) sin tocar el test.

Apunte 18 — Ajustar las perillas (hiperparámetros) sin trampa

Módulo 4 (Preprocesamiento y validación) · Machine Learning desde cero

Ya elegimos varias “perillas”: la profundidad del árbol (Apunte 09), el **alpha** de la regularización (Apunte 15). Lo hicimos medio a mano. Ahora, la forma **sistemática y honesta**: buscarlas con validación cruzada, sin mirar el test.

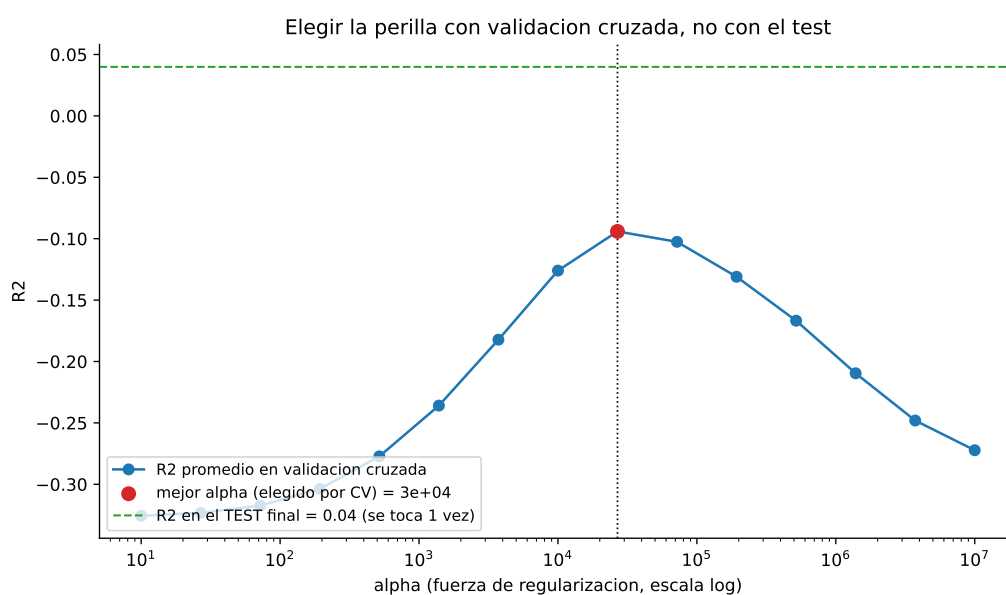
Glosario — en palabras simples

- **Parámetro**: lo que el modelo *aprende* solo (los coeficientes de la recta). No lo tocas tú.
- **Hiperparámetro**: la perilla que *tú* fijas antes de entrenar (profundidad, **alpha**, número de árboles).
- **Grid search**: probar todas las combinaciones de una **grilla** de valores candidatos.
- **Random search**: probar combinaciones al azar —más barato cuando hay muchas perillas.

1. La receta honesta

Concepto

Para cada valor candidato de la perilla: se mide su **nota de validación cruzada** (Apunte 17), *sin* tocar el test. Se elige el valor con mejor nota de CV. Y **solo al final**, con la perilla ya fijada, se evalúa **una única vez** el test, como control de realidad. GridSearchCV de scikit-learn hace todo esto por ti.



La curva azul es la nota de CV para cada **alpha**; el punto rojo, el mejor ($\alpha \approx 2,7 \times 10^4$, elegido **sin** ver el test). La línea verde es el test final ($R^2 = 0,04$), tocado una sola vez. Fíjate que

el test quedó *por encima* de la CV: coherente con el Apunte 17 —agosto–septiembre fue un tramo amable.

Ojo

Si eliges la perilla **mirando el test** (probando valores hasta que el test mejore), contaminaste tu única medida honesta: el test ya no estima el futuro, estima cuánto lo exprimiste. Es fuga de datos disfrazada. Por eso las perillas se eligen en **validación** y el test se reserva. (Con muchas perillas, hasta la CV se puede “exprimir”; para rigor extremo existe la *validación cruzada anidada*, pero eso es tema avanzado.)

Pruébalo

Pregunta de entrevista: “¿Cuál es la diferencia entre un parámetro y un hiperparámetro, y cómo eliges los hiperparámetros?”

Respuesta modelo: “Los parámetros los aprende el modelo al entrenar —los coeficientes—; los hiperparámetros los fijo yo antes —profundidad, alpha, número de árboles. Los elijo con búsqueda (grid o random search) evaluando cada candidato por validación cruzada sobre el train, y me quedo con el de mejor nota promedio. El test se toca una sola vez, al final: si lo uso para elegir, deja de ser una estimación honesta del desempeño futuro.”

La analogía metalúrgica: es **optimizar setpoints con pruebas de planta piloto**. Ensayas varias dosis de reactivo en corridas de validación, eliges la mejor, y **recién entonces** confirmas en una corrida nueva e independiente. Ajustar la dosis mirando la corrida de confirmación sería engañarte: dejaría de confirmar nada.

Módulo 4: cerrado. Ya sabes preprocesar sin fugas (pipelines), evaluar con honestidad (validación cruzada) y elegir perillas sin trampa. Siguiente: **Módulo 5 — ingeniería de variables** (para por fin intentar subir el techo) y **gradient boosting**.

Módulo 5 — Ingeniería de variables y boosting

Apunte 19 — Ingeniería de variables: el salto real

Módulo 5 (Variables y boosting) · Machine Learning desde cero

Durante todo el Módulo 2 y 3 probamos modelos cada vez más sofisticados y todos tocaban el mismo techo bajo. La pregunta que nadie se había hecho: ¿y si el problema no eran los **modelos**, sino las **variables**? Este apunte es el punto de inflexión del proyecto.

Glosario — en palabras simples

- **Ingeniería de variables:** crear columnas nuevas a partir de las que tienes, usando conocimiento del proceso, para darle mejores pistas al modelo.
- **Rezago (lag):** el valor de una variable *hace un rato* (p.ej. la sílice de hace 1 hora).
- **Promedio móvil (rolling):** el promedio de las últimas horas —suaviza el ruido y capta la tendencia.
- **Inercia:** que el proceso no salta al azar; su estado ahora se parece al de hace un rato (autocorrelación, Apunte 03).

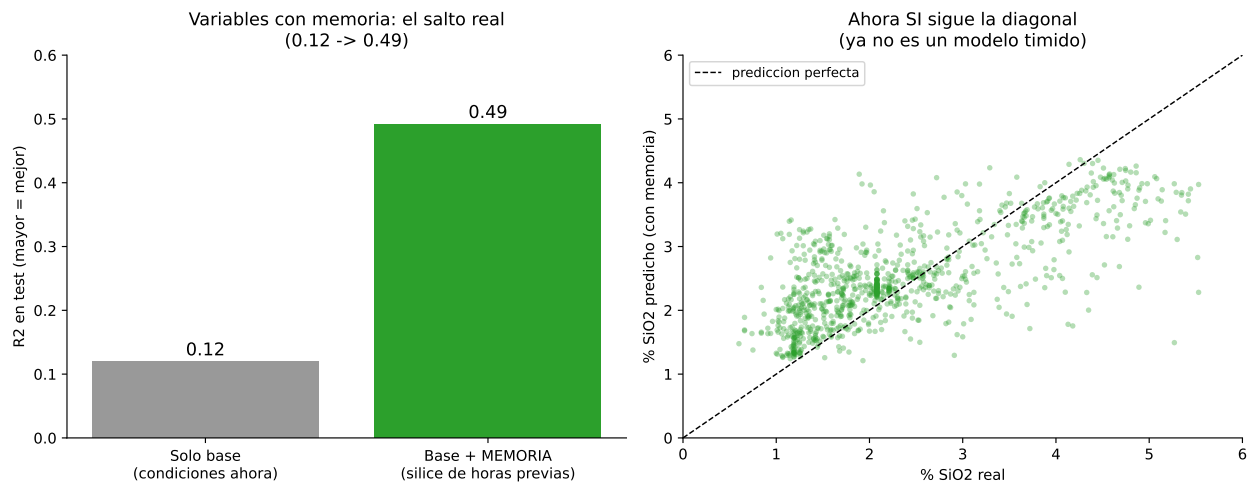
1. La idea: darle memoria al modelo

Hasta ahora el modelo predecía la sílice mirando *solo* las condiciones del instante. Pero la flotación tiene **inercia**: la sílice de ahora se parece mucho a la de hace una hora. Le agregamos esa **memoria** como variables nuevas: la sílice de hace 1, 2 y 3 horas, y el promedio de las últimas 3.

Concepto

Clave y honesta: estas variables **sí las tienes en tiempo real**. Recuerda que el laboratorio tarda 1 hora; entonces, cuando quieres estimar la sílice *ahora*, el resultado de lab de **hace una hora ya llegó**. Usar la sílice pasada no es fuga: es información genuinamente disponible. (Lo que NO puedes usar es la sílice de *esta* hora —esa es la respuesta que aún no existe.)

2. El resultado: por fin subimos el techo



Concepto

El R^2 en test saltó de **0,12** (solo condiciones del instante) a **0,49** (con memoria) —y el RMSE bajó de 1,07 a 0,81. A la derecha, la prueba visual: por primera vez en todo el proyecto la nube **sigue la diagonal**. Ya no es un modelo tímido: se atreve a los valores altos y bajos, porque ahora *sabe de dónde viene* la sílice.

Ojo

La lección más importante del proyecto: unas pocas variables bien pensadas le ganaron a todos los algoritmos sofisticados que probamos. En ML, casi siempre **mejores variables > modelos más complejos**. El salto no vino de un modelo más grande, sino de tu **conocimiento del proceso** (la inercia) convertido en columnas. Eso es lo que aporta un ingeniero de dominio —y no lo reemplaza ningún AutoML.

Pruébalo

Pregunta de entrevista: “*Tu modelo está estancado. ¿Buscarías un algoritmo más potente o harías otra cosa?*”

Respuesta modelo: “Antes de cambiar de algoritmo, revisaría las **variables**: casi siempre el mayor salto viene de ingeniería de variables informada por el dominio, no de un modelo más complejo. En una serie de proceso con inercia, agregaría **rezagos y promedios móviles** del objetivo y de las variables clave —cuidándome de usar solo información disponible en el momento de predecir, para no meter fuga. En este caso, eso subió el R^2 de 0.12 a 0.49, más que cualquier cambio de modelo.”

La analogía metalúrgica: es cómo piensa un **operador experto**: no estima la ley del próximo turno de la nada, la ancla en la **tendencia reciente** —“venía en 2.3 y estable, seguirá cerca”. La mayor mejora en una planta no suele venir de un controlador más sofisticado, sino de **medir y usar la variable correcta**. Aquí, esa variable era la propia historia de la sílice.

Siguiente: Apunte 20 — **gradient boosting**, el modelo que suele ganar en datos de tabla, para exprimir aún más estas nuevas variables.

Apunte 20 — Gradient boosting: el circuito que limpia lo que falta

Módulo 5 (Variables y boosting) · Machine Learning desde cero

El random forest arma cientos de árboles **en paralelo** y los promedia. El **gradient boosting** hace lo contrario: arma los árboles **en fila**, y cada uno se dedica a **corregir los errores** del anterior. Es el modelo que suele ganar en datos de tabla —y aquí dará el mejor resultado del proyecto.

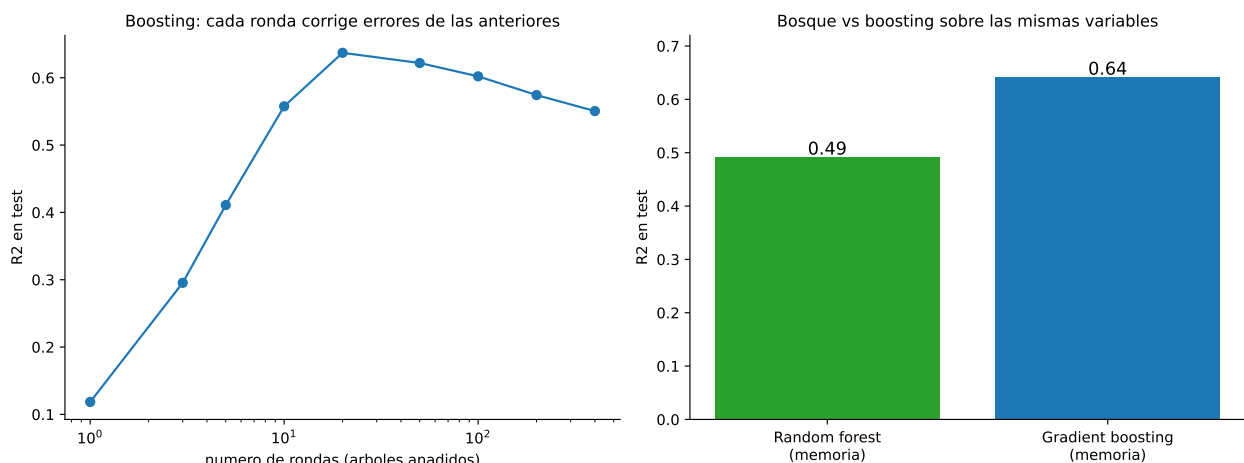
Glosario — en palabras simples

- **Gradient boosting:** muchos árboles pequeños **en secuencia**, cada uno entrenado para corregir lo que el conjunto anterior erró.
- **Ronda:** cada árbol nuevo que se añade.
- **Residuo:** el error que queda por corregir tras las rondas previas.
- **Early stopping (parada temprana):** dejar de añadir rondas cuando ya no mejora —evita sobreajustar.
- **XGBoost / LightGBM:** las librerías famosas que implementan esta idea. Aquí usamos la versión integrada de scikit-learn (misma idea, sin instalar nada).

1. La idea: corregir el error, ronda a ronda

Concepto

El primer árbol hace una predicción tosca. El segundo no vuelve a predecir la sílice: predice el **error** del primero, y se suma para corregirlo. El tercero corrige lo que aún queda mal, y así. Cada ronda empuja la predicción un pasito hacia la verdad —usando, por dentro, el mismo **descenso de gradiente** del Apunte 14 (de ahí el nombre). Al final, la suma de muchos correctores pequeños es muy precisa.



A la izquierda: el R^2 sube rápido con las primeras rondas y luego, con **demasiadas**, *baja* —el boosting empieza a memorizar (sobreajuste, Apunte 09). Por eso se usa **early stopping**. A la

derecha, el resultado honesto: sobre las mismas variables con memoria, el boosting ($R^2 = 0,64$) le gana al bosque (0,49). **Es el mejor modelo de todo el proyecto.**

Ojo

El boosting es potente pero más **delicado** que el bosque: si le pones demasiadas rondas, sobreajusta (la curva de la izquierda baja); y es más sensible a sus perillas (rondas, tasa de aprendizaje, profundidad). Por eso casi siempre se entrena con **early stopping** —que usa una validación interna para parar a tiempo, sin tocar el test. El bosque, en cambio, casi no sobreajusta al agregar árboles: más robusto, algo menos preciso.

Pruébalo

Pregunta de entrevista: “¿Diferencia entre random forest y gradient boosting? ¿Cuándo cada uno?”

Respuesta modelo: “Ambos son conjuntos de árboles, pero el forest los entrena **en paralelo** sobre muestras distintas y los promedia (reduce varianza, muy robusto); el boosting los entrena **en secuencia**, cada uno corrigiendo el error del anterior mediante descenso de gradiente (reduce sesgo, suele ser más preciso en datos de tabla, pero más propenso a sobreajustar y más sensible a hiperparámetros). Uso forest como base robusta y rápida; boosting con early stopping cuando quiero exprimir el máximo desempeño.”

La analogía metalúrgica: el boosting es un **circuito rougher–scavenger– cleaner**. La primera etapa (rougher) hace el grueso del trabajo; la siguiente recupera lo que la anterior **dejó escapar**; la siguiente limpia el residuo restante. Cada etapa ataca *lo que faltó*. El random forest, en cambio, sería como 300 celdas independientes votando en paralelo. Dos filosofías de circuito —y aquí, el de etapas en serie recuperó más.

Módulo 5: cerrado. Con mejores variables (memoria) y un mejor modelo (boosting), pasaste de $R^2 \approx 0,04$ a **0.64**. Siguiendo: **Módulo 6 — clasificación** (predecir “fuera de especificación: sí/no”).

Módulo 6 — Clasificación

Apunte 21 — De regresión a clasificación

Módulo 6 (Clasificación) · Machine Learning desde cero

Hasta ahora pedíamos un **número**: “¿cuánto SiO₂ tendrá el concentrado?” (regresión). Muchas veces a la planta le sirve más una **alarma**: “¿esta hora va a salir fuera de especificación, sí o no?”. Eso es **clasificación**: la otra mitad del aprendizaje supervisado.

Glosario — en palabras simples

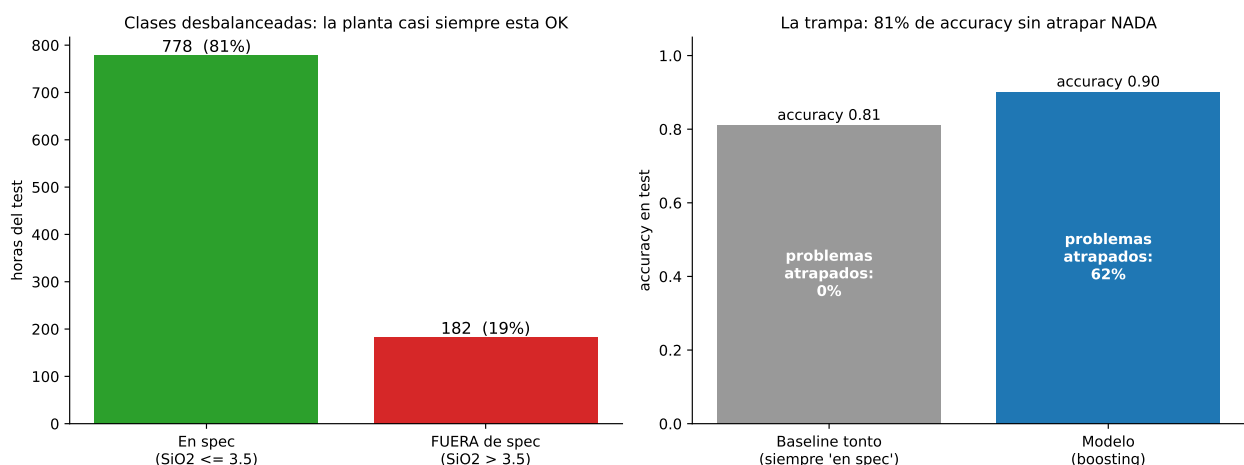
- **Clasificación**: predecir una **categoría** (sí/no, A/B/C) en vez de un número.
- **Clase**: cada categoría posible. Aquí dos: *en spec* y *fuera de spec*.
- **Probabilidad**: el clasificador no dice “sí” de golpe; entrega una **probabilidad** (p.ej. 0.73 de estar fuera de spec).
- **Umbral de decisión**: el corte que convierte esa probabilidad en sí/no (por defecto 0.5).
- **Accuracy**: el % total de aciertos.
- **Desbalance**: que una clase sea mucho más frecuente que la otra.

1. Convertir el objetivo en categoría

Definimos `fuera_de_spec = (% SiO2 > 3.5)`. ¿Por qué 3.5? **Elegido con datos**, no al azar: está cerca del percentil 82 de tus horas y deja ~18% de las horas “fuera de spec” —un desbalance realista (la planta suele andar bien y a veces no). **Este umbral es tuyo**: si el contrato con el cliente fija otro límite, se cambia y todo lo demás sigue igual.

Usamos las *mismas* variables con memoria del Apunte 19 y el mismo modelo ganador del Apunte 20 —pero en su versión **clasificadora**.

2. La trampa de la accuracy



Concepto

A la izquierda, el desbalance: de las 960 horas del test, **778 están en spec (81 %)** y solo 182 fuera (19 %). A la derecha, la trampa: un modelo tramposo que **siempre diga “en spec”** acierta el **81 %** de las veces...y atrapa **0 %** de los problemas. ¡Un 81 % que no sirve para nada! Nuestro modelo saca 90 % de accuracy y atrapa el 62 % de las horas malas —pero ese 90 % por sí solo tampoco cuenta la historia.

Ojo

Con clases desbalanceadas, la accuracy miente. Es la métrica que todo el mundo reporta y la que menos dice cuando lo raro es lo importante. Necesitamos métricas que miren *específicamente* los problemas: cuántos atrapamos y cuántas alarmas fueron falsas. Eso es exactamente el Apunte 22.

Pruébalo

Pregunta de entrevista: “*Tu clasificador tiene 95 % de accuracy. ¿Está bien?*”

Respuesta modelo: “Depende del **balance de clases**. Si el evento que me importa ocurre el 5 % de las veces, un modelo que siempre diga ‘no pasa nada’ ya tiene 95 % de accuracy y es inútil. Antes de celebrar pregunto: ¿qué proporción tiene cada clase?, y miro **matriz de confusión, precisión y recall**, que sí muestran cuántos casos positivos detecté y a qué costo de falsas alarmas.”

La analogía metalúrgica: es un **detector de alarma que nunca suena**. Como la planta casi siempre opera bien, “no hay problema” acierta casi todos los turnos —y por eso mismo no sirve como alarma. Un instrumento se juzga por cómo se comporta cuando *sí* hay un evento, no por cuántos días tranquilos acertó.

Siguiente: Apunte 22 — la **matriz de confusión**, la **precisión** y el **recall**: las métricas que sí cuentan la verdad.

Apunte 22 — Matriz de confusión, precisión y recall

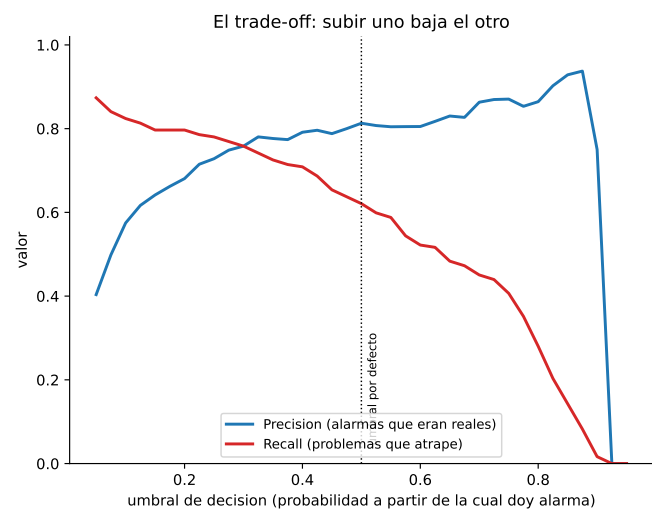
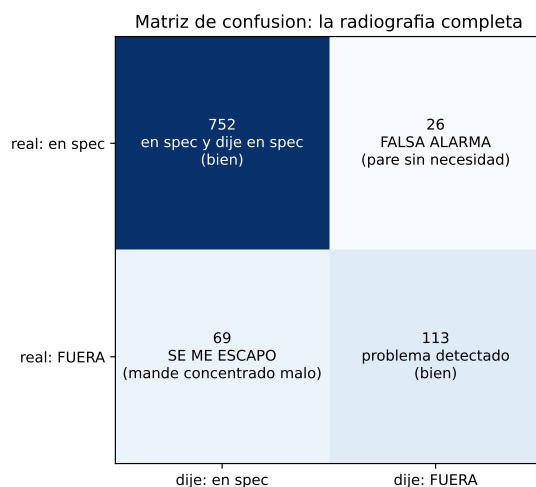
Módulo 6 (Clasificación) · Machine Learning desde cero

La accuracy nos mintió (Apunte 21). Estas son las métricas que sí cuentan la verdad cuando lo que importa es **detectar el problema**.

Glosario — en palabras simples

- **Matriz de confusión:** tabla 2×2 con los aciertos en la diagonal y los **dos tipos de error** en las esquinas.
- **Falso positivo (falsa alarma):** dije “fuera de spec” y estaba bien.
- **Falso negativo (se me escapó):** dije “en spec” y estaba *mala*.
- **Precisión:** de las alarmas que di, ¿cuántas eran reales?
- **Recall:** de los problemas reales, ¿cuántos atrapé?
- **F1:** un promedio equilibrado de precisión y recall (un solo número).

1. La radiografía completa



Concepto

Con nuestras 960 horas de test:

- **752** veces dije “en spec” y estaba en spec. ✓
- **113** veces di la alarma y *sí* había problema. ✓
- **26 falsas alarmas:** paré (o alerté) sin necesidad.
- **69 se me escaparon:** dije “todo bien” y el concentrado salió malo.

De ahí salen las dos métricas clave:

$$\text{Precisión} = \frac{113}{113 + 26} = 0,81, \quad \text{Recall} = \frac{113}{113 + 69} = 0,62.$$

Se leen así: “de cada 10 alarmas que doy, **8 son reales**” y “de cada 10 horas malas, **atrapo 6**”. (El F1, que las promedia, da 0,70.)

2. El umbral: una decisión de ingeniería, no de estadística

El modelo entrega una *probabilidad*; el **umbral** decide a partir de cuándo suena la alarma. La gráfica de la derecha muestra el **trade-off**: bajar el umbral hace sonar más alarmas → atrapas más problemas (**sube el recall**) pero te equivocas más seguido (**baja la precisión**). Subirlo hace lo contrario.

Concepto

¿Dónde poner el umbral? Depende de **qué error cuesta más en tu planta**:

- Si mandar concentrado fuera de ley trae **penalización del cliente** → prioriza **recall** (baja el umbral: mejor alarmar de más).
- Si cada alarma implica **parar o alterar el circuito** → prioriza **precisión** (sube el umbral: solo alarma cuando estés seguro).

Esa elección la hace el **ingeniero de proceso**, no el algoritmo.

Ojo

No existe un solo número que lo diga todo. El F1 equilibra, pero **esconde** el trade-off: dos modelos con el mismo F1 pueden comportarse muy distinto (uno alarmista, otro conservador). Reporta siempre **la matriz completa** y explica qué error priorizaste y **por qué**.

Pruébalo

Pregunta de entrevista: “¿Precisión o recall? ¿Cuál priorizas?”

Respuesta modelo: “Depende del costo de cada error, no del modelo. Precisión mide qué fracción de mis alertas son reales —importa cuando una falsa alarma es cara, como detener un circuito. Recall mide qué fracción de los eventos reales detecto —importa cuando dejar pasar uno es caro, como despachar concentrado fuera de especificación. Ajusto el **umbral de decisión** para moverme por esa curva según el costo del negocio, y reporto la matriz de confusión completa para que la decisión sea explícita.”

La analogía metalúrgica: es calibrar la **sensibilidad de una alarma de proceso**. Si la pones muy sensible, el turno vive parando la planta por nada (mucha falsa alarma, poca precisión). Si la pones muy dura, duermes tranquilo...hasta que despachas un lote fuera de ley (poco recall). El punto correcto no lo decide el instrumento: lo decide cuánto cuesta cada uno de los dos errores.

Módulo 6: cerrado. Ya dominas las dos mitades del aprendizaje supervisado: regresión (predecir números) y clasificación (predecir categorías), con sus métricas propias. Siguiendo: **Módulo 7 — interpretabilidad:** abrir la caja negra y explicar *cada* predicción (SHAP).

Módulo 7 — Interpretabilidad

Apunte 23 — Importancia honesta...y el baseline correcto

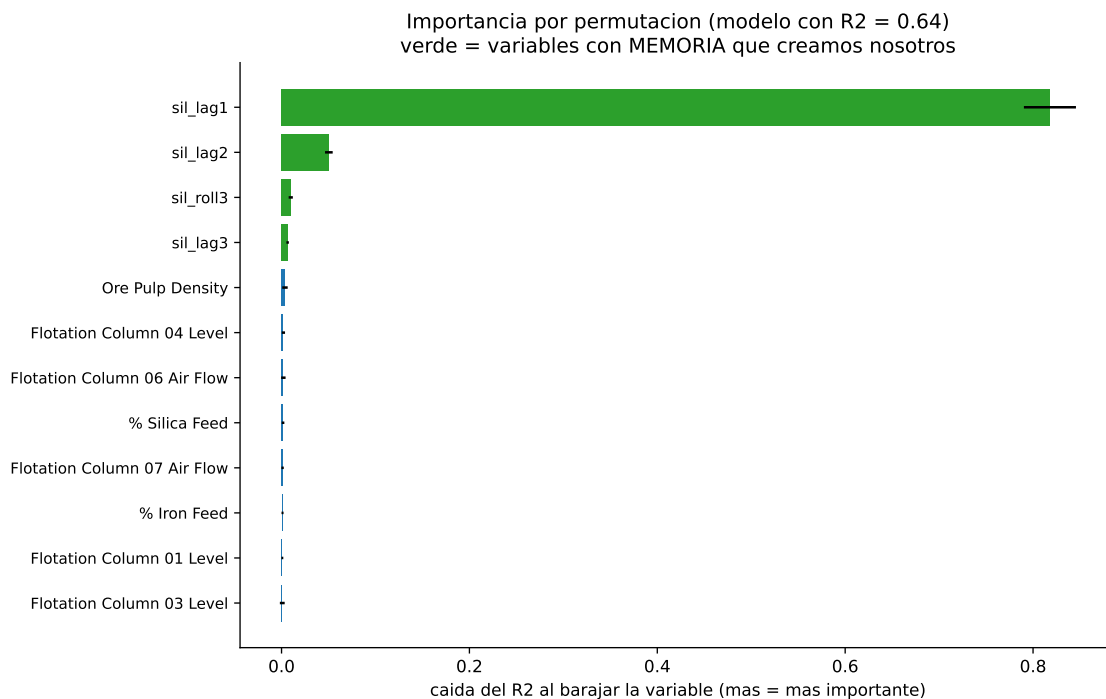
Módulo 7 (Interpretabilidad) · Machine Learning desde cero

En el Apunte 11 medimos qué variables usaba el bosque...y avisamos que esa medida dependía del modelo. Aquí vemos la versión **honest** —y nos lleva a un descubrimiento incómodo que **corrige** lo que celebramos en el Módulo 5.

Glosario — en palabras simples

- **Importancia por permutación:** baraja los valores de *una* variable en el **test** y mide cuánto empeora el modelo. Si empeora mucho, esa variable importaba de verdad.
- **Baseline de persistencia:** en una serie de tiempo, la predicción más tonta posible: “lo de ahora será igual a lo de hace un rato”. Sin modelo, sin nada.
- **Aporte incremental:** cuánto mejora tu modelo *por encima* del baseline adecuado. Es el número que de verdad justifica el proyecto.

1. La importancia bien medida



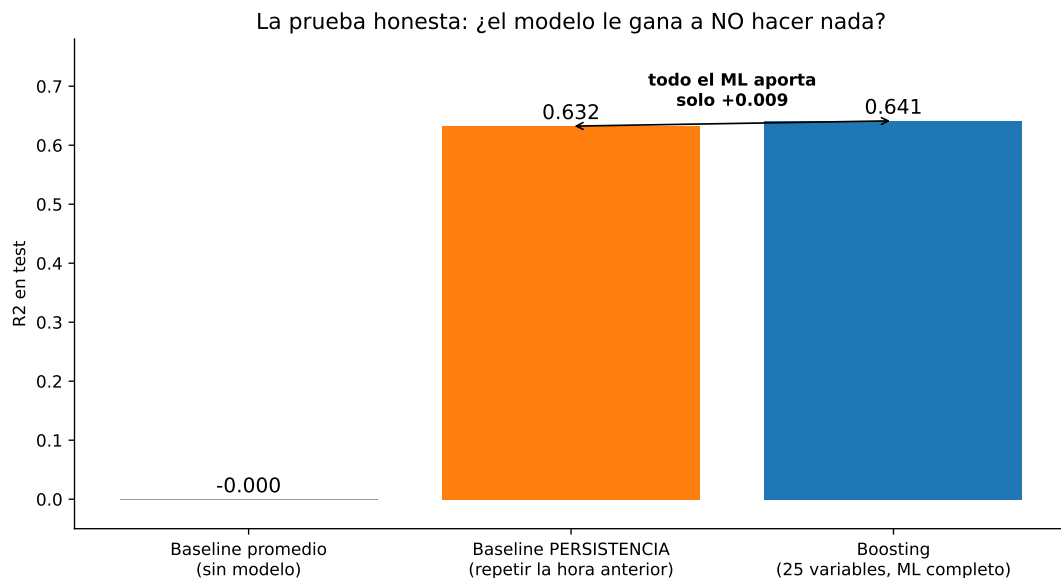
Concepto

A diferencia de la importancia interna del Apunte 11 (que dependía de cómo habíamos configurado el bosque), la **permutación** se mide sobre datos que el modelo nunca vio y sirve para *cualquier* modelo. El veredicto es brutal: **sil_lag1** —la sílice de hace una hora— se

lleva casi toda la importancia (0.82). Todo lo demás —incluidas TODAS tus variables de proceso— aporta prácticamente cero.

2. La pregunta incómoda que eso obliga a hacer

Si el modelo se apoya casi solo en “la sílice de hace una hora”...¿de verdad hace falta el modelo? Comparemos contra el baseline de **persistencia**: predecir, sin ningún algoritmo, que la sílice de ahora es *igual* a la de hace una hora.



Ojo

Corrección honesta del Módulo 5. Ahí celebramos haber subido el R^2 de 0,04 a 0,64 “gracias a las variables con memoria y al boosting”. La verdad completa: el baseline de persistencia —**no hacer nada**— ya logra **0,632**. Nuestro modelo con 25 variables y gradient boosting alcanza 0,641: un aporte real de **+0.009**. Casi todo el “gran salto” no vino del machine learning, sino de la **inercia del proceso**, que es gratis. Comparamos contra el baseline equivocado (el promedio) y por eso el logro pareció mayor de lo que es.

Concepto

La lección: cada problema tiene su baseline *correcto*. Para datos normales, predecir el promedio. Para una **serie de tiempo**, el baseline correcto es la **persistencia**. Un R^2 alto puede venir “regalado” por la autocorrelación sin que tu modelo aporte nada. La pregunta que justifica un proyecto de ML nunca es “¿qué R^2 tengo?”, sino “¿cuánto le gano a la alternativa tonta?”.

3. ¿Y ahora qué? (la salida constructiva)

Esto no invalida el trabajo: lo **sitúa**. A una hora de horizonte, la inercia manda y el ML aporta poco. Donde el ML *sí* podría ganarse su lugar es donde la persistencia falla: horizontes más largos (3, 6 horas), o predecir los **cambios bruscos** —los episodios que a la planta le duelen— en vez del nivel. Ese es el experimento honesto que se seguiría.

Pruébalo

Pregunta de entrevista: “*Tu modelo de serie de tiempo tiene $R^2 = 0.64$. ¿Impresionante?*”

Respuesta modelo: “No lo sé hasta compararlo con el **baseline de persistencia**. En series con inercia, repetir el último valor ya da R^2 altos “gratis”. En mi caso, la persistencia daba 0.632 y el modelo completo 0.641: el aporte real del ML era +0.009, así que reportarlo como un gran logro habría sido engañoso. Lo que justifica un proyecto es el **aporte incremental sobre la alternativa trivial**, no el R^2 absoluto.”

La analogía metalúrgica: es como presumir que “predices” la ley del turno siguiente...reportando la ley del turno anterior. Aciertas seguido, porque el yacimiento y la planta no cambian de golpe —pero no predijiste nada, solo aprovechaste la continuidad. Si tu modelo no le gana a eso, todavía no tienes un modelo: tienes un termómetro con retardo.

Siguiente: Apunte 24 — **SHAP:** abrir la caja negra y explicar *cada* predicción individual.

Apunte 24 — SHAP: el porqué de *cada* predicción

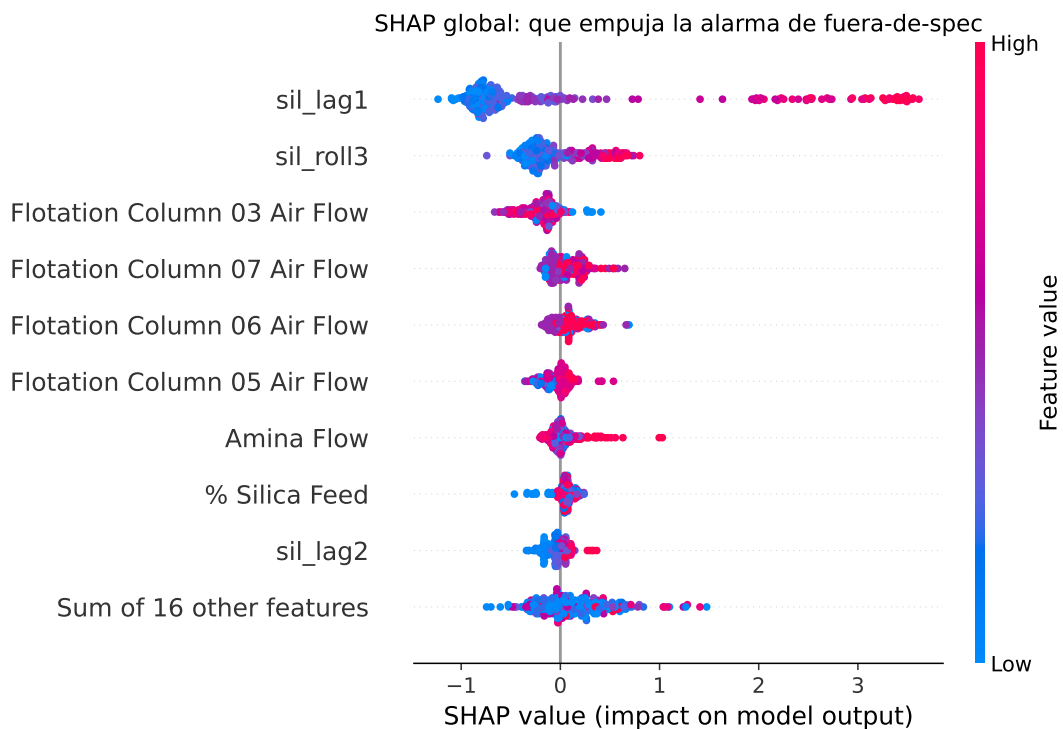
Módulo 7 (Interpretabilidad) · Machine Learning desde cero

La importancia de variables (Apuntes 11 y 23) dice qué pesa **en promedio**. Pero cuando suena una alarma a las 3 de la mañana, el operador no pregunta el promedio: pregunta “¿por qué *esta* alarma?”. Eso lo responde **SHAP**.

Glosario — en palabras simples

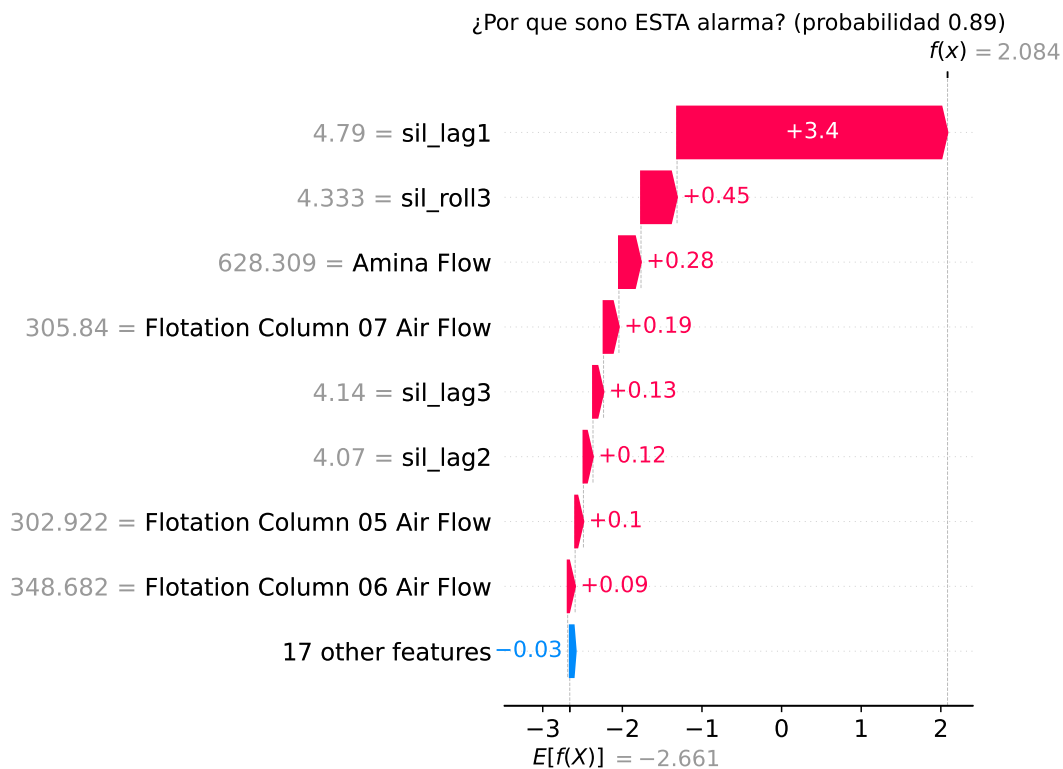
- **SHAP:** método que **reparte** una predicción concreta entre las variables, diciendo cuánto empujó cada una (hacia arriba o hacia abajo).
- **Valor base:** la predicción “de partida”, antes de mirar esta fila (el promedio del modelo).
- **Valor SHAP:** el empujón de una variable en *esta* predicción. Positivo empuja hacia la alarma; negativo, en contra.
- **Waterfall:** el gráfico que muestra, paso a paso, cómo se pasa del valor base a la predicción final.

1. La vista global



Cada punto es una hora; su posición horizontal es cuánto empujó esa variable, y el color es si la variable estaba **alta (rojo)** o **baja (azul)**. Se lee clarísimo: cuando **sil_lag1** viene **alta** (roja), empuja fuertemente **hacia la alarma**. Y aquí sí asoman tus variables de proceso —aire de columnas, amina— aportando empujones menores pero visibles.

2. La joya: explicar UNA alarma



Concepto

Esta es una hora concreta que el modelo marcó con **89 % de probabilidad** de salir fuera de spec. El gráfico cuenta la historia de abajo hacia arriba: se parte del valor base ($-2,66$, o sea “normalmente no hay alarma”) y cada variable empuja:

- `sil_lag1` = 4,79 (¡la sílice ya venía muy alta!) → **+3.4**
- `sil_roll3` = 4,33 (y llevaba 3 horas alta) → **+0.45**
- Amina = 628 (dosificación alta, coherente con estar peleando la sílice) → **+0.28**
- Aire columnas 7, 5 y 6 elevados → **+0.19, +0.10, +0.09**

Total: $f(x) = 2,08$, que traducido a probabilidad es **0.89**. En lenguaje de planta: “*alarma porque la sílice venía alta y sostenida, con amina y aire ya empujados al límite*”. Eso es lo que puedes poner frente a un operador.

Ojo

Dos cautelas. (1) **SHAP explica al MODELO, no a la naturaleza**: dice en qué se apoyó el algoritmo, no qué *causa* la sílice. Con lazos de control de por medio (Apunte 04), confundir eso lleva a decisiones malas. (2) Confirma lo del Apunte 23: el peso enorme de `sil_lag1` es, otra vez, la inercia —SHAP lo hace visible en cada caso.

Pruébalo

Pregunta de entrevista: “¿Qué aporta SHAP que no te dé la importancia de variables?”

Respuesta modelo: “La importancia es **global**: qué variables pesan en promedio. SHAP es **local**: descompone *una* predicción concreta en la contribución de cada variable, con signo

y magnitud, partiendo de un valor base. Eso permite justificar una alerta individual ante el usuario —clave para que un operador confíe y actúe— y detectar casos donde el modelo se apoya en algo absurdo. Eso sí: SHAP explica el comportamiento del modelo, no establece causalidad.”

La analogía metalúrgica: SHAP es el **balance metalúrgico de una predicción**. Así como un balance reparte la pérdida de recuperación entre las etapas del circuito —tanto se fue en el rougher, tanto en el cleaner—, SHAP reparte la predicción entre las variables: tanto puso la sílice previa, tanto la amina, tanto el aire. Un balance no te dice *por qué* el mineral es difícil; te dice *dónde* se fue. Igual SHAP.

Módulo 7: cerrado. Ya sabes medir importancia con honestidad y explicar predicciones individuales. Siguiendo: **Módulo 8**, el experimento que responde la pregunta que nos dejó el Apunte 23: **¿dónde SÍ aporta el ML?**

Módulo 8 — ¿Dónde SÍ aporta el machine learning?

Apunte 25 — ¿Dónde SÍ aporta el machine learning?

Módulo 8 · Machine Learning desde cero

El Apunte 23 nos dejó una pregunta incómoda: si a una hora el ML solo aporta +0,009 sobre repetir el último valor, ¿sirve de algo? Este apunte la responde con un experimento. Es, con diferencia, el trabajo más *científico* del proyecto.

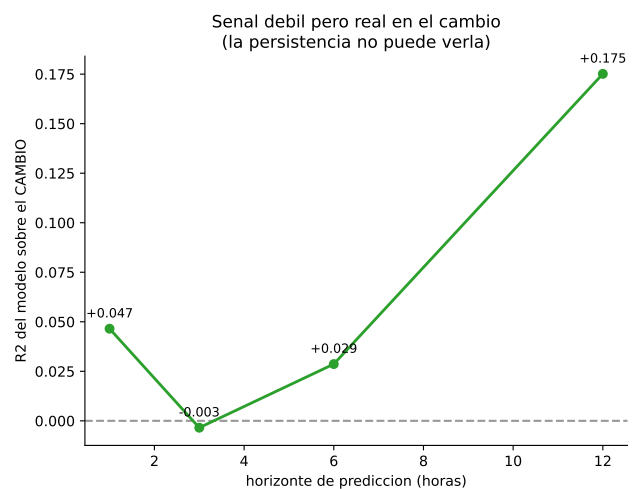
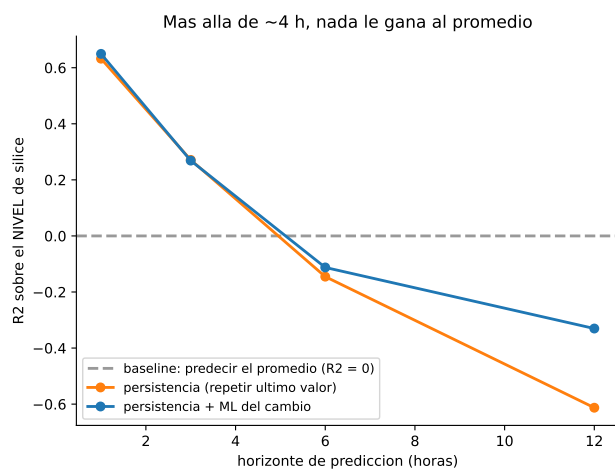
Glosario — en palabras simples

- **Horizonte:** cuántas horas hacia adelante queremos predecir. A mayor horizonte, menos ayuda la inercia.
- **Predecir el nivel:** “¿cuánta sílice habrá?”.
- **Predecir el cambio:** “¿cuánto va a *subir o bajar* respecto de lo último que sé?”. La persistencia, por definición, predice cambio cero: si el modelo acierta algo aquí, es señal genuinamente suya.

1. El experimento

En el instante t conocemos las variables de proceso hasta t y el último análisis de lab (hora $t - 1$). Predecimos la sílice a **1, 3, 6 y 12 horas** de ese último dato conocido, y comparamos tres estrategias: el promedio histórico, la persistencia y el modelo.

Horizonte	Promedio	Persistencia	Modelo (nivel)	Persist. + ML del cambio
1 h	0.000	+0,632	+0,641	+0,649
3 h	0.000	+0,271	+0,273	+0,269
6 h	0.000	-0,145	-0,147	-0,112
12 h	0.000	-0,613	-0,851	-0,330



2. Lo que dicen los datos

Concepto

(a) **La inercia se muere rápido.** La persistencia pasa de 0,632 (1 h) a 0,271 (3 h) y a partir de ~5 h es *peor* que decir simplemente “el promedio de siempre”. Tiene sentido físico: la planta cambia de régimen en pocas horas.

(b) **El ML no rescata el nivel.** En todos los horizontes su aporte sobre la persistencia es $\leq 0,02$...y a 12 h el modelo entrenado sobre el nivel es *peor* ($-0,851$), porque se aferra a un dato viejo que ya no sirve.

(c) **Pero hay señal en el CAMBIO.** Al reformular el objetivo —predecir cuánto se moverá la sílice en vez de dónde estará— el modelo logra $R^2 = +0,175$ a 12 h. La persistencia *no puede* capturar eso (siempre predice cambio cero). Es señal real, propia del modelo...pero **insuficiente**: al reconstruir el nivel sigue por debajo del simple promedio.

Ojo

El veredicto honesto del proyecto: con *estos* datos, el machine learning **no aporta valor operativo** para predecir la sílice, en ningún horizonte probado. A corto plazo gana la inercia (que es gratis); a largo plazo no gana nadie. Esto **no** es un fracaso del trabajo: es un **resultado bien medido**. La mayoría de proyectos de ML que fallan, fallan *en silencio* porque nadie corrió este experimento —despliegan un modelo que parecía bueno contra el baseline equivocado.

3. Y entonces, ¿qué haría falta?

El experimento también dice **dónde buscar**, que es lo valioso:

- **Reducir la latencia del laboratorio** (o poner un analizador en línea): si el dato llegara cada 10 minutos en vez de cada hora, la inercia —que ya demostramos que es fuerte a corto plazo— se volvería *útil* de verdad.
- **Medir otras variables:** las 21 actuales no contienen la información que mueve la sílice a horas vista (granulometría, mineralogía de alimentación, química del agua...).
- **Cambiar la pregunta:** el modelo del cambio a 12 h tiene señal; quizá la pregunta útil no sea “¿cuánta sílice?” sino “¿viene una tendencia al alza?”.

Pruébalo

Pregunta de entrevista: “¿Qué haces cuando tu modelo no le gana al baseline?”

Respuesta modelo: “Primero verifico que la comparación sea justa: baseline correcto para el problema (en series, la persistencia), evaluación temporal y sin fuga. Después exploro si el valor está en otra formulación —otro horizonte, predecir el cambio en vez del nivel, o detectar eventos. Si aun así no aporta, lo reporto: un resultado negativo bien medido evita desplegar un modelo inútil y **orienta la inversión** hacia lo que sí haría falta, normalmente más/mejores mediciones. Recomendar ‘no usar ML aquí’ también es un entregable válido.”

La analogía metalúrgica: es una **campana de pruebas que concluye que el reactivo nuevo no mejora la recuperación**. Nadie celebra...pero ese resultado, bien medido, le ahorra a la planta comprarlo por años, y además señala dónde sí está el cuello de botella. Un buen metalurgista sabe que una prueba negativa rigurosa vale más que una positiva mal hecha.

Módulo 8: cerrado. Siguiente: **Módulo 9** — aprendizaje **no supervisado**: descubrir los estados operativos de la planta sin ninguna respuesta que predecir.

Módulo 9 — Aprendizaje no supervisado

Apunte 26 — k-means: los estados operativos de tu planta

Módulo 9 (No supervisado) · Machine Learning desde cero

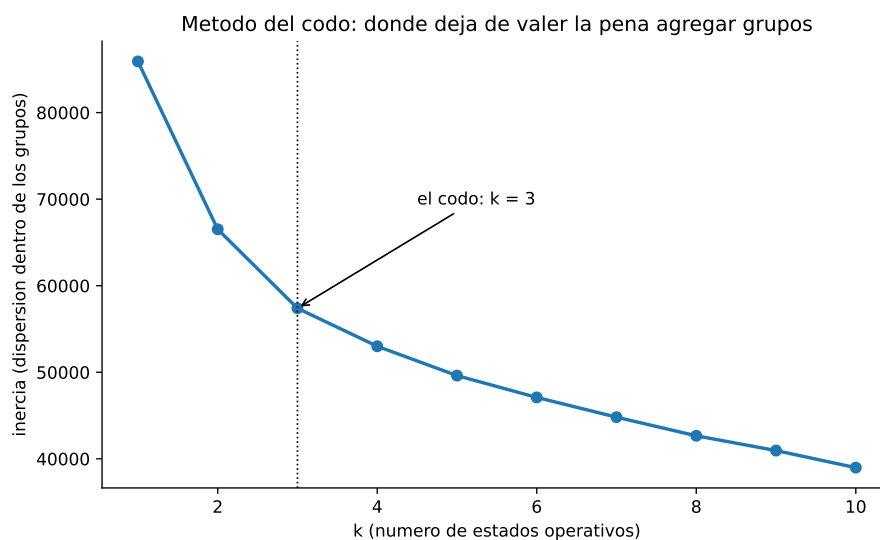
Todo lo anterior fue **supervisado**: teníamos una respuesta (la sílice) y queríamos predecirla. Ahora cambiamos de familia: **no supervisado**. No hay respuesta que adivinar —le pedimos al algoritmo que encuentre **estructura oculta** en tus datos. Y lo que encontró explica, de paso, el problema que nos persiguió todo el proyecto.

Glosario — en palabras simples

- **No supervisado**: aprender sin respuesta correcta; solo se buscan patrones.
- **Clustering (agrupamiento)**: juntar las horas que se *parecen* entre sí.
- **k-means**: pone k centros, asigna cada hora al centro más cercano, mueve los centros al promedio de sus horas, y repite hasta que nada se mueve.
- **Inercia**: qué tan dispersas quedan las horas dentro de su grupo. Baja siempre al subir k .
- **Método del codo**: elegir k donde agregar un grupo más ya casi no reduce la inercia.

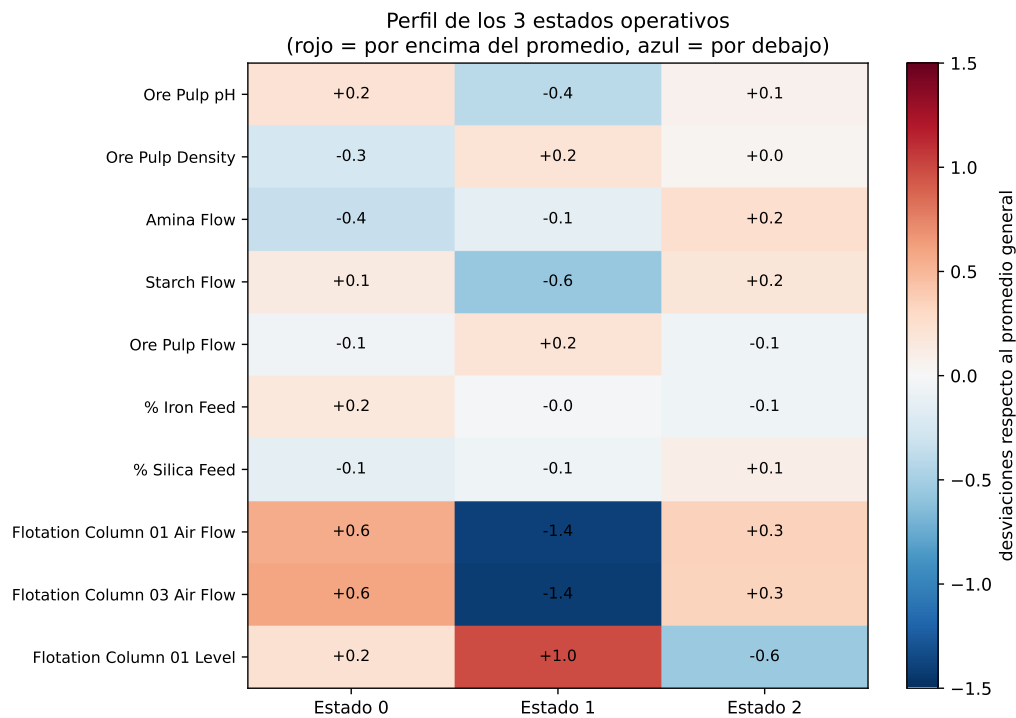
1. Elegir k con datos, no a dedo

Primero **estandarizamos** (Apunte 15): k-means mide *distancias*, y sin escalar el almidón (miles) aplastaría al pH (unidades). Luego probamos k de 1 a 10:



Las caídas grandes están en $k = 2$ ($-22,6\%$) y $k = 3$ ($-13,7\%$); de ahí en adelante cada grupo nuevo aporta apenas $\sim 5\%$. **El codo está en $k = 3$** : tres estados operativos.

2. Qué caracteriza a cada estado



Estado	Horas	Aire col. 1–3	Amina	% SiO ₂ alim.	% SiO ₂ conc.
0 “el bueno”	1083	296 (alto)	459 (baja)	13.7	1.85
1 “aire bajo”	935	239 (muy bajo)	476	14.2	2.60
2 “peleando”	2073	290	509 (máx)	15.4	2.46

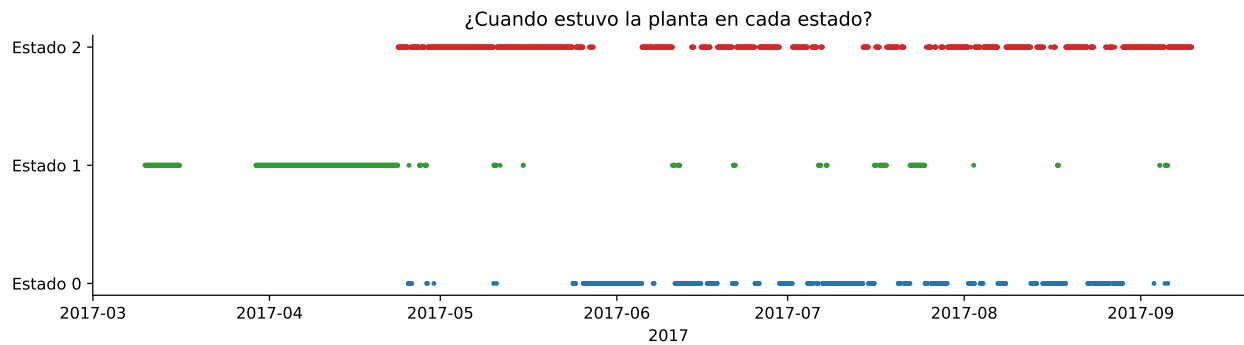
Concepto

Lee los tres regímenes con ojos de metalurgista:

- **Estado 0 — el bueno:** alimentación más limpia, aire alto, *menos* amina...y la mejor calidad (1.85 % de sílice). Menos reactivo, mejor resultado.
- **Estado 1 — aire bajo:** el aire de columnas se desploma (−1,4 desviaciones) y el nivel sube. Peor calidad de todas (2.60 %).
- **Estado 2 — peleando:** alimentación más sucia (15.4 %) y dosificación al máximo (amina y almidón). La planta empujando contra mineral difícil.

Y aquí lo notable: agrupamos usando *solo* variables de proceso —al algoritmo **nunca le dijimos** cuánta sílice salió. Aun así, los tres grupos tienen calidades claramente distintas. Eso confirma que los regímenes son **reales y relevantes**.

3. La revelación: cuándo ocurrió cada estado



Ojo

Esto explica el problema de todo el proyecto. El Estado 1 domina **marzo y abril**...y después prácticamente **desaparece**. Los Estados 0 y 2 aparecen *desde mayo*. Es decir: **la planta cambió de forma de operar a finales de abril**. Por eso los modelos entrenados con marzo–julio fallaban en agosto–septiembre (Módulo 2), por eso la validación cruzada daba resultados tan dispares según el tramo (Apunte 17) y por eso la persistencia se muere a pocas horas (Apunte 25): **no estamos ante un proceso estable, sino ante varias plantas distintas en el tiempo**. El aprendizaje no supervisado acaba de diagnosticar la enfermedad que el supervisado sufría.

Pruébalo

Pregunta de entrevista: “¿Para qué sirve el clustering si no predice nada?”

Respuesta modelo: “Para **entender** los datos, que muchas veces vale más que predecirlos. En mi proyecto agrupé las horas de planta solo con variables de proceso y aparecieron tres regímenes con calidades muy distintas —sin haberle dado nunca la variable de calidad. Además, al ver *cuándo* ocurría cada régimen descubrí un cambio de operación a mitad del histórico, que explicaba por qué mis modelos supervisados se degradaban con el tiempo. El clustering me dio el diagnóstico que la predicción no veía.”

La analogía metalúrgica: es **separar el histórico en campañas**. Un metalurgista revisando meses de datos distingue a ojo “aquí andábamos con mineral duro y dosificación alta”, “aquí el circuito estaba con aire restringido”. k-means hace eso mismo, pero con 21 variables a la vez y sin sesgo humano. Y te dice, con fecha, cuándo cambió la planta.

Siguiente: Apunte 27 — **PCA:** cómo ver en un plano de 2D algo que vive en 21 dimensiones, para *mirar* estos grupos.

Apunte 27 — PCA: ver en 2D lo que vive en 21 dimensiones

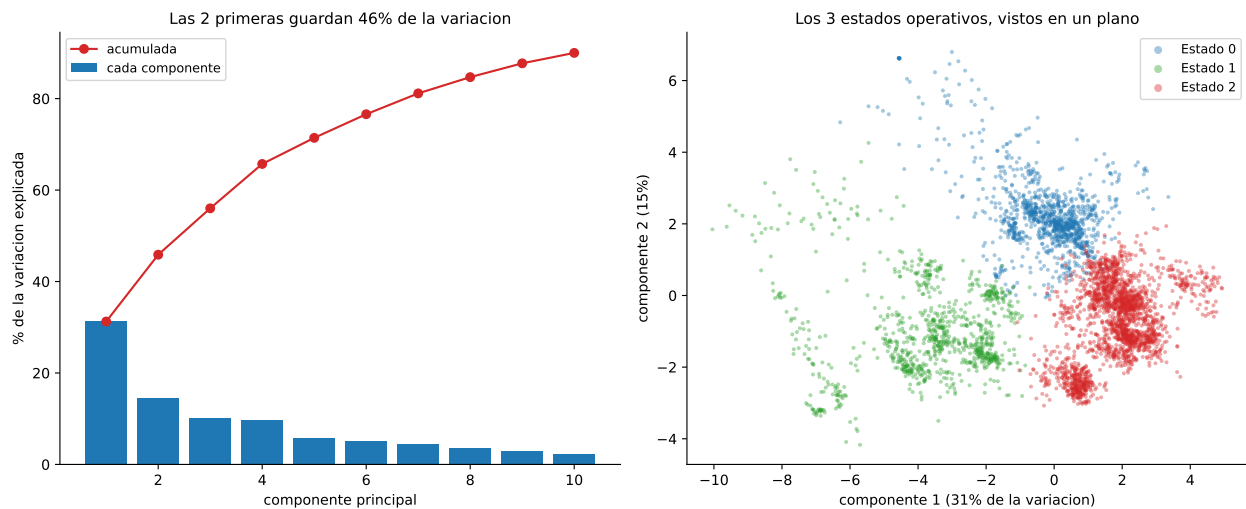
Módulo 9 (No supervisado) · Machine Learning desde cero

En el Apunte 26 encontramos tres estados operativos...pero no pudimos *verlos*: viven en un espacio de 21 variables y nuestros ojos manejan 2. El **PCA** resuelve eso.

Glosario — en palabras simples

- **PCA** (análisis de componentes principales): construye **ejes nuevos** que son combinaciones de tus variables, ordenados de mayor a menor *variación capturada*.
- **Componente principal**: cada uno de esos ejes nuevos. No es una variable física: es una mezcla.
- **Varianza explicada**: qué porcentaje de la variación total conserva cada componente. Es lo que mides al decidir con cuántos quedarte.
- **Carga (loading)**: cuánto pesa cada variable original dentro de un componente —sirve para *interpretar* el eje.

1. Comprimir sin perder (demasiado)



Concepto

Izquierda: el primer componente guarda el **31 %** de toda la variación de tus 21 variables, el segundo un **15 %**. Con solo esos dos ejes conservamos el **46 %** —suficiente para *mirar* los datos. (Con 10 componentes llegaríamos a ~90 %.)

Derecha: los tres estados del Apunte 26, dibujados en ese plano. Se ven **claramente separados**: el Estado 1 (verde) a la izquierda, el 2 (rojo) a la derecha, el 0 (azul) arriba. **Esto es una validación importante:** PCA no sabía nada de los grupos —los calculó solo con la variación— y aun así los grupos caen en zonas propias. Los regímenes son **estructura real** de tu planta, no un invento de k-means.

2. ¿Qué significan los ejes?

Mirando las cargas, los ejes tienen lectura metalúrgica directa:

- **Componente 1 (31%) = el eje de la aireación:** lo dominan los flujos de aire de las columnas 6, 7, 1 y 3. *La mayor fuente de variación de tu planta es cuánto aire está metiendo.*
- **Componente 2 (15%) = el eje de los niveles:** lo dominan los niveles de las columnas 5, 4, 7 y 6.

Con eso, el gráfico se lee solo: el Estado 1 vive en la zona de **poca aireación** (izquierda) —coherente con su aire bajísimo del Apunte 26— y el Estado 0 en la de niveles altos con buena aireación.

Ojo

Tres cautelas: (1) un componente **no es una variable física**, es una mezcla —se interpreta por sus cargas, con cuidado. (2) Quedarnos en 2D conserva el 46%: estamos **perdiendo la mitad** de la información, así que dos puntos cercanos en el dibujo no son necesariamente idénticos. (3) PCA busca **variación**, no lo que a ti te interesa: el eje que más varía no tiene por qué ser el que más afecta la calidad.

Pruébalo

Pregunta de entrevista: “¿Para qué usas PCA y qué cuidado tienes al interpretarlo?”

Respuesta modelo: “Para **visualizar** datos de muchas dimensiones y para reducir variables correlacionadas a unos pocos ejes independientes. Elijo cuántos componentes con la **varianza explicada**. El cuidado: los componentes son combinaciones lineales, no variables físicas, así que los interpreto por sus cargas; y PCA maximiza variación, no relevancia para mi objetivo —puede descartar una variable poco variable pero decisiva. Ah, y exige **estandarizar** antes, o domina la variable de números grandes.”

La analogía metalúrgica: es como resumir un análisis químico de 20 elementos en **dos índices compuestos** que capturan la mayor parte de la variabilidad del mineral. Pierdes detalle, pero por fin puedes **graficar** tus muestras y ver que se agrupan en familias. Igual que aquí: dos ejes —aireación y niveles— y toda la operación de seis meses cabe en una hoja.

Módulo 9: cerrado. Ya conoces las dos familias del ML: supervisado (M2–M8) y no supervisado (M9). Siguiente: **Módulo 10** — redes neuronales, el paso a producto y el cierre del proyecto.

Módulo 10 — Redes, producción y cierre

Apunte 28 — Redes neuronales (y por qué aquí pierden)

Módulo 10 (Cierre) · Machine Learning desde cero

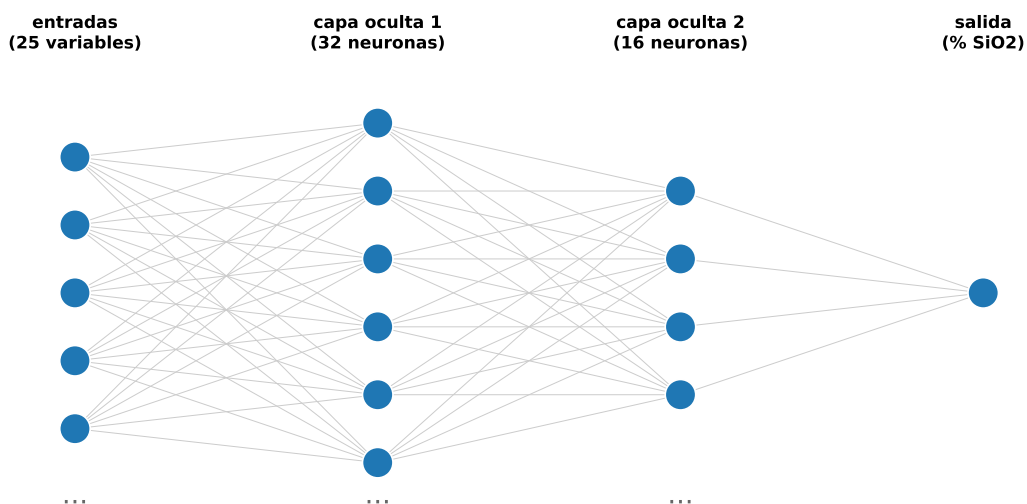
Es el modelo del que todo el mundo habla —“inteligencia artificial”, ChatGPT, etc. Vamos a ver qué es en realidad (más simple de lo que suena) y a probarlo en tus datos, con el resultado honesto por delante.

Glosario — en palabras simples

- **Neurona:** una operación minúscula: multiplica cada entrada por un **peso**, las suma, y pasa el resultado por una función que decide cuánto “se activa”.
- **Capa:** un grupo de neuronas en paralelo. Se apilan capas: la salida de una es la entrada de la siguiente.
- **Peso:** cada número que la red *aprende*. Nuestra red tiene miles.
- **Activación:** la función que dobla la señal (aquí ReLU); sin ella, apilar capas daría...una simple recta (Apunte 13).

1. Qué es, de verdad

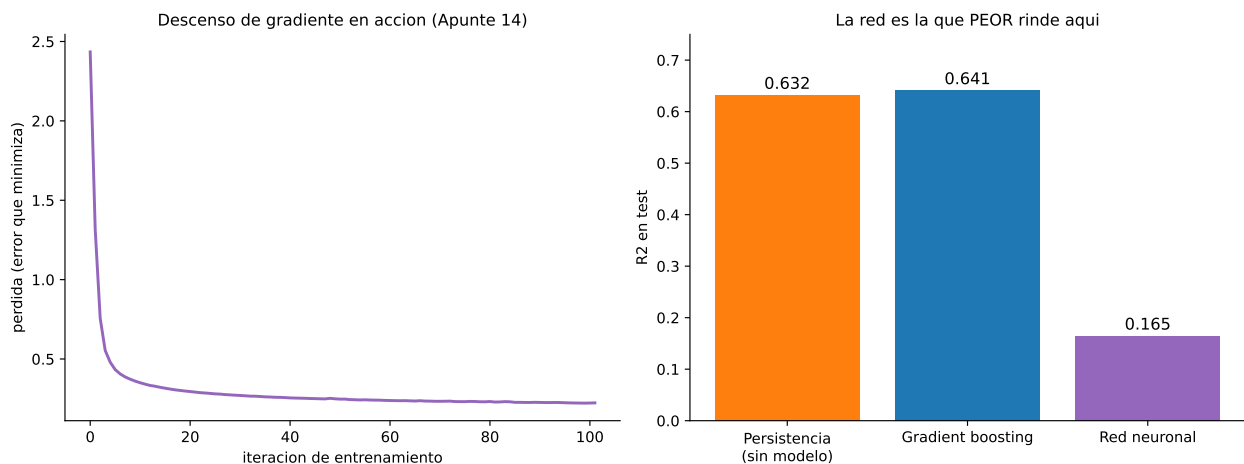
La red que entrenamos: cada línea es un peso que se aprende



Concepto

Cada línea del dibujo es un **peso** que la red debe aprender; cada círculo, una neurona. La información entra por la izquierda (tus 25 variables), atraviesa dos capas ocultas (32 y 16 neuronas) y sale un número: el % de SiO₂. ¿Y cómo aprende esos miles de pesos? Con el **descenso de gradiente del Apunte 14**, exactamente el mismo: calcula el error, mira la pendiente, y ajusta cada peso un pasito en contra. Repetido miles de veces. Eso es todo el “misterio” de una red neuronal.

2. Entrenamiento y resultado honesto



A la izquierda verás el descenso de gradiente *en vivo*: la pérdida cae rápido en las primeras iteraciones y luego se aplana —la red “rodando cuesta abajo” hasta el fondo del valle. Paró sola a las 102 iteraciones (parada temprana).

Ojo

Y el resultado: $R^2 = 0,165$. La red es la que peor rinde de todo el proyecto —peor que el boosting (0.641) y peor incluso que **no hacer nada** (persistencia, 0.632). No está mal configurada: es que **no es la herramienta para este trabajo**.

3. ¿Por qué pierden aquí?

- **Pocos datos:** 4.091 horas es *minúsculo* para una red. Las redes brillan con cientos de miles o millones de ejemplos.
- **Datos de tabla:** los árboles manejan de forma natural variables en escalas y naturalezas distintas, con cortes; la red tiene que *aprender* eso desde cero.
- **Donde sí ganan:** imágenes, texto, audio, señales largas —donde la estructura (píxeles vecinos, palabras en orden) es justo lo que las capas explotan.

Ojo

La lección que cierra el proyecto: más sofisticado **no** es mejor. En estos datos el orden fue: boosting > no hacer nada > bosque > red neuronal. Elegir la herramienta según el problema —y *medirlo*— es la habilidad; usar lo que está de moda, no.

Pruébalo

Pregunta de entrevista: “¿Cuándo usarías una red neuronal y cuándo no?”

Respuesta modelo: “Las uso cuando los datos son de alta dimensión y con estructura que la red pueda explotar —imágenes, texto, audio, señales— y cuando tengo volumen suficiente. Para datos **tabulares** de tamaño moderado, los ensambles de árboles (random forest, gradient boosting) suelen ganar con menos ajuste y más robustez: es un resultado bien conocido y lo comprobé en mi proyecto, donde la red quedó última, por debajo incluso del baseline trivial. La decisión la toma la medición, no la moda.”

La analogía metalúrgica: es traer una **celda de columna de última generación** para separar dos minerales gruesos y bien liberados que un **jig** resolvería mejor y más barato. El equipo sofisticado no es superior en abstracto: es superior *para la faena que fue diseñado*. Con mineral fino y complejo, la columna brilla; aquí, sobra.

Nota: usamos la red de scikit-learn (CPU) porque para este tamaño la GPU no aporta nada. Queda anotado un apunte extra opcional con **PyTorch sobre tu RTX 5060** para cuando entres a deep learning de verdad —ahí sí se justifica.

Siguiente: Apunte 29 — **de modelo a producto:** cómo se despliega, se monitorea y cuándo hay que re-entrenar.

Apunte 29 — De modelo a producto: desplegar, monitorear, re-entrenar

Módulo 10 (Cierre) · Machine Learning desde cero

Un modelo en un notebook no es un producto. Para que sirva en una planta tiene que **correr solo, vigilarse y envejecer con dignidad**. Este apunte es sobre eso —y tenemos la suerte de poder mostrarlo con tus propios datos.

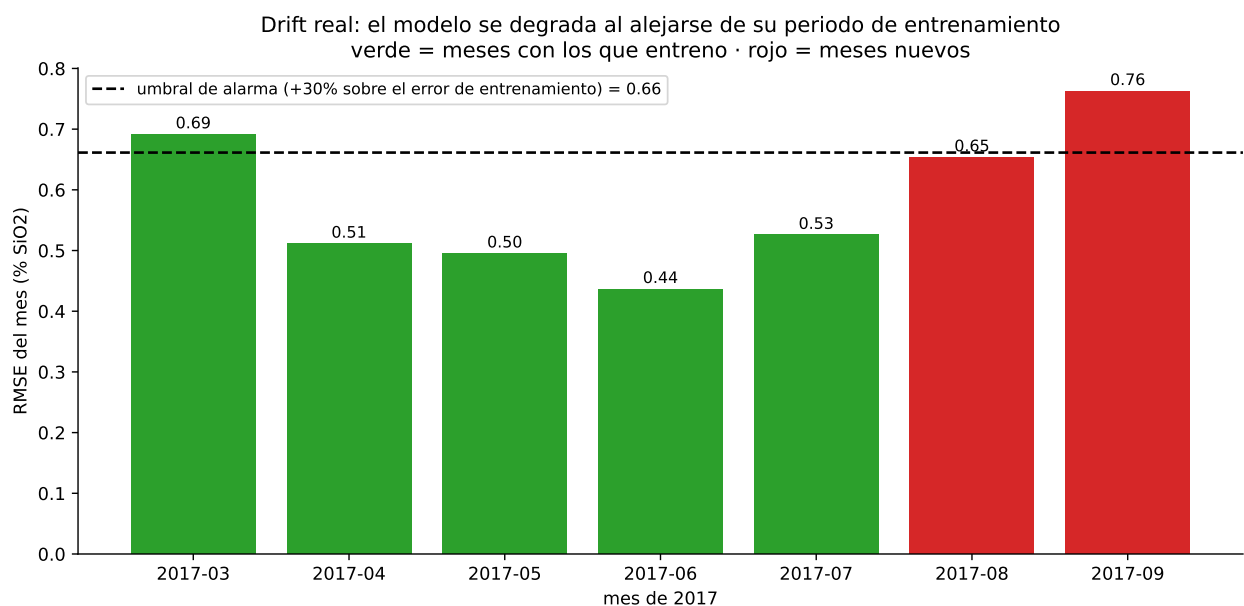
Glosario — en palabras simples

- **Despliegue (deployment):** poner el modelo a funcionar de verdad, recibiendo datos nuevos y entregando predicciones.
- **Inferencia:** cada predicción que hace ya en operación.
- **Drift (deriva):** que el mundo cambie y el modelo se vuelva obsoleto.
- **Monitoreo:** vigilar su desempeño para enterarte *antes* de que haga daño.
- **Re-entrenamiento:** volver a entrenarlo con datos recientes.
- **MLOps:** el conjunto de prácticas que sostiene todo ese ciclo.

1. No es un proyecto, es un ciclo

Entrenar → desplegar → **monitorear** → re-entrenar → desplegar... El error de novato es creer que el trabajo termina cuando el modelo alcanza buena métrica. En realidad ahí *empieza*: el modelo se degrada solo, sin avisar, porque el proceso cambia.

2. El drift, con tus datos



Concepto

Entrenamos con marzo–julio (verde) y medimos el error mes a mes. Junio y julio andan en 0,44–0,53...pero en **agosto sube a 0.65** y en **septiembre a 0.76**: el modelo empeora un 50 % respecto a su mejor mes. Si un sistema de monitoreo tuviera una alarma en “+30 % sobre el error de entrenamiento” (0,66), **septiembre la dispara**: hora de re-entrenar. Fíjate también en **marzo (0.69)**: está *dentro* del entrenamiento y aun así el error es alto, porque marzo pertenece al régimen operativo que descubrimos en el Apunte 26 y que después desapareció.

3. Qué se vigila en producción

- **Error del modelo**: lo más directo, pero necesita conocer la respuesta real. *Aquí tenemos suerte*: el laboratorio la entrega una hora después, así que podemos medir el error casi en vivo. En otros problemas la respuesta tarda meses.
- **Distribución de las entradas** (*data drift*): si el aire, el pH o la alimentación empiezan a moverse en rangos nunca vistos, el modelo está extrapolando —y eso se detecta **sin necesidad de la respuesta**.
- **Distribución de las predicciones**: si de repente predice siempre lo mismo, o valores raros, algo se rompió aguas arriba (un sensor, una unidad, un cambio de formato).

Ojo

Dos verdades incómodas. (1) **Un modelo desplegado sin monitoreo es un pasivo**, no un activo: seguirá entregando números con total seguridad mucho después de haber dejado de servir. (2) **Re-entrenar no siempre salva**: si la planta cambió de régimen (Apunte 26), re-entrenar con datos viejos no alcanza —hay que re-pensar el modelo, no solo refrescarlo.

Pruébalo

Pregunta de entrevista: “*Tu modelo lleva seis meses en producción. ¿Cómo sabes si sigue sirviendo?*”

Respuesta modelo: “Con monitoreo continuo en tres frentes: el **error** contra las etiquetas reales cuando llegan; la **distribución de las variables de entrada**, que delata deriva sin necesidad de etiquetas; y la **distribución de las predicciones**, que detecta fallas aguas arriba. Defino un umbral de degradación —por ejemplo, +30 % sobre el error de referencia— que dispara re-entrenamiento. En mi proyecto medí esa curva: el modelo cruzaba el umbral al segundo mes fuera del periodo de entrenamiento, así que el ciclo de re-entrenamiento tendría que ser mensual.”

La analogía metalúrgica: un modelo en producción es un **analizador en línea**. Nadie lo instala y se olvida: se **recalibra** contra muestras de laboratorio, se vigila su sesgo y su deriva, y si se sale de tolerancia se recalibra o se interviene. El re-entrenamiento *es* la recalibración; el monitoreo *es* el control de calidad del instrumento. Un analizador sin verificación periódica no es una medición: es una opinión con pantalla digital.

Siguiente: Apunte 30 — el **cierre**: qué aprendiste en 30 apuntes y cómo contar este proyecto en una entrevista.

Apunte 30 — Cierre: lo que aprendiste y cómo contarlo

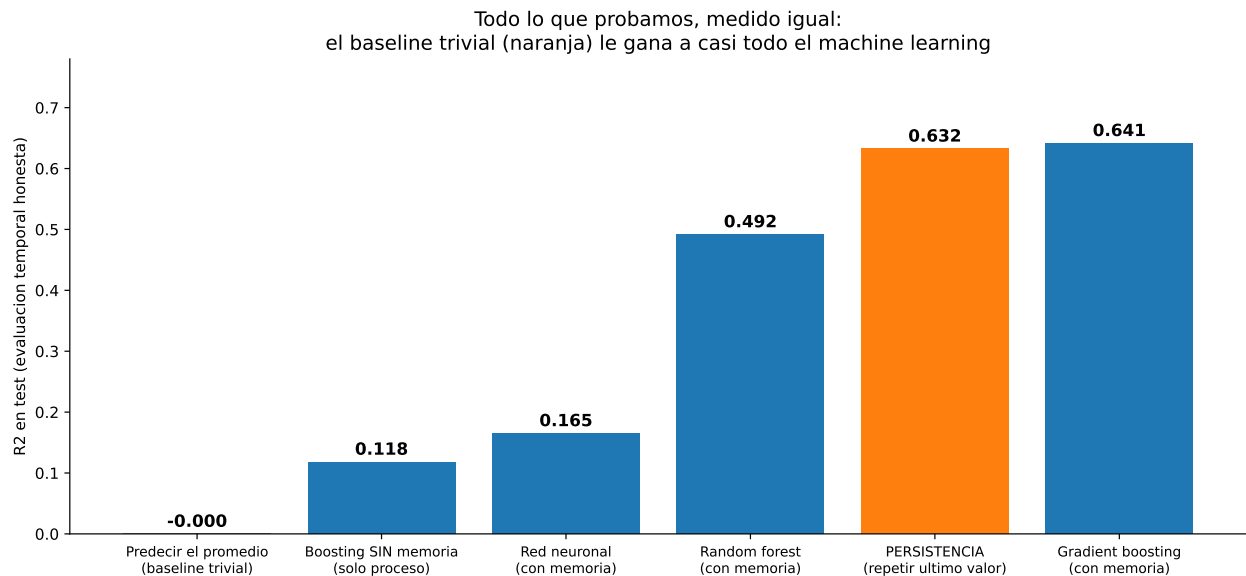
Módulo 10 · Machine Learning desde cero

Treinta apuntes atrás no sabías qué era un modelo. Hoy entrenaste árboles, bosques, boosting, una red neuronal y clustering sobre 737.453 registros reales de planta —y, más importante, aprendiste a **no engañarte** con ninguno. Este apunte cierra el círculo.

1. El recorrido

Módulo	Apuntes	Lo que te llevas
1	01–06	Explorar datos, series de tiempo, correlación, fuga de datos
2	07–12	Árbol, métricas (RMSE/MAE/R ²), sobreajuste, random forest, importancia
3	13–15	Regresión lineal, descenso de gradiente , escalado y regularización
4	16–18	Pipelines sin fuga, validación cruzada, hiperparámetros
5	19–20	Ingeniería de variables (memoria) y gradient boosting
6	21–22	Clasificación: matriz de confusión, precisión y recall, umbral
7	23–24	Importancia honesta (permutación) y SHAP
8	25	¿Dónde SÍ aporta el ML? El experimento de horizontes
9	26–27	No supervisado: k-means (estados operativos) y PCA
10	28–30	Redes neuronales, producción y drift, y este cierre

2. El resultado final, sin maquillaje



Concepto

Todos los enfoques, medidos con la *misma* vara (datos horarios, mismo corte temporal). El **baseline trivial** —repetir el último análisis de laboratorio, sin modelo alguno— logra 0,632 y **le gana a casi todo el machine learning**. Solo el gradient boosting lo supera, y por +0,009. La red neuronal, el modelo más sofisticado, queda en 0,165.

3. Cómo contar este proyecto (guion de 2 minutos)

Pruébalo

“Construí un sistema para predecir en tiempo real el % de SiO_2 del concentrado de una planta de flotación de hierro, con 737 mil registros de proceso de seis meses.

*El hallazgo más valioso fue **negativo y bien medido**. Los análisis públicos de este dataset reportan R^2 de 0.9, pero lo consiguen partiendo las filas al azar —y como cada hora tiene unas 180 filas casi idénticas, eso es **fuga de datos**. Con una evaluación temporal honesta, ningún modelo —árbol, bosque, boosting o red neuronal— supera de forma significativa al baseline trivial de repetir el último análisis: el ML aporta +0,009 de R^2 .*

*Además, con clustering no supervisado encontré la **explicación**: la planta cambió de régimen operativo a mitad del histórico, lo que degrada cualquier modelo entrenado con datos previos —y lo confirmé midiendo la deriva mes a mes.*

*Mi entregable fue una **recomendación fundamentada de no desplegar ML** para esta tarea con estos datos, más una hoja de ruta: reducir la latencia del laboratorio o instalar un analizador en línea, y medir variables que hoy no se registran.”*

Ese guion vale más que un R^2 bonito, porque demuestra **criterio**: sabes elegir el baseline correcto, detectar fuga, diagnosticar por qué falla y traducirlo a una decisión de negocio.

4. Lo que ya puedes defender

Supervisado vs no supervisado · regresión vs clasificación · train/test y validación cruzada temporal · **fuga de datos** (sus tres formas) · sobreajuste y equilibrio sesgo-varianza · árboles, random forest y gradient boosting · regresión lineal y **descenso de gradiente** · escalado, regularización y pipelines · hiperparámetros · RMSE/MAE/R² y matriz de confusión con precisión/recall · importancia por permutación y **SHAP** · k-means, codo y PCA · redes neuronales · drift y monitoreo en producción.

Ojo

Lo que de verdad te distingue no es la lista de algoritmos —esa la tiene cualquiera que hizo un curso. Es que puedes contar, con datos en la mano, **una vez que te equivocaste y cómo lo detectaste**: en el Módulo 5 celebramos haber subido el R² de 0.04 a 0.64...hasta que el Apunte 23 mostró que casi todo era inercia del proceso y el aporte real del ML era +0,009. Corregirlo por escrito, en tu propia biblioteca, es exactamente lo que hace un ingeniero serio.

5. Lo que sigue

Este proyecto cerró, pero el camino no: **deep learning de verdad** (con tu RTX 5060, en imágenes o señales, donde la GPU sí se justifica), **series de tiempo especializadas** (ARIMA, Prophet), o llevar este mismo análisis a otro proceso de tu planta. Tienes el método; cambiar de dataset ahora es solo repetir el ritual.

Fin del proyecto *Machine Learning desde cero*

30 apuntes · 10 módulos · un dataset real · cero autoengaños

Extras — fuera del temario principal

Apunte 31 (extra) — PyTorch y tu GPU: el bucle a mano

Extra · Machine Learning desde cero

En el Apunte 28 entrenamos una red con scikit-learn, que hace todo por dentro y no deja ver nada. Aquí usamos **PyTorch** —el framework que se usa de verdad en la industria— y escribimos el entrenamiento **paso a paso**. De paso, encendemos tu **RTX 5060**.

Glosario — en palabras simples

- **Tensor:** la “matriz” de PyTorch. Igual que un arreglo de números, pero puede vivir en la **GPU** y recordar cómo se calculó.
- **GPU / CUDA:** la tarjeta gráfica y el lenguaje para programarla. Tiene *miles* de núcleos pequeños que hacen la misma operación a la vez.
- **Autograd:** la magia de PyTorch: calcula solo las derivadas (los gradientes) de cualquier cálculo que hayas hecho.
- **Época (epoch):** una vuelta completa del bucle de entrenamiento.

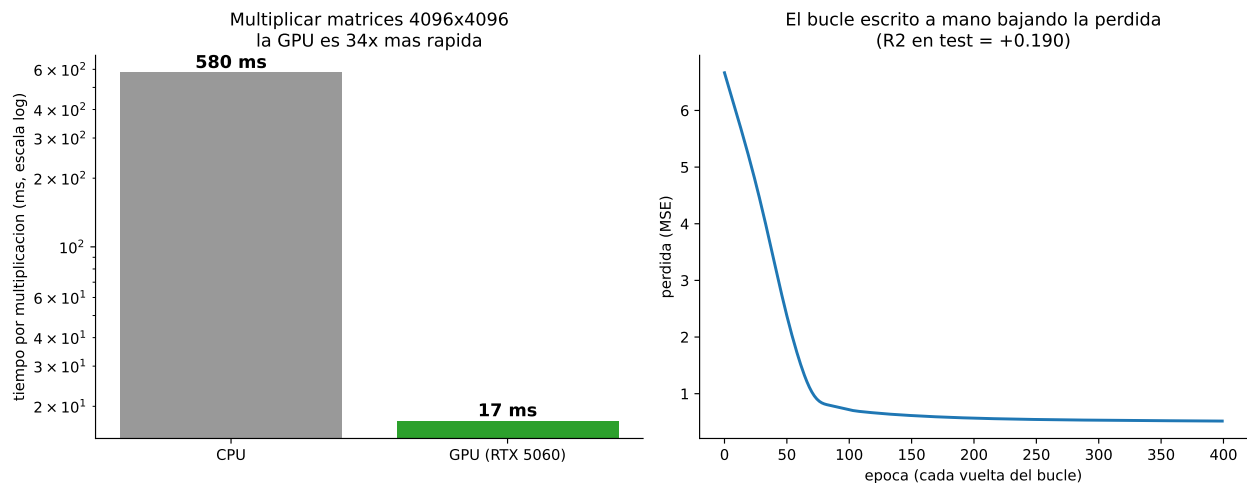
1. La odisea de instalación (esto también es la lección)

Antes de una sola línea de código nos topamos con el **infierno de versiones** del ecosistema ML, y vale la pena que lo veas porque te va a pasar siempre:

- Tu proyecto corre en **Python 3.14**...y PyTorch **no publica versiones con GPU para 3.14** (solo CPU). Hubo que instalar un **Python 3.12** aparte.
- Tu RTX 5060 es arquitectura **Blackwell (sm_120)**, tan nueva que las versiones *estables* de PyTorch todavía no la soportan. Hubo que usar una versión **nightly** (de desarrollo) compilada con CUDA 12.8.
- Se creó un entorno **separado** (`.venv-gpu`) para **no tocar** el que ya funcionaba. Regla de oro: nunca rompas lo que anda.

Y funcionó: `torch.cuda.is_available() = True`, capacidad `sm_120`. **Tu tarjeta está corriendo PyTorch.**

2. ¿Qué te compra realmente una GPU?



Concepto

Izquierda: la prueba en tu máquina. Multiplicar dos matrices de 4096×4096 (unos 137 mil millones de operaciones) tarda **580 ms en tu CPU** y **17 ms en tu RTX: 34 veces más rápido**. ¿Por qué? Tu CPU tiene unos pocos núcleos muy listos que trabajan casi en fila; la GPU tiene *miles* de núcleos simples que hacen la **misma** operación sobre datos distintos *al mismo tiempo*. Y una red neuronal es, literalmente, multiplicar matrices una y otra vez —por eso el deep learning nació cuando las GPU se volvieron accesibles.

3. El bucle de entrenamiento, por fin visible

Esto es lo que scikit-learn te escondía. Cinco líneas que **son** el Apunte 14:

Línea	Qué hace
<code>optimizer.zero_grad()</code>	borra los gradientes de la vuelta anterior
<code>pred = model(X)</code>	forward : predice con los pesos actuales
<code>loss = loss_fn(pred, y)</code>	mide el error
<code>loss.backward()</code>	backward : autograd calcula todos los gradientes
<code>optimizer.step()</code>	da el paso cuesta abajo

Se repite 400 veces y la pérdida cae de 6,67 a 0,52 —la curva de la derecha en la figura. **Eso es entrenar una red neuronal**. No hay más misterio: es el descenso de gradiente del Apunte 14, escrito en cinco líneas y ejecutado en tu GPU.

Ojo

Y el resultado honesto: $R^2 = 0,190$ en test. Apenas mejor que el MLP de scikit-learn (0,165) y muy por debajo del gradient boosting (0,641) —y por debajo incluso de **no hacer nada** (persistencia, 0,632). **La GPU acelera el entrenamiento; no mejora el modelo.** Entrenar 34 veces más rápido un modelo que no es el adecuado para tus datos solo te da la respuesta equivocada más pronto.

Pruébalo

Pregunta de entrevista: “¿Cuándo necesitas GPU y qué te aporta?”

Respuesta modelo: “La GPU acelera operaciones matriciales masivas en paralelo, que es justo lo que hace una red neuronal. La necesito cuando el modelo y los datos son grandes —visión, lenguaje, señales— donde entrenar en CPU tomaría días. Para datos tabulares pequeños, como mi proyecto, no aporta: el cuello de botella no es la velocidad sino la señal disponible. Lo medí: en mi RTX una multiplicación de matrices fue 34 veces más rápida que en CPU, pero el R^2 del modelo no mejoró. Velocidad y calidad son cosas distintas.”

La analogía metalúrgica: la GPU es un **molino SAG de gran capacidad**. Procesa muchísimo más tonelaje por hora que un molino chico —enorme ventaja cuando *tienes* ese tonelaje. Pero moler más rápido **no sube la ley del mineral**: si la alimentación no trae valor, lo único que consigues es producir concentrado pobre más rápido. Eso es exactamente lo que pasó aquí.

Cierre. Ya tienes el entorno GPU montado y funcionando. Cuando entres a deep learning de verdad —imágenes, señales, lenguaje— tu RTX 5060 va a brillar. Este apunte te deja la puerta abierta *y* el criterio para saber cuándo cruzarla.